

Automation of design process with Python

Automatyzacja procesów projektowania z wykorzystaniem Pythona

Stanisław Gepner

Projekt „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca” współfinansowany jest ze środków Unii Europejskiej w ramach Europejskiego Funduszu Społecznego

Zadanie 35 pn. „Modyfikacja programu studiów II stopnia na kierunku Robotyka i Automatyka” realizowane w ramach projektu „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca” współfinansowanego jest ze środków Unii Europejskiej w ramach Europejskiego Funduszu Społecznego

1 Rules and conditions of passing - subject organization

If you running this on <https://mybinder.org/> and see this, this probably works. In case you do not see this, probably it does not.

Person responsible: **Stanisław Gepner**

- Lecture: 15 ()
- Laboratory (computer): 15
- Additional consultations: 10
- Self study and project: 10

Goals: * Writing scripts and programs in Python * Automating the design loop. Application to computations. * Processing data using available tools

What students should know, prerequisites: * Be able to write elementary program in C/C++ using procedural programing, loops and conditional statements.

Literature: * Svein LingeHans Petter Langtangen, Programming for Computations - Python, freely available from Springer:

<https://link.springer.com/book/10.1007/978-3-319-32428-9> * Any on-line resource you can find “and gets things working” for you. * This lecture and laboratories and interactive notes available on GitHub repo.

Rules and conditions of passing: * To be communicated during first class

1.1 What will you need

- A computer with a decent modern OS. In case you use something else, like Windows or the fruity one, you can probably get things working, but no guarantees.
- On Windows you can either install Python interpreter or make due with Windows Subsystem for Linux, an official Microsoft way of making their system less not-decent.
- Python interpreter, we will be using Python 3.
- You can also run all the lectures on <https://mybinder.org/>, so in fact you need nothing but a browser.

Lecture 1 - elementary use of Python

Stanisław Gepner

Projekt „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca” współfinansowany jest ze środków Unii Europejskiej w ramach Europejskiego Funduszu Społecznego

Zadanie 35 pn. „Modyfikacja programu studiów II stopnia na kierunku Robotyka i Automatyka” realizowane w ramach projektu „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca” współfinansowanego jest ze środków Unii Europejskiej w ramach Europejskiego Funduszu Społecznego

1 Introduction

We will: * Look at some basic differences of Python with regard to what you know * Show basic syntax * Discuss dynamic typing, lack of compiler and its consequences * Recall loop, conditional statements, functions, etc.

Through the course of your studies you must have committed some programming. This might have been a simple (or not so much) program in C, C++ or maybe Matlab or Bash? One thing that you might have noted is that in the case of **C** family a compiler was needed to build your program into a usable, machine code, binary executable. In spite of your (bad?) experiences with the compilation process and sorting out errors, the compiler was your best friend. It limited the number of errors, you might have committed that would have otherwise manifested in the working of your program. Some, you might have not even noticed.

On the other hand, there is a number of programming languages that do not require compilation, and use an interpretation engine to perform programmed tasks. Matlab, the popular in academia, suite is one of the possibilities. Here we will be looking at Python, a **dynamically typed** and **interpreted** scripting language that has by now become very popular in scientific, engineering but also in standard development applications.

A Python program requires an interpreter to be run. An interpreter is a program that directly executes provided instructions, without compiling them into the machine code. This has some benefits, as you have no compiler errors, and might lead to a speedy development of the code. But also has some consequences that might prevent you from applying your interpreted code for complicated tasks. One very interesting application of interpreted programming languages, such as Python is in joining work-flow of otherwise separate tools, very much as the gray duct tape is used to connect various elements that have not been previously designed to be connected. Yes Python might be the gray tape that holds things together. During this class I want to share with you my experience of applying Python in such a role.

- Advantages of an interpreted language:
 - Speedy development
 - **No compiler errors**
 - ...
- Disadvantages
 - **No compiler errors**
 - No strict type definition
 - ...

Yes not having a compiler might not be all that nice after all, since compiler can save you from some nasty bugs.

1.1 Basic syntax

We will start by getting familiar with very basic structure of a Python script/program. We will avoid to differentiate the two, and use script and program interchangeably.

A note before we start: With Python almost any task can be realized in more than one way. There is no best approach and we will try to focus on simplest, most comprehensible solutions here, gradually increasing complexity.

1.2 First script

Coding time!: examine script1.py and run it

We start with the most popular first contact program, the famous “Hello World!” implemented in Python. Using your Python interpreter try to run the script1.py from this repo.

Note: we use function `print()` provided by the system, that simply puts data onto the screen.

1.3 Working with Jupyter notebooks

Writing self contained scripts and running them from the command line, or the interpreter engine is one way, but it is usually more convenient to work with the program in an interactive way. In our lectures we will use Jupyter notebook to do this. There also exist other ways to interactively work with Python code.

Our “Hello WOrld!” program looks like this:

```
[1]: print("Hello!")
```

Hello!

And can be run either using the top (or side) bar “Run” button or either ctrl-enter or shift-enter keys.

Coding time! - practice the use of `print()`. Try to test different formating possibilities, line endings and separations. See the list below.

Lets now see what we can do with a `print()` function, as it offers many possibilities for formating the output.

- `print()` simple stuff
- C style - kind of - octal and hex notation works
- different line ends, `\n`, `\r`, `,`, `...`
- different text separation `sep=" sep "`
- examples of formating output with `str.format()` e.g.: `'{0: >5}'.format(24)`
 - aligned with `<>` also other possibilities

```
[2]: print("This is a text %8d with values %8.2f" % (24, 5))
```

This is a text 24 with values 5.00

```
[3]: print("This is sparta!!!", "Ala ma kota", end="\r", sep="| |")  
# print("aaaaaaaaaaaaaaaaaa", end="\r")
```

This is sparta!!! |Ala ma kota

```
[4]: print('{: >10o}'.format(24))
```

30

Illustrate how to work with Jupyter.

1.4 Dynamic types

Variables declared for use have no explicit type. This is different in comparison to what you might have seen in **C** where variable types had to be provided. In Python variables are **dynamically typed**. This means that type is decided on run-time based on the variable value. This has benefits and allows for quick changes to be made to the code without the need to recompile the program. But at the same time might lead to errors that might have been avoided if type was strictly defined.

```
[5]: a = 10
```

Here we have created a new variable, an integer. The type has been decided based on the value provided. Let us have some more variables:

```
[6]: b = 'this is a string'
     c = "and so is this"
     d = 5.0 # and this is a float
```

Note: # - allows to make single line comments

Some operations on variables. We can perform basic arithmetic operations, such as +, -, *, / %.

The resulting type will be decided on run-time.

```
[7]: a = 10
     b = 5
     c = a+b
     print("c =",c)
```

c = 15

But:

```
[8]: b = 'aaa'
     c = a + b
     ???
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In [8], line 2
      1 b = 'aaa'
----> 2 c = a + b

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Coding time!: Try out with different build in types. Declare some variables and initialize them with values. Perform basic arithmetic operations on **ints**, **floats** and others. Use **print** to see the values, and **type()** to examine the type of variables. Is all working as it should, and are you always sure what type a variable is?

1.5 Functions

What would procedural programming be without the ability to write functions. In python functions are defined in the following way:

```
[9]: def fun(a):  
      return a
```

So a keyword **def**, a name and a list of arguments. but what is the type of a and what does the function return?

Well, **dynamic typing**, so a is anything, and the function returns whatever has been provided to it.

```
[10]: fun(a+c)
```

```
[10]: 25
```

Note: passing values to function can be done by name

Note 2: Default argument value

1.6 Structure of the code - indentation as a syntax requirement

In Python indentation of the code is an element of syntax. So you are forced to write your code properly formatted for it to work. Note there is no scope limited by {}, as it was in **C**, but rather code formatting defines the scope of a function, but also loops, ifs etc. So this will not work:

```
[11]: def fun(a):  
      return a
```

```
Cell In [11], line 2  
    return a  
    ^  
IndentationError: expected an indented block after function definition on line 1
```

Because the code is ill formatted! Write your code neatly!

Coding time! Write your own function. It should accept two or more arguments, and return the result of summation.

1.7 Loops and conditional statements

Looping and branching code execution are indispensable in any programming task. Let us start with an **if**:

1.7.1 if, else if, etc.

```
[12]: if a!=0:
        print('zero!')
    elif a != 1:
        print("bla")
    else:
        print('not zero!')
```

zero!

1.7.2 More than one condition

Logical values are slightly different in Python than in C. Logical AND is realized with & and OR with |: For example:

```
[13]: a = 10
```

```
[14]: if a>-1 & a!=0:
        print(a)
```

```
[15]: if a>-1 | a!=10:
        print(a)
```

10

Will not work exactly as you expect. Logical statements need to be additionally enclosed in ()-brackets, like this:

```
[16]: if (a>-1) & (a!=0):
        print(a)
```

10

```
[17]: if (a>-1) | (a!=10):
        print(a)
```

10

Note:, you can also perform logical operations with **and** and **or**!:

```
[18]: if a>-1 and a!=0 or a < 0:
        print(a)
```

10

1.7.3 pass statement

In C scope was determined by {}-brackets, and it was very easy to have an empty statement or a function. In Python this is not possible since any statement needs to be followed by an intended

code. Without indentation the code will simply not work. To circumvent this problem Python uses **pass** statement. For example:

```
[19]: a = 10
      if a>0:
          pass # You shall not pass!
      else:
          print("Shell I pass?")
```

Coding time!: Develop a small code snippet and test various ways to perform conditional statements. Use **if**, **elif** and **else**, remember about indentations and **:** so your code works properly. For example Write a function that accepts a single variable, and test if it is odd or even and prints a proper message to the screen.

1.8 Lists

List is the default Python way to do collections of data. It is easy to create, use and remove. But, as we will see it is not always (actually most often) the best choice. To create a list, use **[]**=brackets, or using **list()**.

Note: Elements of a list can be accessed with **[]** bracket and an index (starting from 0).

```
[20]: a = []
      b = list()
```

or create one that contains some data, by providing it at invocation:

```
[21]: a = [0, 1, 2, 3]
      b = ['a', 'b', 3, 4]
```

We note that **b** contains values of different type!
Now let's use **print()**:

```
[22]: print(a)
      print(b)
      print(a+b)###
      print(a[3])
```

```
[0, 1, 2, 3]
['a', 'b', 3, 4]
[0, 1, 2, 3, 'a', 'b', 3, 4]
3
```

Some methods of a list and examples: * **append** * **insert** * **reverse** * **len** * **in** * slice of a list with **[5]** * segment with **[10:100:5]** - from 10th to 100th every 5 * the **-1** index * **sort()**

Coding time!: Write a program in which you will create two lists that contain some data (be creative!). Examine content of created lists, and try access elements stored with **[]**, use **append()**, **insert()** and **len()** and examine the effects. Try to test if lists you work with contain data, use **in** expression to do that. Write a piece of code that will make the element in the list unique using

if, not, in and append(). Use sort() to sort the resulting list. Than use reverse() to change the order.

Practice slice extraction from lists with the [1:n] semantics. Finally get familiar with accesing elements in a reversed order with the use of negative indices (e.g. [-1]).

1.8.1 Multidimensional lists

Or a list in a list

```
[23]: a = [[1,2], [3,4]]
      print(a[1][0])
```

3

Coding time! Create a multidimensional array, representing a 3x3 matrix. Write a function that prints this matrix on the screen. Try to apply formatting to the output. Use known modifications to the print() function.

1.9 Loops

Loops do not differ much from what you might know from C, no need for brackets and a need to use indentation are the two things that are different. Onother difference is that instead of using a counter, that we know from for loops in C, here we will access elements directly with an in statment. Let's see:

```
[24]: a.append("hello!")
      for i in a:
          print(i)
```

```
[1, 2]
[3, 4]
hello!
```

Allowed us to iterate across all elements of a list. now let's look at some features: * for - see above * in range(a,b) - range is an object! that might give values from a, to b * in list - used to iterate through all elemensts of a list * break and continue - do the same as in C, i.e. control the loop execution * enumerate - provides with a tuple that contains an indx and a value from the list * zip - allows to combine iterations to be performed through a number of collections

```
[25]: for i,v in enumerate(a):
      if i % 3 == 0:
          continue
      print(i, type(i), v, a[i])
```

```
1 <class 'int'> [3, 4] [3, 4]
2 <class 'int'> hello! hello!
```

```
[26]: for i in range(0,10):
      print(i)
```

0
1
2
3
4
5
6
7
8
9

```
[27]: a = [1,2,3,4,5,6]
      b = ['a', 'b', 'c', 'd', 'e', 'f']
      for val in zip(a,b):
          print(val, type(val))
      for va, vb in zip(a,b):
          print(va, type(va))
          print(vb, type(vb))
```

```
(1, 'a') <class 'tuple'>
(2, 'b') <class 'tuple'>
(3, 'c') <class 'tuple'>
(4, 'd') <class 'tuple'>
(5, 'e') <class 'tuple'>
(6, 'f') <class 'tuple'>
1 <class 'int'>
a <class 'str'>
2 <class 'int'>
b <class 'str'>
3 <class 'int'>
c <class 'str'>
4 <class 'int'>
d <class 'str'>
5 <class 'int'>
e <class 'str'>
6 <class 'int'>
f <class 'str'>
```

Coding time!: Using a `for` and `range()` create two lists. The first should contain odd numbers from 0 to 100 and the second even ones. Use `zip` to iterate through both lists and print values that are the same. Add `enumerate()` to print also the index.

1.10 Advanced initialization

Python has its more advanced ways of doing things. Some operations that would normally need to be implemented using a number of lines of code can be done in a single line. Consider a list of integers:

```
[28]: a = []
      for i in range(0,100):
          a.append(i)
      print(a)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21,
22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41,
42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61,
62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81,
82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99]
```

Now let's have a list that contains only even values from *a* (or some other operation on the elements of *a*). We could:

```
[29]: b = []
      for i in a:
          if i%2 == 0:
              b.append(i)
      print(b)
```

```
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40,
42, 44, 46, 48, 50, 52, 54, 56, 58, 60, 62, 64, 66, 68, 70, 72, 74, 76, 78, 80,
82, 84, 86, 88, 90, 92, 94, 96, 98]
```

But also, with Python we could do this like this:

```
[30]: c = [ i for i in a if i%2==0]
      print(c)
```

```
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40,
42, 44, 46, 48, 50, 52, 54, 56, 58, 60, 62, 64, 66, 68, 70, 72, 74, 76, 78, 80,
82, 84, 86, 88, 90, 92, 94, 96, 98]
```

More fun, right?

Lets try something else:

```
[31]: c = [ i*i for i in range(0,10)]
      print(list(range(0,10)))
      print(c)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Coding time!: Using the fancy, single line initialization write a three line code that produces three lists. The first contains even numbers from 0 to 100, the second odd ones and the third only the values that are common to first two lists. The third list should be created based on the content of initial lists, and not on what you know about a result.

1.11 Tuples

A bit different to list type of collection is a tuple. Anything can be put into a tuple. Tuples can not be modified once created. To make one use ()-brackets:

```
[32]: t = (a,b)
      print(t)
```

```
([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21,
22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41,
42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61,
62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81,
82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99], [0, 2,
4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44,
46, 48, 50, 52, 54, 56, 58, 60, 62, 64, 66, 68, 70, 72, 74, 76, 78, 80, 82, 84,
86, 88, 90, 92, 94, 96, 98])
```

Elements of a tuple are accessed with []-brackets:

```
[33]: print(t[0])
      print(t[1])
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21,
22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41,
42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61,
62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81,
82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99]
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40,
42, 44, 46, 48, 50, 52, 54, 56, 58, 60, 62, 64, 66, 68, 70, 72, 74, 76, 78, 80,
82, 84, 86, 88, 90, 92, 94, 96, 98]
```

And modification attempts will fail:

```
[34]: t[0] = 5
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In [34], line 1
----> 1 t[0] = 5

TypeError: 'tuple' object does not support item assignment
```

We might find tuples a convenient way to return more than one result from functions. Something that in C required passing values through pointers. For example:

```
[35]: def fun():
      '''
      In this function a lot happens,
      things get calculated and in the end I need to return
```

```

a list, an average and something else
'''
a = [ i*i for i in range(0,10)]
s = sum(a) # a sum!
# cast to float, just in case
avg = float(s) / float(len(a));

return a, ("s=", s), avg !!! a tuple in the return

```

```

[36]: res = fun()
      for r in res:
          print(r)

```

```

[1, 1, 4, 27, 256, 3125, 46656, 823543, 16777216, 387420489]
('s=', 405071318)
40507131.8

```

Coding time! Examine tuples, create some and try to manipulate their content. Can you do it? Write a function that will be provided with a list of values. This function should sort the list and return a tuple that contains the largest and smallest value in the list. Use this function, and examine if the list that you pass to that function remains unchanged.

1.12 Modules

Any file containing Python code can be reused as a module. The code in those files can be accessed and used. At the same time when using Python we gain access to a vast number of external modules and software packages written by others. In order to use them we need to signal this to the interpreter. Much as was the case with include preprocessor statement in **C**. The counterpart of the **C** include is **import**, let's try:

Note: there is a *function.py* file that contains function `fun`. When working with the interactive Python interpreter such as this Jupyter notebook we have access to all the system calls. For those familiar with Linux OS, we will list the content of the file:

```

[37]: ls

```

```

function.py  'Lecture 1.ipynb'  __pycache__ /  script1.py

```

```

[38]: cat function.py

```

```

def fun():
    print("Hello!!")

```

and import it for use:

```

[39]: import function
      function.fun()

```

```

Hello!!

```

To access code implemented in *function.py* we need to use the dot: `.`-operator, this works very much like **namespace** you might know from **C++** and allows to separate the code, so our *fun()* and *fun()* from *function.py* remain separated

```
[40]: fun()
```

```
[40]: ([1, 1, 4, 27, 256, 3125, 46656, 823543, 16777216, 387420489],  
      ('s=', 405071318),  
      40507131.8)
```

Import can also be performed in a different way:

```
[41]: from function import fun # explicitly list what should be imported
```

```
[42]: from function import * #import everything
```

```
[43]: import function as ff # import and assign an alias
```

now, *fun()* from *function.py* overrides our local *fun()*

```
[44]: fun()
```

Hello!!

But also we can:

```
[45]: ff.fun()
```

Hello!!

Coding time!: Move the function from the previous excersies to a separate Python file. Import it here using different approaches and examine how it works for you.

There are many modules that allow to perform various programing tasks. Some are more popular than others. During this class we will work with some. An incomplete list: * NumPy - numerical mathematics library, with functions, arrays, etc. * SciPy - algorithms for optimization, integration, etc. * Matplotlib - plotting and visualization library * And many others

**Projekt „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca”
współfinansowany jest ze środków Unii Europejskiej w ramach Europejskiego
Funduszu Społecznego**

**Zadanie 35 pn. „Modyfikacja programu studiów II stopnia na kierunku
Robotyka i Automatyka” realizowane w ramach projektu „NERW 2 PW. Nauka
– Edukacja – Rozwój – Współpraca” współfinansowanego jest ze środków Unii
Europejskiej w ramach Europejskiego Funduszu Społecznego**

Lecture 2 - class and object with Python

Stanisław Gepner

Automation of design process with Python

Projekt „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca” współfinansowany jest ze środków Unii Europejskiej w ramach Europejskiego Funduszu Społecznego

Zadanie 35 pn. „Modyfikacja programu studiów II stopnia na kierunku Robotyka i Automatyka” realizowane w ramach projektu „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca” współfinansowanego jest ze środków Unii Europejskiej w ramach Europejskiego Funduszu Społecznego

1 Introduction

Procedural and object oriented paradigms of programming.

Procedural programming centers around developing procedures (functions) and passing data in between while performing programmed tasks. So in a sense data and functionality remain separated. With Object Oriented Programming (OOP) we create objects that merge data and functionality together, which should in principle allow for better code organization, limit code redundancy and give a more real life relation between program parts.

Object Oriented Programming (OOP) in Python is based on a couple of concepts, we will now discuss:

* Class and object * Methods and attributes * Encapsulation * Inheritance and polymorphism

Note: OOP, allows to do some things in a more natural and understandable manner, but there should be limits to the excessive use. What I mean, it is good when it is good, and sometimes too much is just too much.

1.1 Class and object:

Class is a recipe for creating objects. It defines what object has (data) and what it can do (methods). In Python creating a class is very simple:

```
[1]: class Cat:
      pass
```

We defined a class *Cat*, it has nothing (*pass*) for now. We can create a number of variables of type *Cat*:

```
[2]: a = Cat()
      b = []
      b.append(Cat())
      b.append(a)
      print(a)
      print(b)
```

```
<__main__.Cat object at 0x7fdb04f30>
[<__main__.Cat object at 0x7fdb04f30>, <__main__.Cat object at
0x7fdb04f30>]
```

We created two *Cats* and stored them in a list *b*. Note that *b[0]* is the same as *a*. Although right now our *Cat* class has no attributes, we might add those on the run. Which in a long run might not be such a good idea, but is possible, due to dynamic typing paradigm:

```
[3]: a.b = 10 # now object a of type Cat has an attribute b!
```

Still, this will not work:

```
[4]: print(b[0].b)
```

```
-----
AttributeError
```

```
Traceback (most recent call last)
```

```
Cell In [4], line 1
```

```
----> 1 print(b[0].b)
```

```
AttributeError: 'Cat' object has no attribute 'b'
```

But this will (Why?):

```
[5]: print(b[1].b)
```

10

We will return to the “additional” attributes in a second to illustrate possible problems.

Coding time!: In a separate file define a class `Animal`. Import the module here and create a list to store `Animal` objects.

1.2 Methods and attributes

Classes define data an object of the class has. Let expand the `Cat` so it has a `breed` attribute:

```
[6]: class Cat:
      breed = ''
```

```
[7]: cat1 = Cat()
      cat1.breed = 'Sjam'
      cat2 = Cat()
      cat2.breed = 'Persian'
      # Let have cats in a cat list
      clist = [cat1, cat2]
```

```
[8]: for c in clist:
      print(c.breed)
```

Sjam

Persian

Coding time!: Extend `Animal` to include taxonomic information stored as a string. For now just phylum (typ). Create some objects of different phylum (check Wiki for possible choices).

1.2.1 self

We will now add methods to perform some operation on the `Cat` object. Before we do we need to mention the **self** parameter. **self** is used to refer to the attributes of the object of the class, and is similar to the **C++ this** pointer. It is also used to define that a procedure is a method of the class, by being the first argument.

```
[9]: class Cat:
      breed = ''

      def Print(self): # Note self as an argument!
          print('This cat is a', self.breed)
```

Since we redefined the Cat class, we need to recreate the objects (check if the previous ones have method `Print()`):

```
[10]: cat1 = Cat()
      cat1.breed = 'Sjam'
      cat2 = Cat()
      cat2.breed = 'Persian'
      # Let have cats in a cat list
      clist = [cat1, cat2]
```

```
[11]: for c in clist:
      c.Print()
```

This cat is a Sjam
This cat is a Persian

Coding time!: Extend the Animal class to poses a method that returns the number of letters in the phylum (need a better idea). Test this method here.

1.2.2 Going back to additional attributes:

Let's see some consequences of the additional attribute:

```
[12]: class Cat:
      def fun(self):
          print(self.b) # this Cat has no b!
```

```
[13]: a = Cat()
      a.fun()
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In [13], line 2
      1 a = Cat()
----> 2 a.fun()

Cell In [12], line 3, in Cat.fun(self)
      2 def fun(self):
----> 3     print(self.b)

AttributeError: 'Cat' object has no attribute 'b'
```

```
[14]: a.b = 0
      print(a.b)
      # but:
      a.fun()
```

0
0

1.2.3 init method

Is used to define a way to construct an object of the class. It is run the moment you create an object and can be used to create one based on some parameters. Our new cat will look like this:

```
[15]: class Cat:
        def __init__(self, b, c, a):
            self.breed = b
            self.color = c
            self.age = a
        def Print(self):
            print('This', self.breed, 'cat is', self.color, 'and is', self.age,
                  '→years old')
```

```
[16]: a = Cat('Sjam', 'black', 10)
        a.Print()
```

This Sjam cat is black and is 10 years old

Coding time!: Add `init(self, ...)` method to the Animal. Set phylum and test it here by creating some Animal objects.

1.3 Encapsulation or hermetization

One of the crucial concepts in OPP is encapsulation or hermetization. That is ability to limit access to some of the object's attributes and making them unavailable for modification or even reading. Various languages offer different levels at which such restriction is possible (e.g. C++ has a sophisticated suite of possibilities). In Python this is limited to a naming convention. Any attribute that starts with `__` (two low bars?) is restricted. Let's see an example of a Cat's bowl, that we are going to make unavailable:

```
[17]: class Cat:
        def __init__(self):
            self.__bowl = 'empty' # cats have bowls with food
        def FillBowl(self):
            self.__bowl = "Filled with food"
        def Eat(self):
            self.__bowl = 'empty'
        def Print(self):
            print(self.__bowl)
            cathappy = "Cat is happy" if self.__bowl != 'empty' else "Cat is sad"
            print(cathappy)
```

```
[18]: a = Cat()
        a.FillBowl()
```

```
[19]: print(a.__bowl)
```

```
-----  
AttributeError                                Traceback (most recent call last)  
Cell In [19], line 1  
----> 1 print(a.__bowl)  
  
AttributeError: 'Cat' object has no attribute '__bowl'
```

```
[20]: a.__bowl = 'empty'
```

```
[21]: a.Print()
```

```
Filled with food  
Cat is happy
```

```
[22]: a.Eat()  
      a.Print()
```

```
empty  
Cat is sad
```

We note that we can not access Cat's bowl for reading (AttributeError), and if we try to assign to it it has no effect on the attribute.

Coding time!: Extend the Animal class to contain a private member variable. This variable could be a string describing if the animal exists presently or is extinct. Add Methods, to read (get) write (set) this variable, and test those here.

1.4 Inheritance and polymorphism

Inheritance is central to OOP. The concept is based on extending functionality of existing classes by deriving from them new ones, that conceptually perform similar, but more specific tasks. So inheritance introduces hierarchy to classes from abstract concepts (such as a general shape in geometry) down to very specific ones (for shapes, a square or a triangle, etc.).

The class that is more abstract, from which we derive is called the **base** class (general shape), and the less abstract class formed via inheritance is the **derived** one (a square is less general than a shape, so it could be derived from it).

Concept of inheritance allows for: * Less code, since code is reused * Possibly more order in the code * Polymorphism, i.e. handling objects in a unified way

Let us present the concept with

```
Mammal<-Dog  
      <-Cat
```

Note: Show step-by-step, not all at once:

```
[23]: class Mammal():
        def __init__(self):
            self.drinks_milk = 'Yes'
class Cat(Mammal):
    pass
class Dog(Mammal):
    pass
```

Finally we get something like this:

```
[24]: class Mammal():
        def __init__(self, n):
            self.name = n
            self.drinks_milk = "Yes"
        def make_noise(self): # all derived classes make noise
            print(self.noise)
        def print_name(self):
            print('Has no name')

class Cat(Mammal):
    def __init__(self):
        self.noise = "Meow"
        Mammal.__init__(self, "Cat")
    def print_name(self):
        print('Behemoth')

class Dog(Mammal):
    def __init__(self):
        self.noise = "Bark"
        Mammal.__init__(self, "Cat")
    def print_name(self):
        print('Cerberus')
```

```
[25]: m = Mammal('')
print(m.drinks_milk)
m.print_name()
m.make_noise()
```

Yes

Has no name

```
-----
AttributeError                                Traceback (most recent call last)
Cell In [25], line 4
      2 print(m.drinks_milk)
      3 m.print_name()
----> 4 m.make_noise()
```

```
Cell In [24], line 6, in Mammal.make_noise(self)
      5 def make_noise(self): # all derived classes make noise
----> 6     print(self.noise)
```

```
AttributeError: 'Mammal' object has no attribute 'noise'
```

```
[26]: c = Cat()
      print(c.drinks_milk)
      c.print_name()
      c.make_noise()
```

Yes
Behemoth
Meow

```
[27]: d = Dog()
      print(c.drinks_milk)
      d.print_name()
      d.make_noise()
```

Yes
Cerberus
Bark

Coding time! Extend the Animal class to be base for Cat and Dog via Mammal class. Add Reptiles to go along with mammals and some derived classes that inherit from a reptile. In the end you should be able to create objects of 4 different classes.

1.4.1 Some operations on lists

Since we are now able to create objects and connect data to procedures, we will try to experiment with operations on lists that concern lists of objects:

- filter()
- sort with lambda
-

```
[28]: numbers = [1, 2, 3, 4]
      a = filter(lambda l: l < 3, numbers)
```

```
[29]: print(list(a))
```

[1, 2]

**Projekt „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca”
współfinansowany jest ze środków Unii Europejskiej w ramach Europejskiego
Funduszu Społecznego**

**Zadanie 35 pn. „Modyfikacja programu studiów II stopnia na kierunku
Robotyka i Automatyka” realizowane w ramach projektu „NERW 2 PW. Nauka
– Edukacja – Rozwój – Współpraca” współfinansowanego jest ze środków Unii
Europejskiej w ramach Europejskiego Funduszu Społecznego**

Lecture 3 - Monitoring the process

Stanisław Gepner

Automation of design process with Python

Projekt „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca” współfinansowany jest ze środków Unii Europejskiej w ramach Europejskiego Funduszu Społecznego

Zadanie 35 pn. „Modyfikacja programu studiów II stopnia na kierunku Robotyka i Automatyka” realizowane w ramach projektu „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca” współfinansowanego jest ze środków Unii Europejskiej w ramach Europejskiego Funduszu Społecznego

1 Lecture 3

Monitoring solution process is important. While looking at numbers stored in a text file is fun, it is much easier to comprehend data using graphs and plots.

We will: * Look at a (possibly) better way than lists to handle sequences of numerical data - NumPy and arrays * Look at reading numerical data from files (stored as columns of numbers, but other formats can also be read) * Use the Matplotlib library to produce some simple line, bar and color plots.

- Have a look at SciPy to illustrate capabilities, find some function extrema and roots, perform some interpolation (and extrapolation).

1.1 NumPy and arrays

We will now work with NumPy and SciPy and introduce selected features that might be useful to us. By no means we will cover all features, and when in need please consult appropriate (and very rich) online documentation.

1.1.1 NumPy

NumPy is a numerical method library that implements a number of functionalities. The most important to us is going to be the *array*. While using *list* is fine there is a cost associated with the fact that list can hold anything (actually mixing types). This has a consequence that any operation needs first to verify the type, making lists not so efficient, especially for larger problems. On the other hand NumPy arrays store data of only the declared type and have a constant size. Also much of NumPy functions are calls to compiled binary code (we will be writing such code!).

Let's start by importing arrays from NumPy:

```
[2]: import numpy as np
```

We will refer to elements of NumPy using *np* (less writing). Our first array will be a 1D vector of integers:

```
[3]: a = np.array([1,2,3,4])
      print(a)
```

```
[1 2 3 4]
```

Coding time!: Experiment with arrays. Make and print some. Try passing lists in lists.

```
[4]: print(a[2])
      print(type(a), type(a[3]))
```

```
3
<class 'numpy.ndarray'> <class 'numpy.int64'>
```

1.1.2 Creating arrays

There is a number of ways arrays can be created with `numpy.array()` function. The easiest is to use an existing list, or use the advanced list initialization with `[]`-brackets. There is also a number of

special *numpy* functions to create arrays. Let's see.

- Array from list:

```
[5]: l = list(range(0,10))
      print(l)
      a = np.array(l)
      print(a)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[0 1 2 3 4 5 6 7 8 9]
```

```
[6]: a = np.array([x for x in l if x%2])
      print(a)
      a = np.array([ 2*x+3 for x in l ])
      print(a)
```

```
[1 3 5 7 9]
[ 3  5  7  9 11 13 15 17 19 21]
```

Coding time! Create some arrays using the advanced syntax as above. E.g. use **range** to create an array of 50 values from -10 to 10 and another one that is the square of the first one.

fun, right? * From other array, by simple arithmetic operation:

```
[7]: x = []
      x.append(-10)
      h = 20 / 49.
      for r in range(0,50):
          x.append(x[-1] + h)
      x = np.array(x)
      y = [yy**2 for yy in x]
      print(x)
      print(y)
```

```
[-10.          -9.59183673  -9.18367347  -8.7755102   -8.36734694
 -7.95918367  -7.55102041  -7.14285714  -6.73469388  -6.32653061
 -5.91836735  -5.51020408  -5.10204082  -4.69387755  -4.28571429
 -3.87755102  -3.46938776  -3.06122449  -2.65306122  -2.24489796
 -1.83673469  -1.42857143  -1.02040816  -0.6122449   -0.20408163
  0.20408163   0.6122449    1.02040816   1.42857143   1.83673469
  2.24489796   2.65306122   3.06122449   3.46938776   3.87755102
  4.28571429   4.69387755   5.10204082   5.51020408   5.91836735
  6.32653061   6.73469388   7.14285714   7.55102041   7.95918367
  8.36734694   8.7755102    9.18367347   9.59183673  10.
 10.40816327]
[100.0, 92.00333194502292, 84.33985839233654, 77.00957934194088,
 70.01249479383593, 63.34860474802169, 57.01790920449817, 51.02040816326535,
 45.356101624323244, 40.024989587671854, 35.02707205331117, 30.3623490212412,
 26.030820491461945, 22.0324864639734, 18.36734693877556, 15.035401915868436,
```

```
12.03665139525202, 9.371095376926318, 7.038733860891328, 5.039566847147049,
3.3735943356934834, 2.0408163265306296, 1.041232819658488, 0.3748438150770586,
0.04164931278634147, 0.04164931278633658, 0.3748438150770439,
1.0412328196584635, 2.0408163265305954, 3.373594335693439, 5.039566847146995,
7.038733860891264, 9.371095376926245, 12.036651395251937, 15.035401915868343,
18.367346938775455, 22.03248646397328, 26.030820491461817, 30.362349021241066,
35.027072053311024, 40.0249895876717, 45.35610162432308, 51.020408163265174,
57.017909204497975, 63.34860474802149, 70.01249479383573, 77.00957934194066,
84.33985839233631, 92.00333194502268, 99.9999999999976, 108.32986255726753]
```

```
[8]: b = a**2 #!
      print(b)
      b = a+a
      print(b)
      b = 2*a + 8
      print(b)
```

```
[ 9  25  49  81 121 169 225 289 361 441]
[ 6 10 14 18 22 26 30 34 38 42]
[14 18 22 26 30 34 38 42 46 50]
```

```
[9]: x = []
      x.append(-10)
      h = 20 / 49.
      for r in range(0,50):
          x.append(x[-1] + h)
      x = np.array(x)
      y = x**2
      print(x)
      print(y)
```

```
[-10.          -9.59183673 -9.18367347 -8.7755102   -8.36734694
 -7.95918367 -7.55102041 -7.14285714 -6.73469388 -6.32653061
 -5.91836735 -5.51020408 -5.10204082 -4.69387755 -4.28571429
 -3.87755102 -3.46938776 -3.06122449 -2.65306122 -2.24489796
 -1.83673469 -1.42857143 -1.02040816 -0.6122449  -0.20408163
  0.20408163  0.6122449   1.02040816  1.42857143  1.83673469
  2.24489796  2.65306122  3.06122449  3.46938776  3.87755102
  4.28571429  4.69387755  5.10204082  5.51020408  5.91836735
  6.32653061  6.73469388  7.14285714  7.55102041  7.95918367
  8.36734694  8.7755102   9.18367347  9.59183673 10.
 10.40816327]
[1.00000000e+02 9.20033319e+01 8.43398584e+01 7.70095793e+01
 7.00124948e+01 6.33486047e+01 5.70179092e+01 5.10204082e+01
 4.53561016e+01 4.00249896e+01 3.50270721e+01 3.03623490e+01
 2.60308205e+01 2.20324865e+01 1.83673469e+01 1.50354019e+01
 1.20366514e+01 9.37109538e+00 7.03873386e+00 5.03956685e+00
 3.37359434e+00 2.04081633e+00 1.04123282e+00 3.74843815e-01]
```

```

4.16493128e-02 4.16493128e-02 3.74843815e-01 1.04123282e+00
2.04081633e+00 3.37359434e+00 5.03956685e+00 7.03873386e+00
9.37109538e+00 1.20366514e+01 1.50354019e+01 1.83673469e+01
2.20324865e+01 2.60308205e+01 3.03623490e+01 3.50270721e+01
4.00249896e+01 4.53561016e+01 5.10204082e+01 5.70179092e+01
6.33486047e+01 7.00124948e+01 7.70095793e+01 8.43398584e+01
9.20033319e+01 1.00000000e+02 1.08329863e+02]

```

Coding time!: Modify the previous example using the ****** operator directly on an array.

- Using *numpy.array* functions (see docs)
 - empty - an uninitialized array,
 - ones - an array with values set to one,
 - zeros - an array with values set to zeros,
 - full - an array filled with value.
 - *_like - array with the shape and type of input

```

[10]: a = np.ones(5)
      print(a*5)
      a = np.zeros(5)
      print(a)
      a = np.full(5, 10)
      print(a)
      b = np.zeros_like(a)
      print(b)

```

```

[5.  5.  5.  5.  5.]
[0.  0.  0.  0.  0.]
[10 10 10 10 10]
[0 0 0 0 0]

```

For now, we did not specify the type stored by the array. Rather the type was determined for us based on our input. Look closely at the elements of arrays above. What is their type?

dtype member of array stores this information.

```

[11]: a = np.ones(5)
      print(a.dtype)
      a = np.full(5, 10)
      print(a.dtype) # why the difference?

```

```

float64
int64

```

We can specify the desired type at creation:

```

[12]: a = np.array(range(0,10))
      print(a.dtype)
      a = np.array(range(0,10), dtype=float)
      print(a.dtype)

```

int64
float64

- Numpy `arange` and `linspace`
 `arange`: regularly incrementing values
 `linspace`: array with a specified number of elements

```
[13]: np.arange(5.0)
```

```
[13]: array([0., 1., 2., 3., 4.])
```

```
[14]: np.arange(-1, 1, 0.5)
```

```
[14]: array([-1. , -0.5,  0. ,  0.5])
```

```
[15]: np.linspace(-1, 1, 11)
```

```
[15]: array([-1. , -0.8, -0.6, -0.4, -0.2,  0. ,  0.2,  0.4,  0.6,  0.8,  1. ])
```

- random values
NumPy provides a random number generator to create random values from 0 to 1. It is a part of `random` module via e.g. `default_rng(seed)` generator. Like this:

```
[16]: import time  
rand_rng = np.random.default_rng(int(time.time()))  
a = rand_rng.random(5)  
b = rand_rng.random(5)  
c = rand_rng.random(5)  
print(a, b, c)
```

```
[0.96535339 0.61153788 0.76335091 0.29784935 0.5850686 ] [0.93206249 0.27286263  
0.38630556 0.58248308 0.08845105] [0.4495736  0.43394031 0.28901602 0.57797708  
0.23365829]
```

1.1.3 Multidimensional arrays

NumPy allows for creation of multidimensional arrays. Personally I try not to use those too often, assuming that if you have a problem complex enough to need a matrix you would be probably better off using C or C++ for it. But that is an opinion and a biased one at that!

NumPy arrays have `shapes`, that defines how values can be accessed.

Lets see:

```
[17]: a = np.ones(5)  
print(a)  
print(a.shape)  
print(type(a.shape))
```

```
[1.  1.  1.  1.  1.]  
(5,)  
<class 'tuple'>
```

So `shape` is an attribute of an object, and a tuple at that - we recall it is **immutable**, meaning we can read but we can not modify the entries. Upon creation we can set the shape, by passing a tuple.

```
[18]: a = np.ones((2,2))
      print(a)
```

```
[[1.  1.]
 [1.  1.]]
```

and access elements like this:

```
[19]: a[0,0] = 0
      a[0,1] = 1
      a[1,0] = 10
      a[1,1] = 11
      print(type(a[0]))
      print(a[0])
      print(a[0,0])
      print(a[1:2].shape)
      print(a[0:2,0])
```

```
<class 'numpy.ndarray'>
[0.  1.]
0.0
(1, 2)
[ 0. 10.]
```

Another (possibly useful) attribute is `size`:

```
[20]: a.size
```

```
[20]: 4
```

Special arrays: `eye` - identity matrix

```
[21]: np.eye(5)
```

```
[21]: array([[1., 0., 0., 0., 0.],
            [0., 1., 0., 0., 0.],
            [0., 0., 1., 0., 0.],
            [0., 0., 0., 1., 0.],
            [0., 0., 0., 0., 1.]])
```

`diag` - exactly what you think:

```
[22]: np.diag(np.linspace(-1,1,5))
```

```
[22]: array([[ -1. ,  0. ,  0. ,  0. ,  0. ],
            [  0. , -0.5,  0. ,  0. ,  0. ],
            [  0. ,  0. ,  0. ,  0. ,  0. ],
            [  0. ,  0. ,  0. ,  0.5,  0. ],
            [  0. ,  0. ,  0. ,  0. ,  1. ]])
```

There is much more than is possible with arrays creation and modification. For now let us just try to use arrays for some simple operations:

```
* trnaspase() * dot()
```

```
[23]: x = np.linspace(-1,1,3)
      print(x)
      x = np.transpose(x)
      print(x)
```

```
[-1.  0.  1.]
[-1.  0.  1.]
```

```
[24]: x = np.linspace(-1,1,3)
      # x = np.transpose(x)
      A = np.array([[1,1,0],[0,2,0],[0,0,2]])
      print(A)
      print(x)
      # print(x*A)
      print(x.dot(A))
```

```
[[1 1 0]
 [0 2 0]
 [0 0 2]]
[-1.  0.  1.]
[-1. -1.  2.]
```

And some fun with arrays, that might come in handy (or not), that is solving an eigen problem:

```
[25]: from numpy import linalg as LA
      sig, v = LA.eig(A)
      print(sig)
      print(v)
```

```
[1.  2.  2.]
[[1.         0.70710678 0.         ]
 [0.         0.70710678 0.         ]
 [0.         0.         1.         ]]
```

1.2 Processing files - IO operations on files

We will now perform simple read and write operations on files. Our ultimate goal is to process files which contain settings for software packages we use. For example in an optimization loop, we might

need to manipulate files defining parameters for the simulator and computed by the optimizer. Similarly we will need the value of the objective function to be read and passed to the optimizer.

We will start just with opening, writing and reading. Simple stuff:

```
[26]: f = open('burza', 'r') # similarly to printf there is a flag that specifies what
      ↪to do: 'w', 'r', 'a'
      a = f.readline()
      print(a, end='')
      a = f.readline()
      print(a, end='')
      f.close()
```

Burza
OSOBY:

```
[27]: f = open('burza', 'r')
      alllines = f.readlines()
      for line in alllines:
          if 'ARIEL' in line:
              print(line)
      f.close()
```

ARIEL, duch powietrzny.

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

```
[28]: alllines[502]
```

```
[28]: '\n'
```

```
[29]: f = open('burza', "r")
      g = open('burza2', "w")

      for line in f:
          if 'Ariel' in line or 'ARIEL' in line:
              # a = line.split(' ')
              line = line.replace('Ariel', 'Perwol')
              print(line)
              # line.replace('ARIEL', 'Perwol')
              pass
          if 'KALIBAN' in line:
              pass
      g.write(line)
```

```
f.close()  
g.close()
```

ARIEL, duch powietrzny.

Przyjdź, Perwolu[10]!

ARIEL

Wykona Perwol.

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

(Perwol wychodzi)

(wchodzi Perwol w postaci nimfy wodnej)

Piękne zjawisko! Lotny Perwolu,

ARIEL

Wychodzi Kaliban - Wchodzi Perwol niewidzialny, gra i śpiewa, za nim Ferdynand.

ARIEL

ARIEL

ARIEL

ARIEL

Wzrok się ich zmierzył. Piękny Perwolu,

(na stronie) Dobrześ się sprawił, Perwolu! (głośno) Idźmy!

(do Perwola) A teraz słuchaj, co jeszcze masz zrobić!

ARIEL

Wchodzi Perwol niewidzialny, grając uroczystą melodię.

Zasypia - Perwol wychodzi.

Rozmawiają na stronie. Muzyka. Wchodzi Perwol niewidzialny.

ARIEL

ARIEL

Wchodzi Perwol niewidzialny.

ARIEL

ARIEL

ARIEL

ARIEL

Perwol gra na piszczałce i na bębenku.

Grzmoty i błyskawice. Wbiega Perwol w postaci harpii, roztacza skrzydła nad stołem; bankiet znika.

ARIEL

Udałeś pysznie harpię, Perwolu,

Hej, Perwolu, słuگو mój przebiegły!

ARIEL

ARIEL

ARIEL

Lecz się nie zbliżaj, piękny Perwolu,

ARIEL

Więc wszystko dobrze. - Przybądź Perwolu!

Dzięki wam. - Perwolu,

Jak myśli szybki, przybądź, Perwolu!

Wchodzi Perwol.

ARIEL

ARIEL

ARIEL

ARIEL

(wchodzi Perwol, niosąc błyszczące materie itd.)

Prospero i Perwol zostają niewidzialni. - Wchodzą: Kaliban, Stefano i Trynkulo, przemokli.

Słychać krzyk polujących. Wchodzą duchy w postaci psów, które podszczuwane przez Prospera i Perwola gonią za Stefanem, Trynkulem i Kalibanem.

ARIEL

ARIEL

Wchodzą: Prospero w czarnoksięskiej odzieży i Perwol.

ARIEL

ARIEL

ARIEL

Idź więc i wolność daj im, Perwolu,

ARIEL

Uroczysta muzyka. - Wchodzi Perwol za nim Alonso z gestami obłąkanego, prowadzony przez Gonzala, za nimi Sebastian i Antonio w tej samej postawie, prowadzeni przez Adriana i Franciska. Wszyscy wstępują w koło zakreślone przez Prospera i stoją w nim zaczarowani. Prospero, patrząc na nich, mówi:

Śpiesz Perwolu, przynieś mi z mej celi

(wychodzi Perwol)

Śpiesz Perwolu, wkrótce będziesz wolny!

Wraca Perwol, a śpiewając, pomaga ubierać się Prosperowi.

ARIEL

ARIEL

(wchodzi Perwol, za nim Kapitan i Bosman, odurzeni)

ARIEL

ARIEL

(do Perwola) Leć, duchu, rozwiąż potęgę mych czarów,

Wróć teraz wolność. (wychodzi Perwol)

Wraca Perwol, pędząc przed sobą Kalibana, Stefana i Trynkula, w ukradzionej odzieży.

Mój Perwolu, ptaszku mój kochany,

Perwolu - ze względu na rytm wiersza należy wymawiać jak wyraz czterosylabowy: Aryjelu. [przypis edytorski]

Sebastian - imię to (podobnie jak: Perwol, Mediolan) w tym tłumaczeniu w partiach wierszem należy czytać dla zachowania rytmu wiersza jak wyraz

czterosylabowy: Sebastyan; a za tym również inne formy odmiany: Sebastyanie itd. [przypis edytorski]

Using the above we could already manipulated setting files, or any files with some algorithm we choose. For the read and replace it is interesting to use line by line processing with `fileinput`

```
[30]: import fileinput
```

```
[31]: f = fileinput.input('burza2', openhook=fileinput.hook_encoded('utf-8'))
      for line in f:
          if 'ARIEL' in line:
              print(line)
          if line.startswith('KALIBAN'):
              print(line)
      f.close()
```

KALIBAN, dziki i potworny niewolnik.

ARIEL, duch powietrzny.

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

KALIBAN

ARIEL

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

ARIEL

KALIBAN

KALIBAN

KALIBAN

KALIBAN

ARIEL

KALIBAN

ARIEL

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

ARIEL

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

KALIBAN

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

ARIEL

KALIBAN

KALIBAN

KALIBAN

```
[32]: print(line)
```

pobieżyć (daw.) - podążyć. [przypis edytorski]

```
[33]: print(line.split('-'))
```

['pobieżyć (daw.) ', ' podążyć. [przypis edytorski]\n']

```
[34]: a = ['ala', 'ma']  
      b = ['kota']  
      print(' '.join(a+b))
```

ala ma kota

```
[35]: print(line)  
      a = line.split()  
      print(a)
```

```
b = ' '.join(a)
print(b)
```

pobieżyć (daw.) - podążyć. [przypis edytorski]

```
['pobieżyć', '(daw.)', '-', 'podążyć.', '[przypis', 'edytorski]']
pobieżyć (daw.) - podążyć. [przypis edytorski]
```

```
for line in fileinput.input(mshfile, inplace=1):
    if line.startswith("S="):
        print ("S="+str(S)+";")
    elif line.startswith("a="):
        print "a="+str(alfa)+";"
    else:
        print line
```

Coding Time!: Use `fileinput` with `inplace` to modify the file. For example change names of selected characters. Ariel->Perwol Kaliban->Taliban etc.

1.3 Matplotlib

is a very popular plotting library for python. It can be used to produce almost publication quality figures. Personally I use it for most of my plotting needs, but for publications I prefer Latex Tix library.

Let us start with our first plot of some $f(x)$ function. We will be using `numpy.array` to handle our data. To use `matplotlib` we need to import it:

1.3.1 First plot

```
[36]: import matplotlib.pyplot as plt
```

To have a plot for $x \in [-1, 1]$ we will use `numpy.linspace()` we skipped last time:

```
[37]: import numpy as np

x = np.linspace(-10, 10, 1001)
# Generate 101 points between -1 and 1 including boundaries
y = np.sin(x**2)
```

And plotting $y(x)$ is just:

```
[38]: plt.plot(x, y)
```

```
<IPython.core.display.Javascript object>
```

```
<IPython.core.display.HTML object>
```

```
[38]: [<matplotlib.lines.Line2D at 0x7fb591447ee0>]
```

We can do many things with the plot. Add grid, set axis limits, add text, labels, change sizes, colors, etc.

For now let just increase the size of the plot, add a legend entry, a grid, change the font size and thickness of the line.

We can also do a scatter plot:

```
[39]: f = plt.figure(figsize=(20,12)) # figure size
font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
      ↪font I like
plt.rc('font', **font)

plt.plot(x, y, label='$f(x)$', lw=10)
plt.scatter(0,0, s=500, c='black')

plt.grid()
plt.legend()
plt.xlim(x[0], x[-1])
plt.ylim(min(y)-0.05, max(y)+0.05)

plt.xlabel("$\\alpha$")
plt.ylabel('y')
# plt.savefig('aaa.png')
```

<IPython.core.display.Javascript object>

<IPython.core.display.HTML object>

```
[39]: Text(0, 0.5, 'y')
```

Coding Time!: Try to plot a nice saw function, i.e. $/\backslash/\backslash/\backslash$. You might need to create an additional function to do that.

Or a scatter plot all on itsown:

```
[40]: rand_rng = np.random.default_rng(int(time.time()))
x = rand_rng.random(50)
y = rand_rng.random(50)
z = 1000*rand_rng.random(50)

plt.scatter(x,y, s=z, c='pink')
```

```
[40]: <matplotlib.collections.PathCollection at 0x7fb591458fa0>
```

Coding Time!: With scatter plot colors can be passed as an array, so that each point has a different color. Try to produce a plot with multiple colors. Colors can be provided in multiple ways One would be with a string. Examples are:

- b: blue
- g: green

- r: red
- c: cyan
- m: magenta
- y: yellow
- k: black
- w: white

Or with a tuple passing the RGB values, so (0,0,0) would be black

```
[41]: rand_rng = np.random.default_rng(int(time.time()))
x = rand_rng.random(5)
y = rand_rng.random(5)
z = 1000*rand_rng.random(5)

cmap = np.array(['r', 'g', 'b'])
cidx = np.array([0, 2, 1, 1, 1])

plt.scatter(x,y, s=z, c=cmap[cidx])
```

```
[41]: <matplotlib.collections.PathCollection at 0x7fb59145fc70>
```

So we can make a simple plot. We can generate a number of them with **subplots**: (there are more ways to do this)

```
[42]: fig, axs = plt.subplots(2,2, figsize=(20,12))

x = np.linspace(-10, 10, 1001)
y = np.sin(x**2)
axs[0,0].plot(x, y)
axs[0,0].grid()

y = np.cos(x)
axs[0,1].plot(x, -y)

y = 1/x
axs[1,0].plot(x, y)
axs[1,0].set_xlim([-1, 1])

y = np.exp(x)
axs[1,1].plot(x, y)
```

```
<IPython.core.display.Javascript object>
```

```
<IPython.core.display.HTML object>
```

```
<ipython-input-42-c9247cbd6306>:11: RuntimeWarning: divide by zero encountered
in true_divide
```

```
    y = 1/x
```

```
[42]: [matplotlib.lines.Line2D at 0x7fb5913092e0]
```

Or:

```
[43]: f = plt.figure(figsize=(20,12)) # figure size
      axs = f.subplots(2,2)

      x = np.linspace(-10, 10, 1001)
      y = np.sin(x**2)
      axs[0,0].plot(x, y)
      axs[0,0].grid()

      y = np.cos(x)
      axs[0,1].plot(x, -y)

      y = 1/x
      axs[1,0].plot(x, y)
      axs[1,0].set_xlim([-1, 1])

      y = np.exp(x)
      axs[1,1].plot(x, y)
```

```
<IPython.core.display.Javascript object>
```

```
<IPython.core.display.HTML object>
```

```
<ipython-input-43-4dce7bf22c07>:12: RuntimeWarning: divide by zero encountered
in true_divide
      y = 1/x
```

```
[43]: [matplotlib.lines.Line2D at 0x7fb59124ed60]
```

Coding Time!: ?

It is possible to generate more complex, 2 and 3 dimensional plots, vector fields etc. We will have a brief look at what is possible, but if something is not covered here on line documentation will have it.

1.3.2 Plotting 2D data

Consider a function of the form $f(x, y)$ which you would like to look at. This could be temperature distribution on a plane or surface. We might be interested in producing a color map, a contour map, or a 3D plot, where value of f determines the third dimension.

Let f be described on a $(-1, 1) \times (-1, 1)$ domain as a simple Gaussian distribution that is:

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

Plotting this will be a bit tricky, since we need to follow the Matplotlib convention.

We will select some reasonable (i.e. good looking) value of, e.g. $\sigma = 1$. Let's start simple:

```
[44]: sig = 1
```

```
[45]: f = plt.figure(figsize=(10,10)) # figure size
font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
      ↪font I like
plt.rc('font', **font)

# Make data.
X = np.linspace(-3, 3, 1000)
Y = np.linspace(-3, 3, 1000)
X, Y = np.meshgrid(X, Y) # make a coordinate matrix

R = 1.0/(2*np.pi*sig**2)*np.exp(-(X**2 + Y**2)/(2*sig**2))

plt.pcolormesh(X, Y, R)
```

<IPython.core.display.Javascript object>

<IPython.core.display.HTML object>

```
[45]: <matplotlib.collections.QuadMesh at 0x7fb59120d970>
```

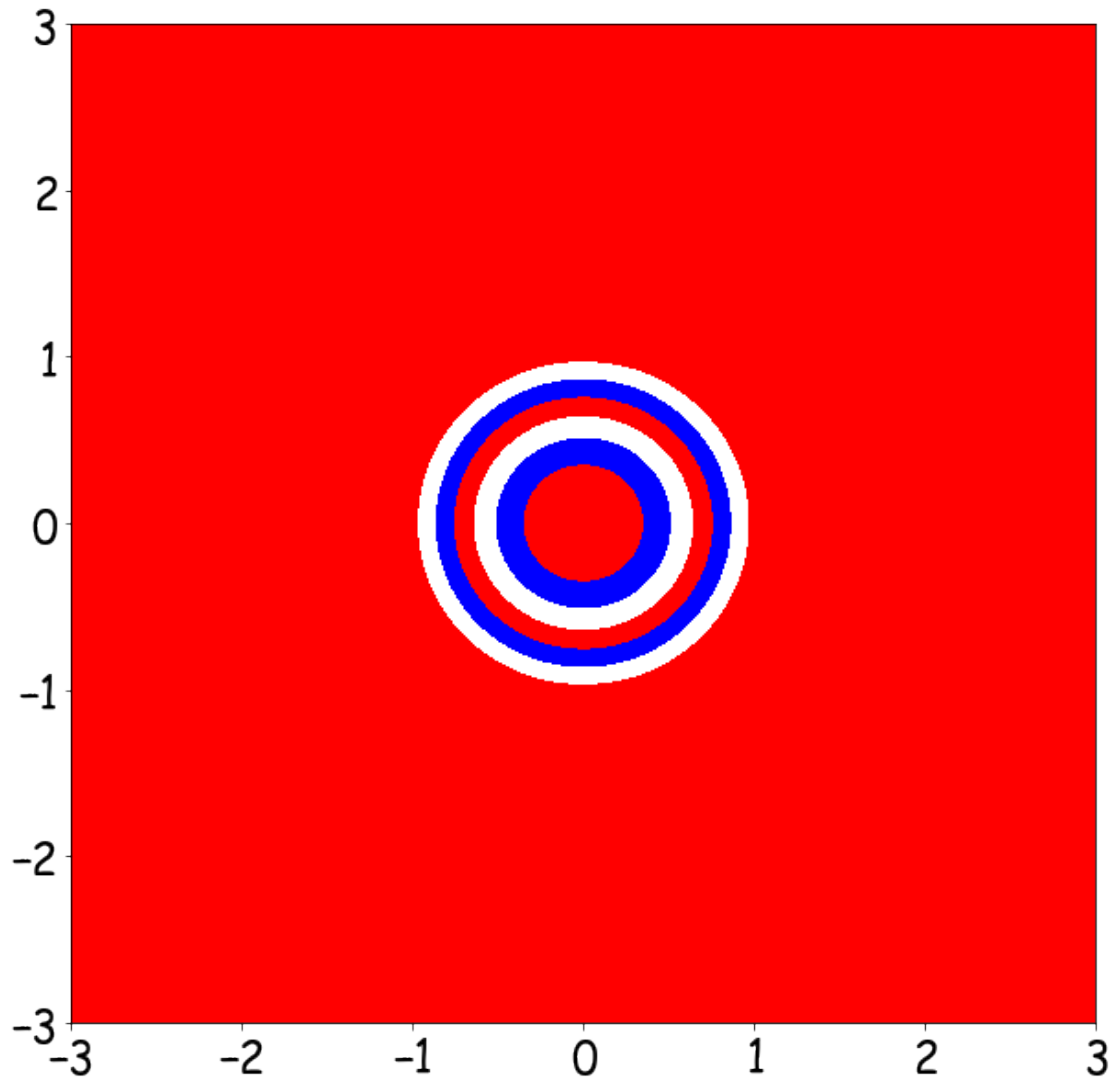
Note: `meshgrid` and `pcolormesh`. The first prepares a matrix of coordinates, the second is used to create a color plot on a rectangular grid. There are a number of options.

now add some contours:

```
[46]: %matplotlib inline
f = plt.figure(figsize=(10,10)) # figure size
font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
      ↪font I like
plt.rc('font', **font)

# plt.pcolormesh(X, Y, R)
# plt.contour(X, Y, R, levels=[0, 0.1, 0.11, 0.12], colors=['black'])
plt.contourf(X, Y, R, levels=[0, 0.1, 0.11, 0.12, 0.13, 0.14, 0.15, 0.16, 0.
      ↪161], colors=['red', 'white', 'blue'])

plt.show()
```



1.4 3D

Is more fun, and there is almost no boundaries to what can be done

```
[47]: %matplotlib notebook

from mpl_toolkits.mplot3d import Axes3D # Produces 3D axes setting
from matplotlib import cm

f = plt.figure(figsize=(5,5)) # figure size
font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
    ↪font I like
plt.rc('font', **font)
```

```

ax = f.gca(projection='3d')

sig = 1
R = 1.0/(2*np.pi*sig**2)*np.exp(-(X**2 + Y**2)/(2*sig**2))

surf = ax.plot_surface(X, Y, R, cmap=cm.jet)

# fig.colorbar(surf, shrink=0.5, aspect=5)

plt.show()

```

<IPython.core.display.Javascript object>

<IPython.core.display.HTML object>

1.5 Reading from files (with data)

We consider a computational process monitoring of which is performed with a text data stored to a file during the calculation. Here I have a file `mean.avg` that contains average values of some vector field in time:

[228]:

```
ls
burza  burza2  data/  data.dat  'Lecture 3.ipynb'  mean.avg
```

Let's start by examining the content of the file (in notepad).

The file contains columns of numbers (possibly comma separated, or not) so the best way to treat is to precess it directly to NymPy arrays using NumPy functions. Note: Processing of `noraml` text files can also be performed, but in case of data files using NumPy functions is more natural.

[229]:

```

T, c1, c2, c3 = np.loadtxt('mean.avg', comments="#", skiprows=1,
↪usecols=(0,1,2,3), unpack=True)

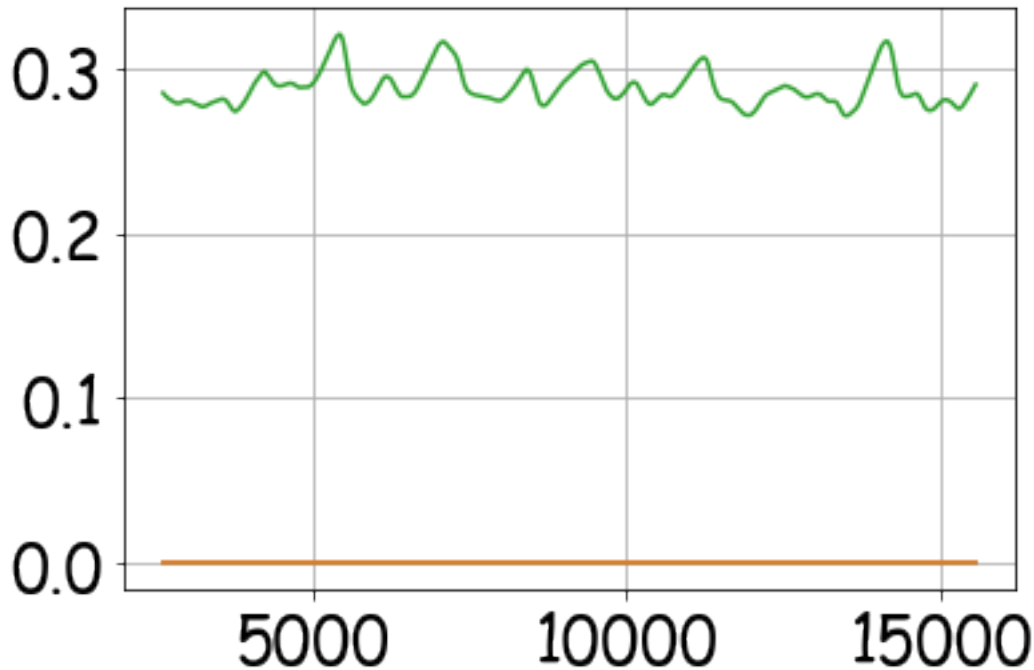
```

[230]:

```

plt.plot(T, c1, T, c2, T, c3)
plt.grid()

```



Coding Time!: This one is a bigger one. A small visualization project, but graphs only! Use data in `data.tar.gz` to plot history of flow rate of the perturbation in time. The files contain mean flow values in `mean.avg` and modal energy in `energy.mdl`. Modal energy is energy associated with Fourier modes. I.e. Modes are numbered from 0 up to a certain number. Mode zero corresponds to spatially averaged value and other modes to consecutive harmonic elements (1 - one period of sin and cosine in a domain, 2 - two waves in a domain, etc.).

The goal is to 1. prepare a plot of mean w velocity in time. 2. prepare a plot of the energy of mode zero in time 3. prepare a plot of the sum of the remaining modes in time ($\sum_{M=1,2,3,\dots}$)

```
[231]: ls

burza  burza2  data/    data.dat  'Lecture 3.ipynb'  mean.avg
```

```
[232]: import os
```

We will use `os` module and its functionality to traverse files on the disk.

```
[233]: dfiles = []

depth = 2
stuff = os.path.abspath(os.path.expanduser(os.path.expandvars('data/data/')))
# stuff = os.path.abspath(os.path.expanduser(os.path.expandvars('./A1_4/
↳ Mystartingpositions/0.5/')))
for root,dirs,files in os.walk(stuff):
```

```

    if root[len(stuff):].count(os.sep) < depth:
        for f in files:
            if f[-3:] == '.mdl':
                dfiles.append(os.path.join(root,f))

dfiles.sort()

```

```

[234]: print(len(dfiles))
       for f in dfiles:
           print(f)

```

```

26
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
0625/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
09375/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
109375/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
1171875/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
1181640625/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
11840820313/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
1184387207/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
1184425354/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
1184434877/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
11844372578/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
11844377339/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
11844384481/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
11844396385/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
11844515629/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
11844539505/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
1184463501/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1

```

```

11845397949/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
11846923828/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
11853027344/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
119140625/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
12109375/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
125/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.1
25/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.2
/energy.mdl
/home/sgepner/OneDrive/Dydaktyka/Python/Lectures/Lectures/Lecture3/data/data/0.3
/energy.mdl

```

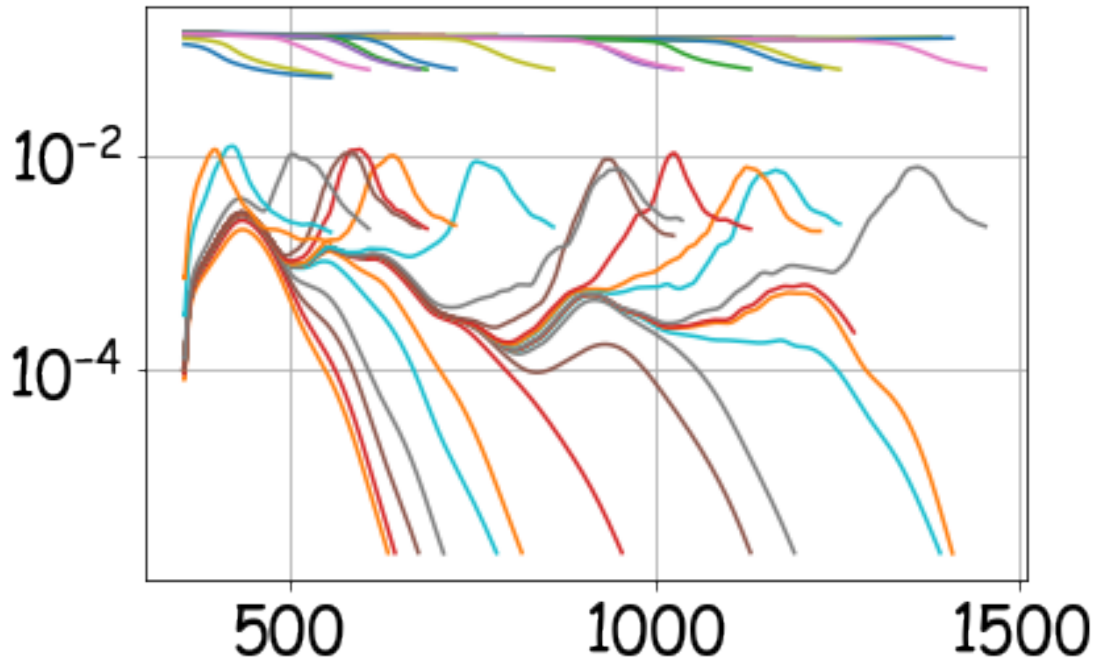
```

[235]: for f in dfiles:
        # read the data from file:
        T, Mod, En = np.loadtxt(f, comments="#", skiprows=1, usecols=(0,1,2),
        ↪unpack=True)
        # select the time range using fancy conditional statment
        T = T[(Mod==0)]
        # select energy associated with mode 0
        e0= En[(Mod==0)]
        # sum the remaining modes:
        E = [] # will store arrays in time for all modes
        for i in range(0,int(max(Mod))+1):
            e = np.array(En[(Mod==i)]) # get all timesteps for mode i
            E.append(e)

        e = E[1] # we will use array summation
        for i in range(2,int(max(Mod))+1):
            e += E[i]

        plt.plot(T, e0, T, e)
plt.yscale('log')
plt.grid()

```



1.6 SciPy

Is the second probably most commonly (my opinion rather than fact) used library when it comes to processing data. You could say, that when it comes to numerical procedures - SciPy has it all. We will focus on only some of the available algorithms, but should you need more - you know where to look.

We will: * Decimate data with slice operations * Use interpolation to build a continuous view of the data (e.g. to be used for estimating maximum or roots) * Look for extrema * Look for roots * Integrate * Solve an ODE * Attempt some curve fitting

and all that in a day work.

We will use a slice of data from mean.avg, using the slice operation with collection[start : stop : every n'th]. The last entry is used to select only some of the data.

```
[236]: T, c1, c2, c3 = np.loadtxt('mean.avg', comments="#", skiprows=1,
    ↪ usecols=(0,1,2,3), unpack=True)
```

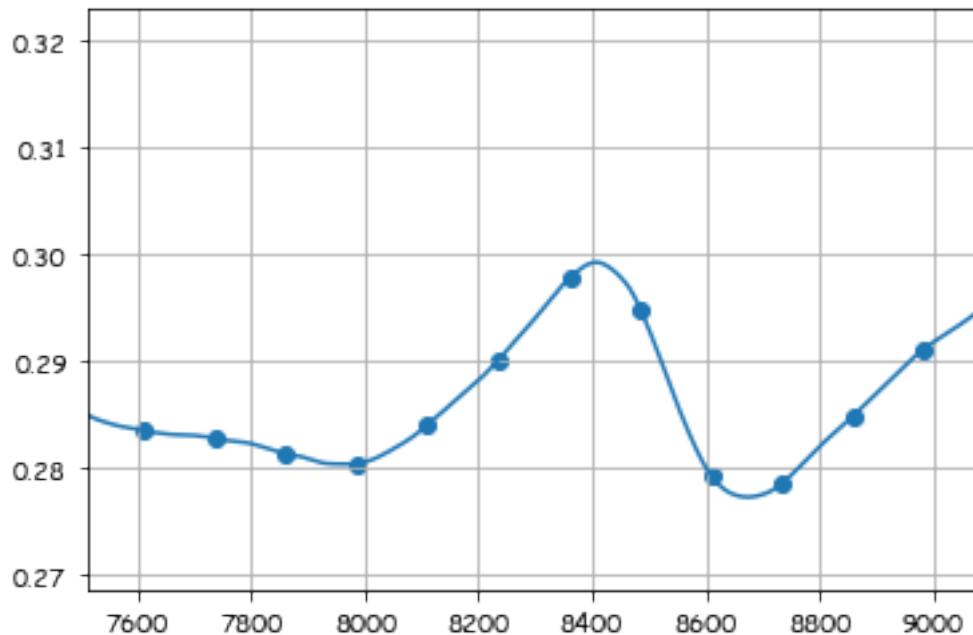
```
[237]: X = T[10000:13000:250] # We can select every other value from an array!
Y = c3[10000:13000:250]
print(len(X), len(Y))
```

12 12

Lets plot the original data, and the points selected from it and stored in X and Y:

```
[238]: %matplotlib inline
plt.plot(T, c3)
plt.scatter(X, Y)

plt.xlim(X[0]-100, X[-1]+100)
plt.grid()
```



1.6.1 Interpolation

There is a number of interpolation procedures available from SciPy, the most up to date list can be found here: <https://docs.scipy.org/doc/scipy/reference/interpolate.html>

Choosing the right interpolation technique is very much problem dependant, might require some trial-and-error and here we will focus on the use of only some more general SciPy interpolation procedures, allowing to easily switch between interpolation techniques.

Note: We will only do 1D interpolation, but higher dimensions are also possible, see e.g. [CloughTocher2DInterpolator](#).

`scipy.interpolate.interp1d` The first is the `interp1d`, it allows for different types of **interpolation**, but does not allow for **extrapolation**. The advantage is that it is very easy to use. A general call to `interp1d` needs the pass of arrays of values used to build the interpolating function of a selected kind. The `kind` can be any of the implemented. The default is `'linear'`, but we will look at: `zero`, `slinear`, `quadratic` and `cubic` - featuring [splines](#) of a respective order:

Note the difference between *interpolation*, which is the act of approximating a value in between the known values and *extrapolation*, which is guessing outside of the known values.

To use `interp1d` lets import it from `scipy` module:

```
[239]: from scipy.interpolate import interp1d
```

`interp1d` returns a function object to be used with data:

```
[240]: f = interp1d(X, Y) # default is liner
```

Which we can use as any other function with Python, with a single variable, a list or a NumPy array:

```
[241]: print(f(8000))
       print(f(X[3]))
       print(f(X[3:6]))
```

```
0.2808821257384416
0.280445966782
[0.28044597 0.28410502 0.29017429]
```

Lets, see how good our interpolation is, using linear distribution of points:

```
[242]: x = np.linspace(X[0], X[-1], 10000) # only interpolation
       # x = np.linspace(X[0]-100, X[-1]+100, 1000) # interp1d does not work with
       ↪extrapolation!
```

Note: Show the pitfalls of dynamic typing.

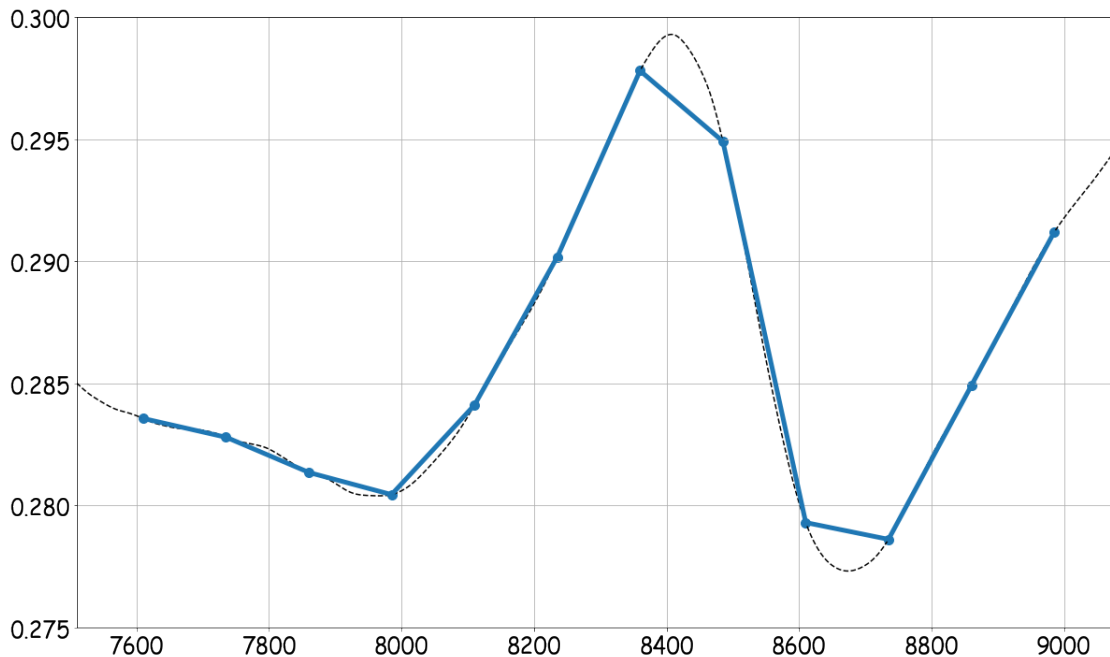
Replace `fig = plt.figure(figsize=(20,12))`

with `f = plt.figure(figsize=(20,12))` which you normally use and have a nice error!

```
[243]: fig = plt.figure(figsize=(20,12)) # figure size
       font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
       ↪font I like
       plt.rc('font', **font)

       plt.plot(T, c3, '--', c='black') # '--' - dashed, c - colour,
       plt.scatter(X, Y, s=100) # s sets size
       plt.plot(x, f(x), lw=5) # lw - line width

       plt.xlim(X[0]-100, X[-1]+100)
       plt.ylim(0.275, 0.3)
       plt.grid()
```



And we have a nice linear interpolation. Let's try the remaining types, and get a glimpse at how interpolating functions differ:

```
[244]: fzero      = interp1d(X, Y, kind='zero') # default is liner
        fslinear = interp1d(X, Y, kind='slinear')
        fquadratic = interp1d(X, Y, kind='quadratic')
        fcubic = interp1d(X, Y, kind='cubic')
```

We will take this occasion to add a legend to our plot. This is done by placing a `label='label text'` in the `plot()` call and calling on the `legend()` function.

Note: `label=''` accepts Latex mathematical environment with $$$$.

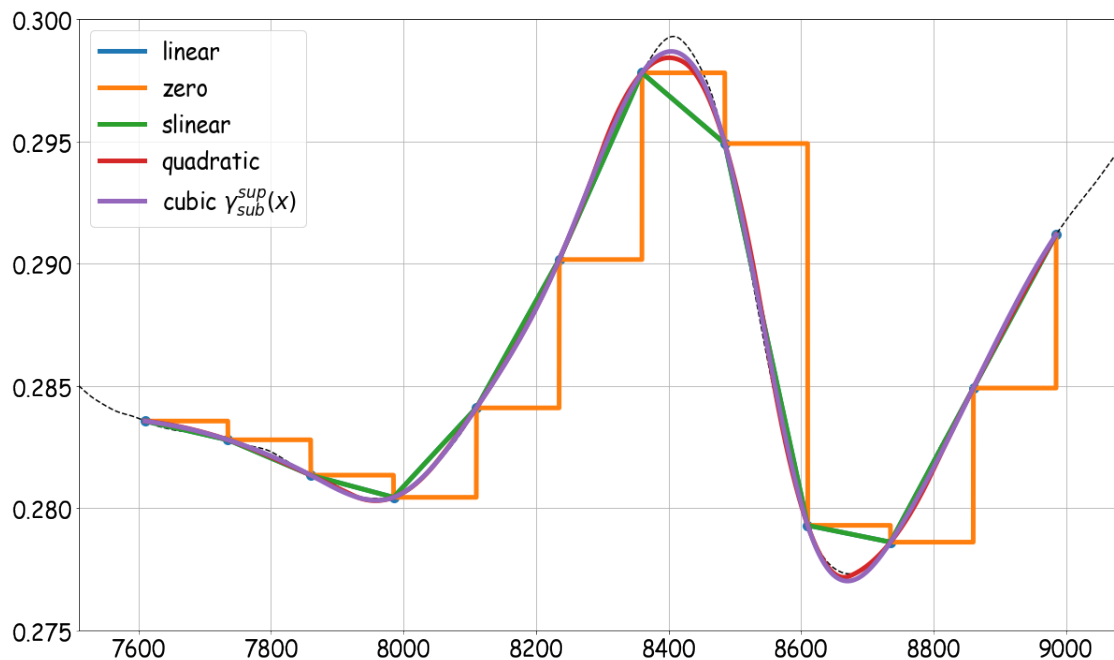
```
[245]: fig = plt.figure(figsize=(20,12)) # figure size
        font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
        ↪font I like
        plt.rc('font', **font)

        plt.plot(T, c3, '--', c='black') # '--' - dashed, c - colour,
        plt.scatter(X, Y, s=100) # s sets size

        # Note the use of label in the function call - we will have a legend!
        plt.plot(x, f(x), lw=5, label='linear') #
        plt.plot(x, fzero(x), lw=5, label='zero') #
        plt.plot(x, fslinear(x), lw=5, label='slinear') #
        plt.plot(x, fquadratic(x), lw=5, label='quadratic') #
```

```
plt.plot(x, fcubic(x), lw=5, label='cubic  $\gamma_{sub}^{sup}(x)$ ') #  $\gamma_{sub}^{sup}(x)$ 
    ↳ goes here  $\gamma_{sub}^{sup}(x)$ 

# It is going to be Legendary!
plt.legend()
plt.xlim(X[0]-100, X[-1]+100)
plt.ylim(0.275, 0.3)
plt.grid()
```



`scipy.interpolate.InterpolatedUnivariateSpline` Returns a 1D interpolating spline for a provided data. Aside from data we may pass weights, degree of the spline ($1 \leq k \leq 5$) and a flag specifying what should happen out of data bounds, the extrapolation `ext` $\in (0,3)$.

```
[246]: from scipy.interpolate import InterpolatedUnivariateSpline

x = np.linspace(X[0]-100, X[-1]+100, 1000) # we will be extrapolating!

f = InterpolatedUnivariateSpline(X, Y, k=1)
```

We will use this moment to show how to look for array indices that correspond to a given value. For the case of a sorted array it is probably best to use the `numpy.searchsorted` function. It accepts a sorted collection and values (or a value) and returns an index where the value should be inserted to maintain the order (sortiness??) of the collection. Example:

```
[247]: a = np.searchsorted(x, X[0])
      b = np.searchsorted(x, X[-1])
      print(a, b)

      print(x[a], X[0])
      print(x[b], X[-1])
```

```
64 936
7611.000900900901 7610.1
8985.775675675675 8985.1
```

`a` and `b` are indices of the first greater than `X[0]` and `X[-1]` elements of `x`.

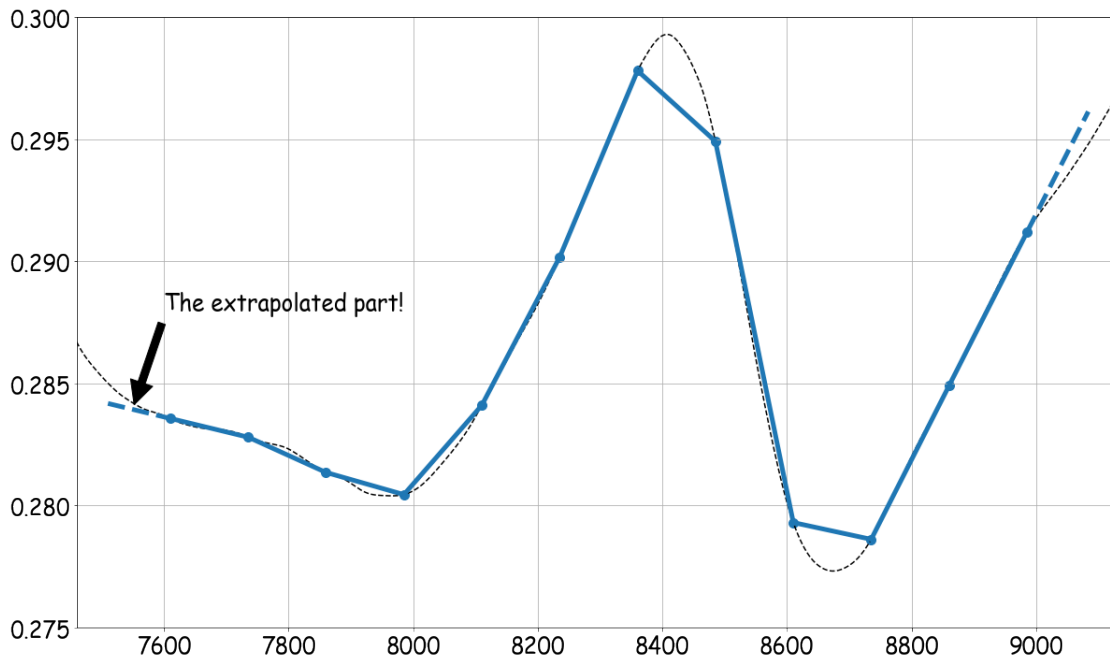
```
[248]: fig = plt.figure(figsize=(20,12)) # figure size
      font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
      ↪font I like
      plt.rc('font', **font)

      plt.plot(T, c3, '--', c='black') # '--' - dashed, c - colour,
      plt.scatter(X, Y, s=100) # s sets size

      a = np.searchsorted(x, X[0])
      b = np.searchsorted(x, X[-1])
      p = plt.plot(x[a:b], f(x[a:b]), lw=5) # note, we return to p
      plt.plot(x[0:a], f(x[0:a]), '--', lw=5, c = p[0].get_color()) # to get the same
      ↪colour
      plt.plot(x[b:-1], f(x[b:-1]), '--', lw=5, c = p[0].get_color())

      ax = plt.gca()
      ax.annotate('The extrapolated part!', xy=(x[25], f(x[25])), xytext=(7600, 0.288),
                  arrowprops = dict(facecolor='black', width=8, headwidth=25,
                  ↪headlength=25, shrink = 0.05),)

      plt.xlim(X[0]-150, X[-1]+150)
      plt.ylim(0.275, 0.3)
      plt.grid()
```



Note the use of `annotate` to place an arrow and some text to our plot.

Lets experiment with `exp` and `k`: * `ext=0` or 'extrapolate' - default. * `ext=1` or 'zeros', returns 0. * `ext=2` or 'raise', raise a `ValueError`. * `ext=3` of 'const', return boundary value.

```
[249]: f = InterpolatedUnivariateSpline(X, Y, k=2, ext=0)

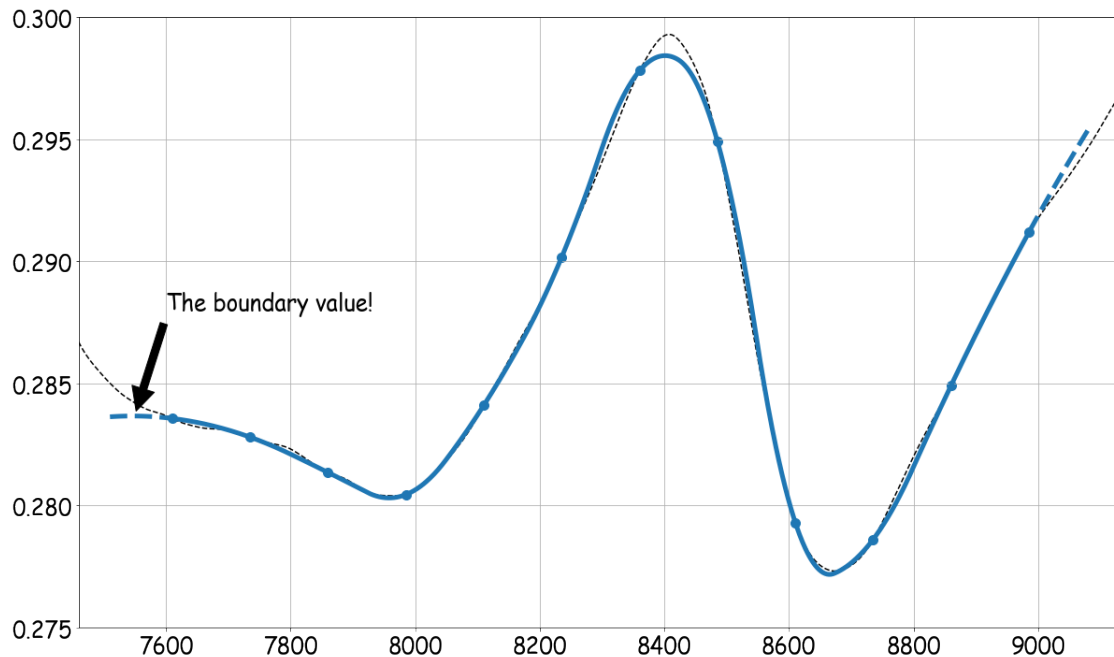
fig = plt.figure(figsize=(20,12)) # figure size
font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
    ↳font I like
plt.rc('font', **font)

plt.plot(T, c3, '--', c='black') # '--' - dashed, c - colour,
plt.scatter(X, Y, s=100) # s sets size

a = np.searchsorted(x, X[0])
b = np.searchsorted(x, X[-1])
p = plt.plot(x[a:b], f(x[a:b]), lw=5) # note, we return to p
plt.plot(x[0:a], f(x[0:a]), '--', lw=5, c = p[0].get_color()) # to get the same
    ↳colour
plt.plot(x[b:-1], f(x[b:-1]), '--', lw=5, c = p[0].get_color())

ax = plt.gca()
ax.annotate('The boundary value!', xy=(x[25], f(x[25])), xytext=(7600, 0.288),
           arrowprops = dict(facecolor='black', width=8, headwidth=25,
    ↳headlength=25, shrink = 0.05),)
```

```
plt.xlim(X[0]-150, X[-1]+150)
plt.ylim(0.275, 0.3)
plt.grid()
```



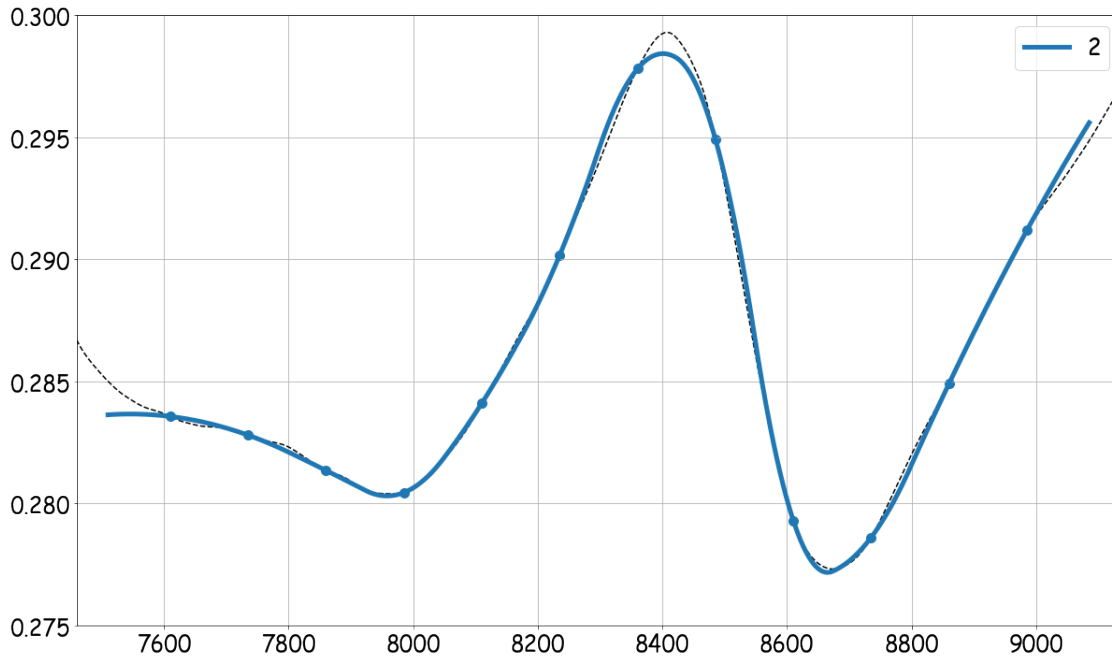
```
[250]: f1 = InterpolatedUnivariateSpline(X, Y, k=1)
f2 = InterpolatedUnivariateSpline(X, Y, k=2)
f3 = InterpolatedUnivariateSpline(X, Y, k=3)
f4 = InterpolatedUnivariateSpline(X, Y, k=4)
f5 = InterpolatedUnivariateSpline(X, Y, k=5)
```

```
[251]: fig = plt.figure(figsize=(20,12)) # figure size
font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
      ↳font I like
plt.rc('font', **font)

plt.plot(T, c3, '--', c='black') # '--' - dashed, c - colour,
plt.scatter(X, Y, s=100) # s sets size

# plt.plot(x, f1(x), lw=5, label='1')
plt.plot(x, f2(x), lw=5, label='2')
# plt.plot(x, f3(x), lw=5, label='3')
# plt.plot(x, f4(x), lw=5, label='4')
# plt.plot(x, f5(x), lw=5, label='5')
```

```
plt.legend()
plt.xlim(X[0]-150, X[-1]+150)
plt.ylim(0.275, 0.3)
plt.grid()
```



1.6.2 Finding extreme value

We know how to build an interpolating function from data. Let's say the points we have result from some form of a process we monitor and that we would like to find the extreme value. We will assume the process to be represented by some interpolant and look for the maximum. Note we are not interested in the maximum, stored in the array ($\max(f(x))$), but in the extrema of a function, that corresponds to whatever our problem is.

Let start with just examining the arrays:

```
[252]: print( max(Y), max( f2(x)) )
      i = np.where(f2(x) == max(f2(x)))
      print(i, x[i], f2(x[i]))
```

```
0.29781560279 0.29843369850530627
(array([565]),) [8400.86576577] [0.2984337]
```

So an element of x corresponds to a value returned by an interpolating function $f2$ (one of many) that happens to be the elemental maximum - nice, but not what we want yet.

In what follows we will use a `scipy.optimize` module minimization procedure `scipy.optimize.minimize`. As before should you need something more fancy, have a look at the documentation [here](#). I will use the $f2$ created from interpolation before.

We need to import an appropriate module:

```
[253]: from scipy.optimize import minimize
```

`minimize` takes a number of arguments. Two are required the function to be minimized and the initial guess. You can also specify the particular method you wish to use (see docs), bounds, tolerance, etc.

The returned value is an object containing a number of informations. To us the most important is the result.

(experiment with starting value, and bounds)

```
[254]: # res = minimize(f2, 8450)
res = minimize(f2, x0=8600, bounds=((8400, 8800),) ) # note the nasty way bounds
        ↪ are provided
print(res)
```

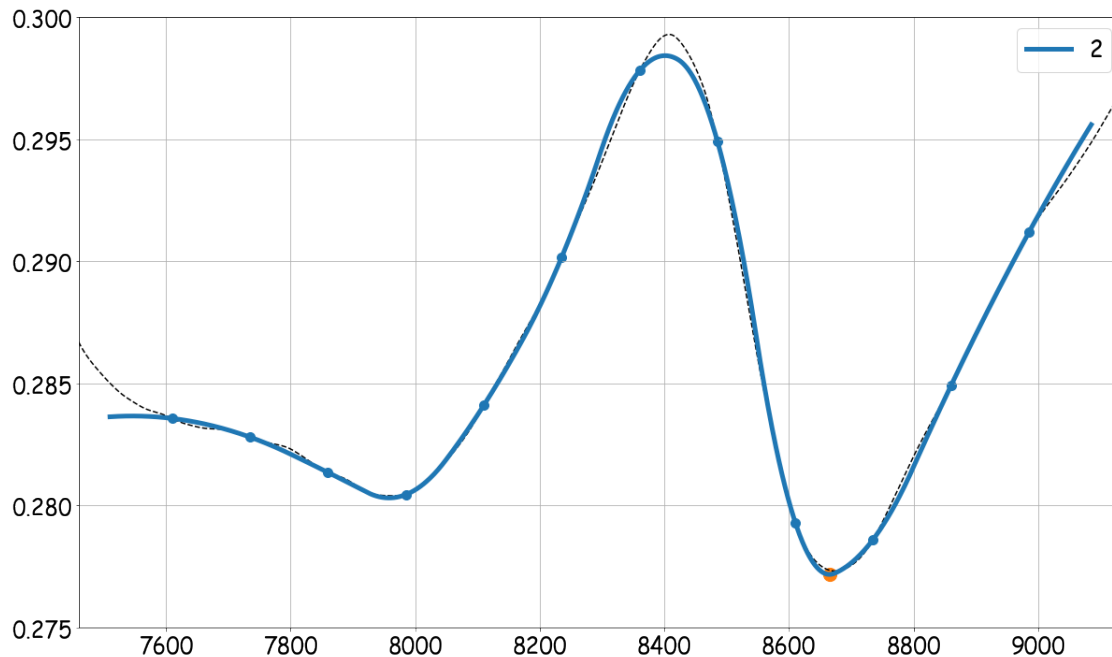
```
fun: array([0.27717336])
hess_inv: <1x1 LbfgsInvHessProduct with dtype=float64>
jac: array([2.22044605e-08])
message: b'CONVERGENCE: NORM_OF_PROJECTED_GRADIENT_<=_PGTOL'
nfev: 24
nit: 3
status: 0
success: True
x: array([8664.98580191])
```

```
[255]: fig = plt.figure(figsize=(20,12)) # figure size
font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
        ↪ font I like
plt.rc('font', **font)

plt.plot(T, c3, '--', c='black') # '--' - dashed, c - colour,
plt.scatter(X, Y, s=100) # s sets size

plt.plot(x, f2(x), lw=5, label='2')
plt.scatter(res.x, f2(res.x), s=200)

plt.legend()
plt.xlim(X[0]-150, X[-1]+150)
plt.ylim(0.275, 0.3)
plt.grid()
```



Note: Should we want to find the maximum, a small addition is needed:

```
[256]: def f22(x):
        return -1*f2(x)
```

```
[257]: res = minimize(f22, x0=0.5*(x[0]+X[-1]), bounds=((X[0], X[-1]),) )
        print(res)
```

```

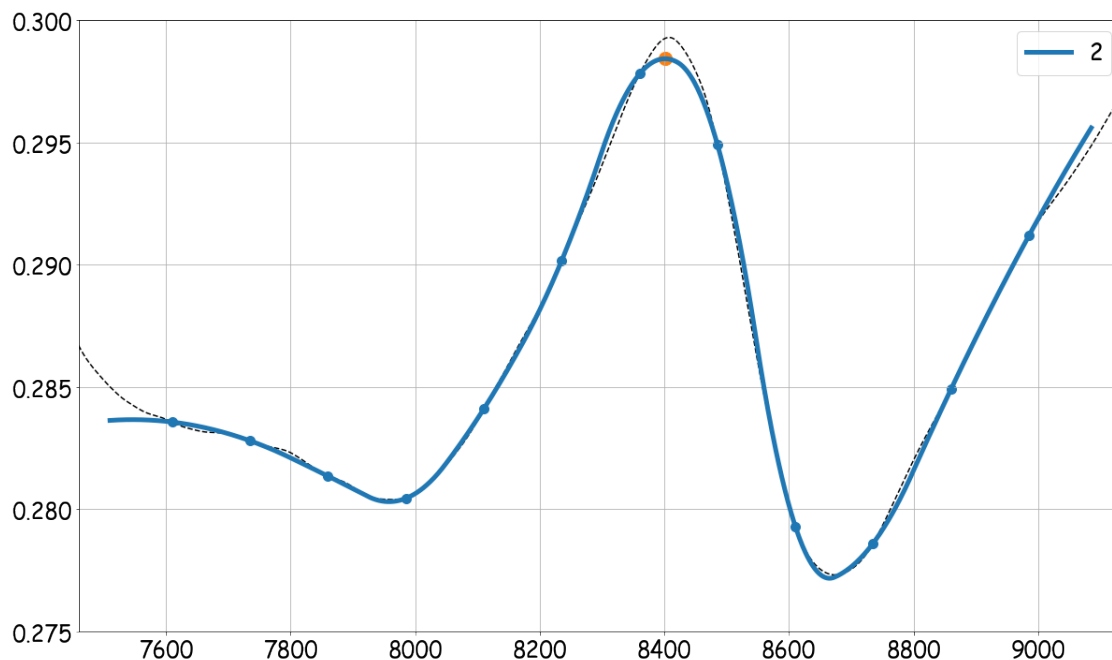
    fun: array([-0.29843369])
 hess_inv: <1x1 LbfgsInvHessProduct with dtype=float64>
   jac: array([1.55431223e-07])
 message: b'CONVERGENCE: NORM_OF_PROJECTED_GRADIENT_<=_PGTOL'
    nfev: 18
     nit: 3
  status: 0
 success: True
     x: array([8401.21539562])
```

```
[258]: fig = plt.figure(figsize=(20,12)) # figure size
font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
      ↪font I like
plt.rc('font', **font)

plt.plot(T, c3, '--', c='black') # '--' - dashed, c - colour,
plt.scatter(X, Y, s=100) # s sets size
```

```
plt.plot(x, f2(x), lw=5, label='2')
plt.scatter(res.x, f2(res.x), s=200)

plt.legend()
plt.xlim(X[0]-150, X[-1]+150)
plt.ylim(0.275, 0.3)
plt.grid()
```



Example: We will now write a piece of code to improve localization of an minimum. Starting with some X and Y arrays that mimic a limited numbers of tests that we have on the actual process represented by x and y we will iterate by finding minima of an interpolating functions, than choosing those as points to probe x and y and inserting values data arrays used for interpolation:

```
[259]: X = T[10000:14000:1000] # We can select every other value from an array!
Y = c3[10000:14000:1000]
x = np.linspace(X[0], X[-1], 10000)
print(len(X), len(Y))
```

4 4

-> iteration starts here <-

```
[272]: f2 = InterpolatedUnivariateSpline(X, Y, k=2)
res = minimize(f2, x0=8600, bounds=((X[0], X[-1])), tol=1e-12)
print(res)
print(res.x[0], f2(res.x[0]))
```

```

    fun: array([0.27669255])
hess_inv: <1x1 LbfgsInvHessProduct with dtype=float64>
jac: array([0.])
message: b'CONVERGENCE: NORM_OF_PROJECTED_GRADIENT_<=_PGTOL'
nfev: 28
nit: 5
status: 0
success: True
      x: array([8671.11900068])
8671.119000675793 0.2766925458000439

```

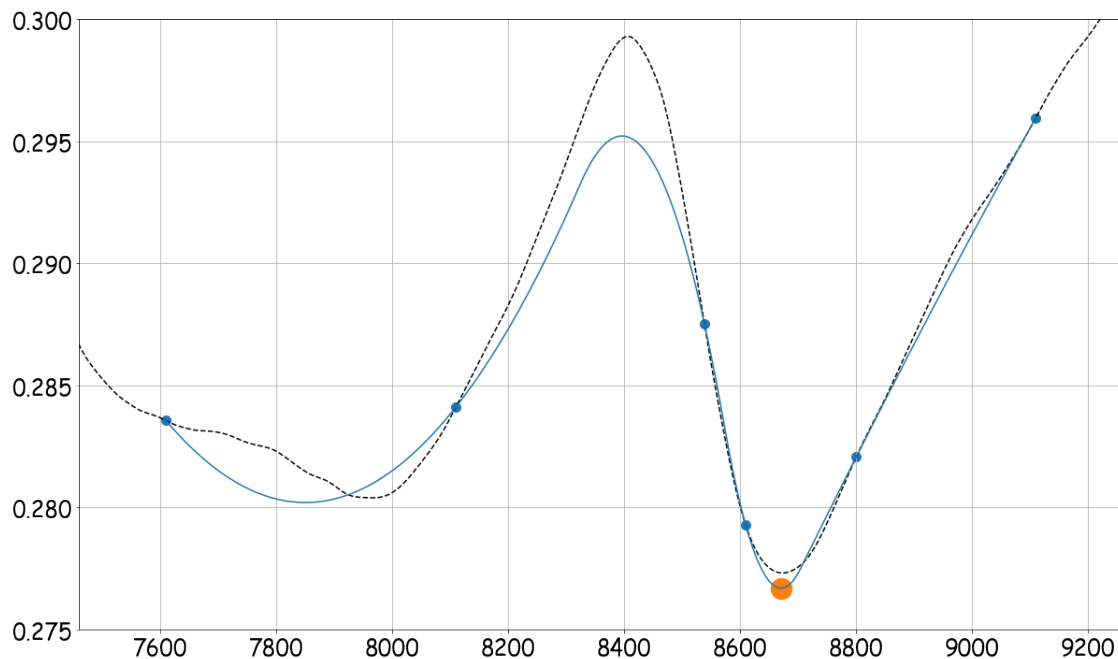
```

[273]: fig = plt.figure(figsize=(20,12)) # figure size
font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
      ↪font I like
plt.rc('font', **font)

plt.plot(T, c3, '--', c='black') # '--' - dashed, c - colour,
plt.scatter(X, Y, s=100) # s sets size
plt.plot(x, f2(x))
plt.scatter(res.x, f2(res.x), s=500)

plt.xlim(X[0]-150, X[-1]+150)
plt.ylim(0.275, 0.3)
plt.grid()

```



We now need to run our process for the value of an argument we determined to result in the

minimum of the interpolating function (if it is the best strategy, that is another story). In our case this is equivalent to finding the values in `T` and `c3` arrays:

```
[274]: idx = np.searchsorted(T, res.x)
newX = T[idx]
newY = c3[idx]
print(idx, res.x, newX, newY)
```

```
[12123] [8671.11900068] [8671.6] [0.27731442]
```

We will use `numpy.insert` to update our `X` and `Y` with newly acquired values. `insert` needs to know the index before which it should insert:

```
[275]: a = np.searchsorted(X, newX)
X = np.insert(X, a, newX)
Y = np.insert(Y, a, newY)
```

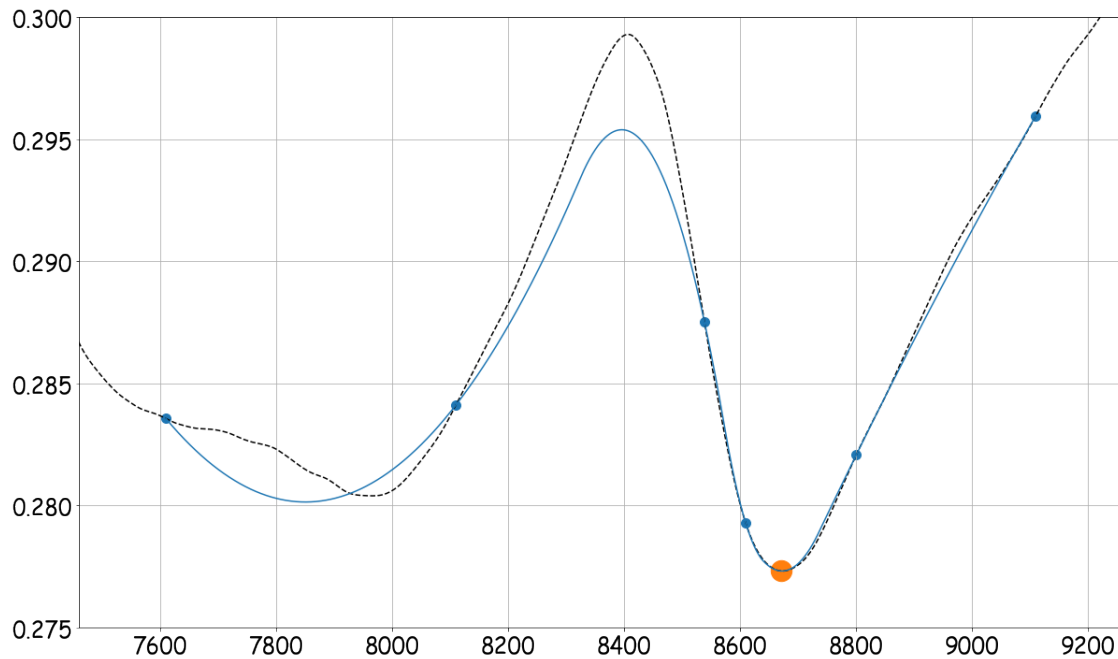
Redo the interpolation:

```
[276]: f2 = InterpolatedUnivariateSpline(X, Y, k=2)
```

```
[277]: fig = plt.figure(figsize=(20,12)) # figure size
font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
      ↳font I like
plt.rc('font', **font)

plt.plot(T, c3, '--', c='black') # '--' - dashed, c - colour,
plt.scatter(X, Y, s=100) # s sets size
plt.plot(x, f2(x))
plt.scatter(res.x, f2(res.x), s=500)

plt.xlim(X[0]-150, X[-1]+150)
plt.ylim(0.275, 0.3)
plt.grid()
```



We can reiterate, or close everything in a nice loop and a procedure.

1.6.3 Roots

As with the others there is a number of procedures, and it is up to you to select the one that is right for your problem.

```
[278]: from scipy.optimize import root_scalar
```

```
[279]: def fun(x):
        return f2(x) - 0.280
```

```
[284]: def fun(x):
        return 2*np.sin(x)-np.cos(x)
```

```
[285]: res = root_scalar(fun, x0=0, x1=np.pi)
```

```
[286]: print(res, res.root, fun(res.root))
```

```

    converged: True
      flag: 'converged'
function_calls: 9
  iterations: 8
      root: 3.6052402625905993 3.6052402625905993 2.220446049250313e-16

```

1.6.4 Curve fitting

Here we show an example of curve fitting using least-square approximation. We try to find parameters of a function that best fits some points:

```
[287]: from scipy.optimize import curve_fit
```

```
[288]: def fun(x, a, b, c):  
        return a*x**5 + b*x**3 + c
```

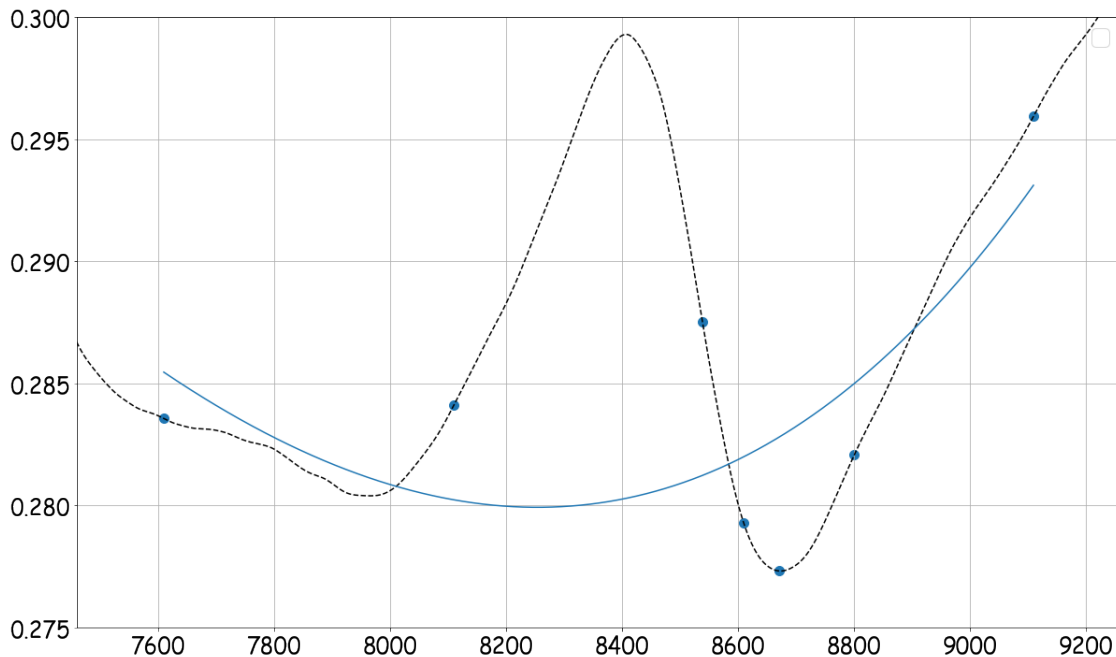
```
[292]: x = np.linspace(X[0], X[-1], 10000)
```

```
[293]: popt, pcov = curve_fit(fun, X, Y)  
        print(popt)
```

```
[ 5.41474979e-21 -6.14903243e-13  4.18260747e-01]
```

```
[295]: fig = plt.figure(figsize=(20,12)) # figure size  
        font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger  
        ↪font I like  
        plt.rc('font', **font)  
  
        plt.plot(T, c3, '--', c='black') # '--' - dashed, c - colour,  
        plt.scatter(X, Y, s=100) # s sets size  
        plt.plot(x, fun(x, a=popt[0], b=popt[1], c=popt[2]))  
  
        plt.legend()  
        plt.xlim(X[0]-150, X[-1]+150)  
        plt.ylim(0.275, 0.3)  
        plt.grid()
```

No handles with labels found to put in legend.



Not the best fit, but well, works and for some real data might be actually the thing.

Zadanie:

Dysponując danymi zawartymi w pliku `data.dat` przedstawiających wartości pewnej funkcji `y` od `x` dla zmieniającego się parametru `Re` oszacuj minimalne `Re` dla którego przebieg wartości `y` jest styczny do osi `x`. Oszacuj też wartość `y` dla którego oszacowałeś to przecięcie. Skorzystaj z metod interpolacji, znajdowania max i min oraz wyszukiwania pierwiastków. Porównaj wyniki uzyskane dla różnych metod interpolacji.

1.6.5 Integration

SciPy provides a number of integration techniques:

```
[296]: from scipy.integrate import quad # general purpose integration/
```

```
[297]: res = quad(f2, a=X[0], b=X[-1])
       print(res[0])
```

```
428.17893493911527
```

1.6.6 Solving a simple ODE

via `from scipy.integrate.ode` an ODE system of the form $x'(t) = f(t, x)$ might be solved.

```
[298]: from scipy.integrate import odeint
```

ODE needs the right hand side function defining derivative `f` for a given state `x` and time `t`.

Let consider a system [Lorenz System](#) (kind of famous - see why), for the evolution of a vector of variables $\mathbf{x} = [x, y, z]$ in time t , according to:

$$\begin{cases} \frac{dx}{dt} = \sigma(y - x) \\ \frac{dy}{dt} = x(\rho - z) - y \\ \frac{dz}{dt} = xy - \beta z \end{cases}$$

With initial conditions $\mathbf{x}(t = 0) = [x_0, y_0, z_0]$.

The function defining the derivative is simply:

```
[299]: sigma=10
      beta=8./3
      rho=28.0

      def lor_rhs(x0, t0):
          return [sigma * (x0[1] - x0[0]),
                  x0[0] * (rho - x0[2]) - x0[1],
                  x0[0] * x0[1] - beta * x0[2]
                  ]

[300]: x0 = [1, 1, 1]
      t = np.linspace(0, 20, 10000) # one thousand time steps
      xt = odeint(lor_rhs, x0, t) # solution is here

[178]: xt

[178]: array([[ 1.          ,  1.          ,  1.          ],
              [ 1.00051639,  1.0519658 ,  0.99672746],
              [ 1.00205113,  1.10388964,  0.99357817],
              ...,
              [13.85478323, 13.4349955 , 34.57312499],
              [13.84437813, 13.22367104, 34.75748977],
              [13.82994989, 13.00793583, 34.93464877]])
```

We will now plot. New thing is the use of `%matplotlib notebook` that allows us to produce an interactive plot:

```
[301]: # %matplotlib notebook

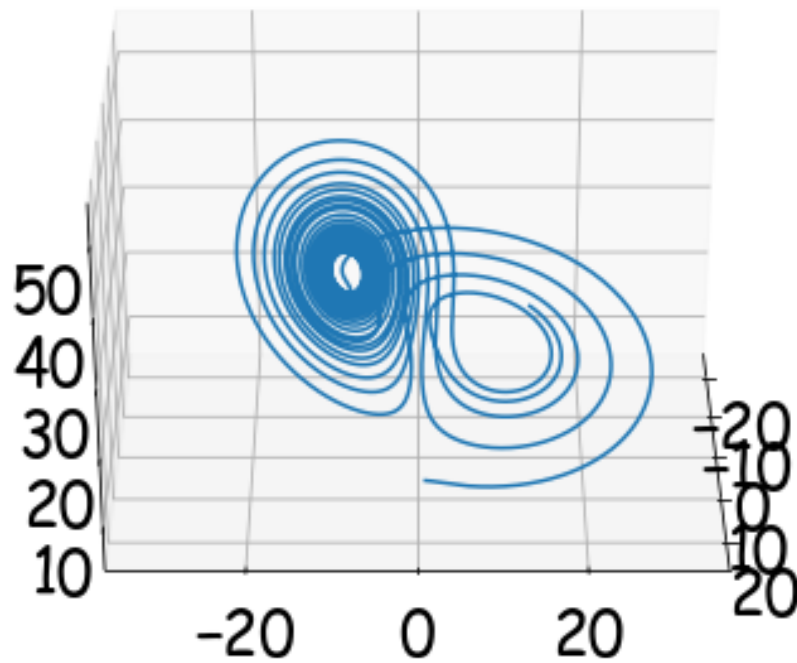
      from matplotlib import pyplot as plt # this one we know
      from mpl_toolkits.mplot3d import Axes3D # Produces 3D axes setting

      fig = plt.figure()
      ax = fig.add_axes([0, 0, 1, 1], projection='3d')

      x, y, z = xt.T # transpose
      ax.plot3D(x, y, z)
```

```
# prepare the axes limits
ax.set_xlim((-25, 25))
ax.set_ylim((-35, 35))
ax.set_zlim((5, 55))

# set point-of-view: specified by (altitude degrees, azimuth degrees)
ax.view_init(30, 0)
```



Zadanie:

Korzystając z metod rozwiązywania ODE spróbuj rozwiązać zagadnienie katastrofy rozrostu populacji Spruce Budworm (rodzaj ćmy zjadającej liście) w lasach wschodnie Kanady. Uproszczony model tego zagadnienia omawia artykuł zamieszczony poniżej. Rozdział w książce o dynamice układów poświęcił mu też Strogatz w *NONLINEAR DYNAMICS AND CHAOS*.

Model składa się z populacji ćmy N , która zmienia się zgodnie z tzw. modelem logistycznym:

$$\dot{N} = RN\left(1 - \frac{N}{K}\right) - p(N)$$

Gdzie R to współczynnik przyrostu naturalnego, K to możliwości podtrzymania populacji w ekosystemie a $p(N)$ to ubytek populacji N w wyniku działania ptaków żywiących się ćmą (predation function). Pomijając p mamy proste, liniowe równanie.

Przyjmujemy model predacji czyli jak ptaki zjadają ćmy w zależności od N :

$$p(N) = \frac{BN^2}{A^2 + N^2}$$

gdzie $A, B > 0$ są pewnymi stałymi. Znaczenie modelu myśliwego jest takie, że gdy populacja ćmy jest niewielka, ptaki szukają pożywienia gdzie indziej, natomiast gdy wzrośnie ona do dużego poziomu, ptaki nie nadążają ze zjadaniem ćmy.

Cały model można przedstawić w formie bezwymiarowej:

$$\dot{x} = rx\left(1 - \frac{x}{k}\right) - \frac{x^2}{1+x^2}$$

Zagadnienie posiada rozwiązanie stacjonarne do którego zmierza z wybranego warunku początkowego. Twoim zadaniem jest zbadanie zachowania tego układu, po osiągnięciu rozwiązania stacjonarnego w zależności od wartości definiujących go parametrów, tj. r i k . Przygotuj wykres powierzchni $x(r, k)$. W końcu spróbuj zmodyfikować zadanie tak, aby r rosło wolno z czasem (wzrost lasu i związana z tym zmiana powierzchni liści). Ma to odpowiadać sytuacji przyrostu powierzchni liści w lesie, zwiększenia pożywienia ćmy i możliwości jej ukrycia się przed ptakami. Wypróbuj różne wartości i spróbuj doprowadzić do katastrofy. Podobne rozumowanie można by zastosować do naszego polskiego kornika.

[]:

**Projekt „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca”
współfinansowany jest ze środków Unii Europejskiej w ramach Europejskiego
Funduszu Społecznego**

**Zadanie 35 pn. „Modyfikacja programu studiów II stopnia na kierunku
Robotyka i Automatyka” realizowane w ramach projektu „NERW 2 PW. Nauka
– Edukacja – Rozwój – Współpraca” współfinansowanego jest ze środków Unii
Europejskiej w ramach Europejskiego Funduszu Społecznego**

Lecture 3 apendix - Spruce Budworm

Stanisław Gepner

Automation of design process with Python

Projekt „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca” współfinansowany jest ze środków Unii Europejskiej w ramach Europejskiego Funduszu Społecznego

Zadanie 35 pn. „Modyfikacja programu studiów II stopnia na kierunku Robotyka i Automatyka” realizowane w ramach projektu „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca” współfinansowanego jest ze środków Unii Europejskiej w ramach Europejskiego Funduszu Społecznego

1 Spruce Budworm

We will now use the tools we have learned to perform a simple study of a certain first order ODE problem. The problem comes from Spruce Budworm that has a tendency to periodically eat up fir tree forests in eastern Canada. The original research comes from Ludwig 1976 available here: <https://www.jstor.org/stable/3939?seq=1> but is also discussed in a very good book by Strogatz “Nonlinear Dynamics and Chaos”.

1. Examine the problem. Try to determine fixed points of the solution.
2. Solve the ODE for different initial conditions. Check what the solution approaches to.
3. Prepare the solution surface as a function of system parameters and determine “refuge”, “bistable” and “outbreak” states. Try out different initial conditions.
4. Examine evolution of the system as parameters are slowly varied in time.
5. What should we do with the bugs?

```
[1]: import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D # Produces 3D axes setting
from scipy.integrate import odeint
from matplotlib import cm
```

```
[2]: def lor_rhs(x0, t0):
    return [ r * x0[0] * (1.0-x0[0]/k) - x0[0]**2 / (1.0+x0[0]**2)]
```

We could just get on and solve the equation. It seems to be very simple! BUT let's start by having a look at it and find possible fixed points. That is points such that $\dot{x} = 0$ so the solution does not change, since the time derivative is zero. This is simpler than one might think, as those correspond to intersections of the two parts of the model equation. So let plot the:

$$f(x) = r(1.0 - \frac{x}{k})$$

and

$$p(x) = \frac{x}{1.0 + x^2}$$

(x can be divided by). We will select some values of r and k :

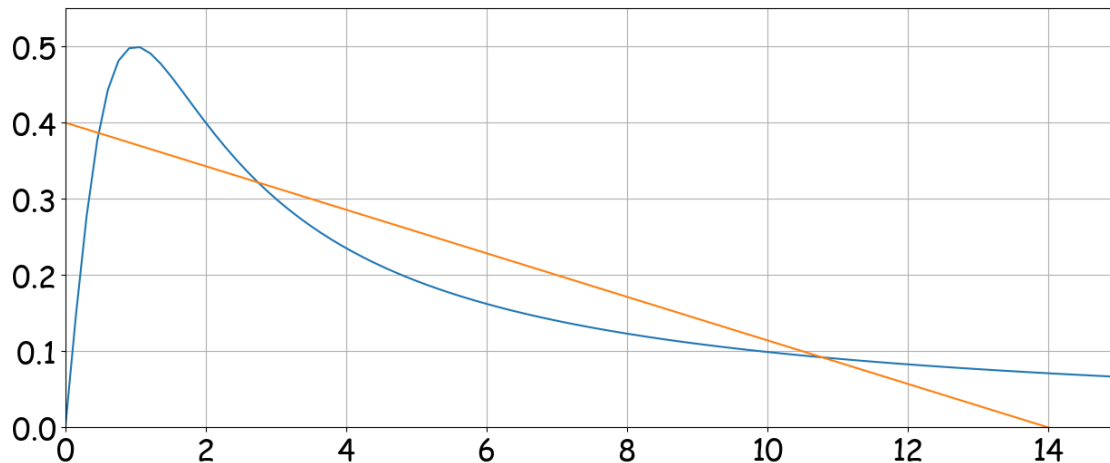
```
[3]: # need to select some values - see the plot below to get a glimpse at the
    ↪ meaning of parameters:
r = 0.4
k = 14
```

```
[4]: fig = plt.figure(figsize=(15,6)) # figure size
font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
    ↪ font I like
plt.rc('font', **font)

x = np.linspace(0, 15, 100)
plt.plot(x, x/(1+x**2))
```

```
plt.plot(x, r*(1-x/k))

plt.ylim(0,0.55)
plt.xlim(0,15)
plt.grid()
```



For this particular choice of parameters we get three intersection points, we can estimate them with:

```
[5]: from scipy.optimize import root_scalar
```

```
[6]: def fun(xx):
      val = lor_rhs([xx], 0)
      return val[0] # we note equation is not explicitly dependant on time
```

```
[7]: res1 = root_scalar(fun, x0=0.1, x1=2)
      res2 = root_scalar(fun, x0=2, x1=4)
      res3 = root_scalar(fun, x0=10, x1=12)
```

```
[8]: print(res2)
```

```

      converged: True
      flag: 'converged'
function_calls: 6
iterations: 5
      root: 2.7457280539128814
```

So we can perform an improved plot:

```
[9]: fig = plt.figure(figsize=(15,6)) # figure size
      font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger font I like
      plt.rc('font', **font)
```

```

x = np.linspace(0, 15, 100)
plt.plot(x, x/(1+x**2))
plt.plot(x, r*(1-x/k))

x = res1.root
y = x/(1+x**2)
print(x, y)
plt.scatter(x, y)
x = res2.root
y = x/(1+x**2)
print(x, y)
plt.scatter(x, y)
x = res3.root
y = x/(1+x**2)
print(x, y)
plt.scatter(x, y)

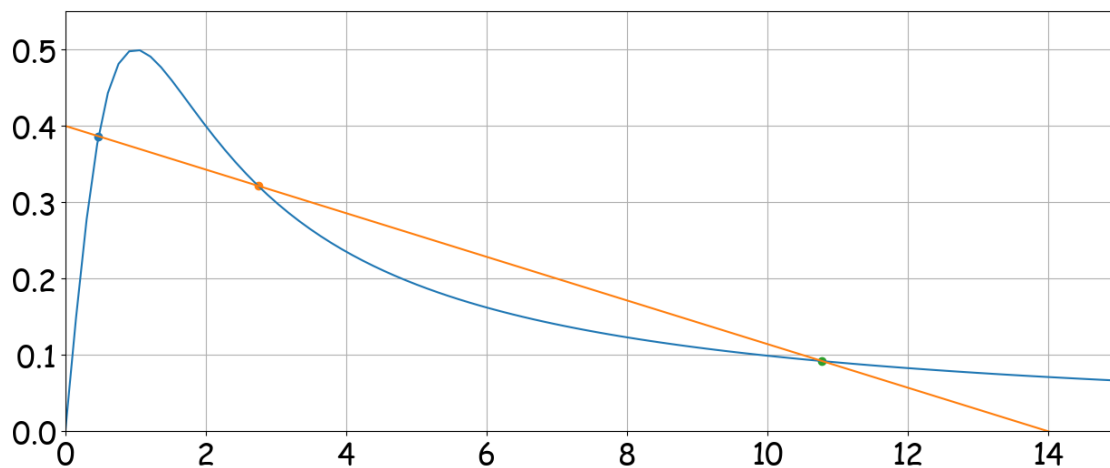
plt.ylim(0,0.55)
plt.xlim(0,15)
plt.grid()

```

```

0.4729309453155196 0.38648768727669947
2.7457280539128814 0.3215506270310605
10.781341000771599 0.09196168569224002

```



Now examine the sign of the left hand of the ODE and decide which of those three points can pose as a stable solution. All comes down to estimating the sign and deciding should x increase or decrease?

Now let move on, solve the problem and examine if points we determined are attainable depending on the initial condition?

```
[10]: t = np.linspace(0, 50, 10000) # one thousand time steps

x0 = 2
xt1 = odeint(lor_rhs, x0, t)

x0 = 4
xt2 = odeint(lor_rhs, x0, t)

print(xt1[-1], xt2[-1])
```

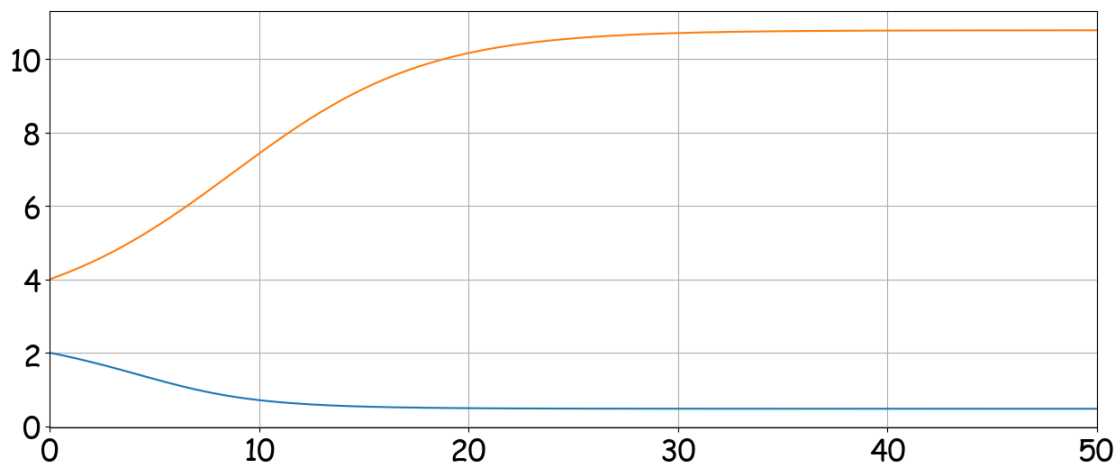
[0.47293762] [10.7803549]

```
[11]: %matplotlib inline
f = plt.figure(figsize=(15,6)) # figure size
font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
    ↪font I like
plt.rc('font', **font)

plt.plot(t, xt1)
plt.plot(t, xt2)

plt.grid()
plt.xlim(0,50)
```

[11]: (0.0, 50.0)



We note the two points “on the outsides” seem like stable solutions. That is solution approaches those in time. How about the middle point? Is it stable? No, it is not, any small deviation will send the system to either of the stable points. We could try this:

```
[12]: %matplotlib inline
f = plt.figure(figsize=(15,6)) # figure size
font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
    ↪font I like
plt.rc('font', **font)

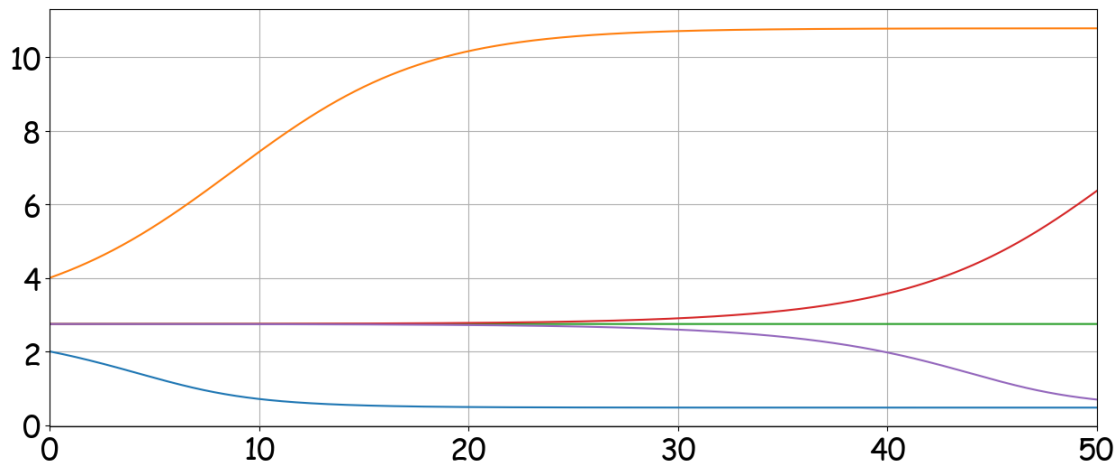
x0 = res2.root
xt3 = odeint(lor_rhs, x0, t)

x0 = res2.root+1e-3
xt3e1 = odeint(lor_rhs, x0, t)
x0 = res2.root-1e-3
xt3e2 = odeint(lor_rhs, x0, t)

plt.plot(t, xt1)
plt.plot(t, xt2)
plt.plot(t, xt3)
plt.plot(t, xt3e1)
plt.plot(t, xt3e2)

plt.grid()
plt.xlim(0,50)
```

[12]: (0.0, 50.0)



But any small disturbance will cause the solution to deviate.

We can now attempt to plot the solution surfaces. We could solve the ODE for each configuration, or just look for roots.

```
[13]: r = np.linspace(0.1, 1, 5)
      k = np.linspace(0.1, 20, 5)
      R,K = np.meshgrid(r,k)
```

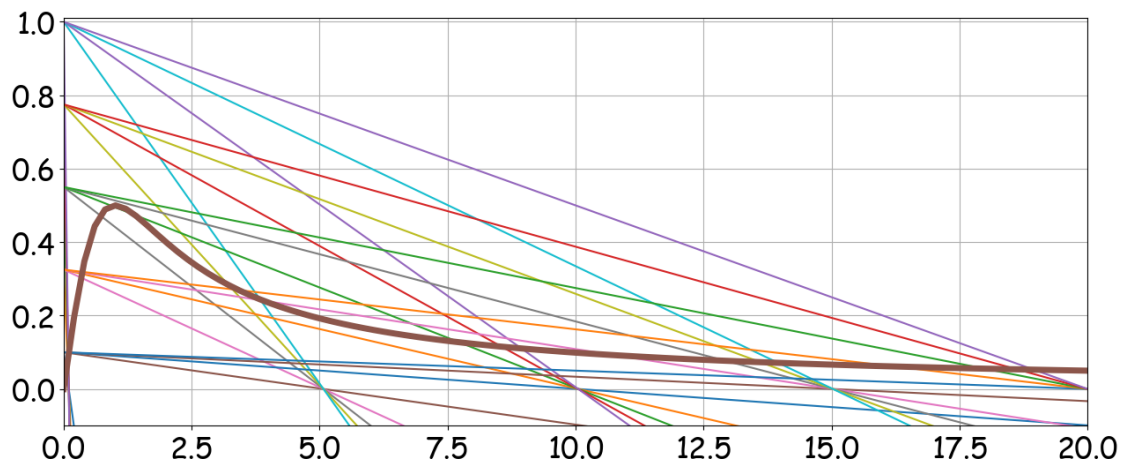
```
[14]: fig = plt.figure(figsize=(15,6)) # figure size
      font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
      ↪font I like
      plt.rc('font', **font)

      x = np.linspace(0, 20, 100)

      sh = R.shape
      for i in range(0,sh[0]):
          for j in range(0,sh[1]):
              r = R[i,j]
              k = K[i,j]
              if k == 0:
                  continue
              plt.plot(x, r*(1-x/k))

      plt.plot(x, x/(1+x**2), lw=5)

      plt.ylim(-0.1,1.01)
      plt.xlim(0,20)
      plt.grid()
```



```
[15]: r = np.linspace(0.1, 1, 5)
      k = np.linspace(0.1, 20, 5)
      R,K = np.meshgrid(r,k)
```

```
sh = R.shape
```

```

for i in range(0,sh[0]):
    for j in range(0,sh[1]):
        r = R[i,j]
        k = K[i,j]
        if k == 0:
            continue
        res1 = root_scalar(fun, x0=0, x1=2)
        res2 = root_scalar(fun, x0=2, x1=5)
        res3 = root_scalar(fun, x0=5, x1=20)

```

But deciding which solution to pick is tricky.

```

[16]: r = np.linspace(0.01, 1.0, 50)
      k = np.linspace(0.01, 20, 50)
      R,K = np.meshgrid(r,k)

      t = np.linspace(0, 200, 1000)  # one thousand time steps
      sh = R.shape
      X1 = np.zeros_like(R)
      X2 = np.zeros_like(R)
      print(R.shape)

```

(50, 50)

```

[17]: for i in range(0,sh[0]):
      for j in range(0,sh[1]):
          r = R[i,j]
          k = K[i,j]

          x0 = 2
          xt1 = odeint(lor_rhs, x0, t)
          X1[i,j] = xt1[-1]

          x0 = 4
          xt2 = odeint(lor_rhs, x0, t)
          X2[i,j] = xt2[-1]
      #      print(i, j, xt1[-1], xt2[-1])

```

```

[18]: from mpl_toolkits.mplot3d import Axes3D # Produces 3D axes setting
      from matplotlib import cm

      f = plt.figure(figsize=(5,5)) # figure size
      font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
      ↪font I like
      plt.rc('font', **font)

      ax = Axes3D(f)

```

```

surf = ax.plot_surface(R, K, X2, color='red', alpha=1)
surf = ax.plot_surface(R, K, X1, color='blue', alpha=0.5)

plt.show()

```

<Figure size 500x500 with 0 Axes>

So we can produce a two-surface plot and illustrate different solutions depending on the initial condition. Now time for a catastrophe to happen. We will evolve the spruce population in time and assume that conditions of the habitat change slowly in time. That is, as the time goes by, forest grows, there is more leaves to feed on and to hide in. This will translate to a relatively slow change of the habitat carrying capacity k . We will fix $r = 0.4$ and vary k from 5 up:

```

[19]: x0 = 0.5
      r = 0.6
      k = 5
      t = np.linspace(0, 200, 1000)
      xt = odeint(lor_rhs, x0, t)

```

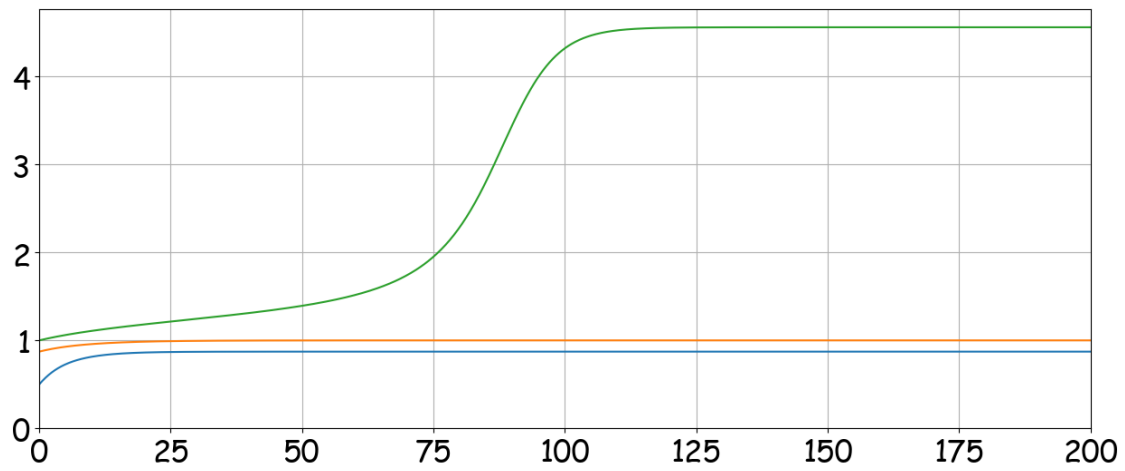
```

[20]: %matplotlib inline
fig = plt.figure(figsize=(15,6)) # figure size
font = {'family' : 'Comic Sans MS', 'weight' : 'normal', 'size' : 24} #larger
    ↪font I like
plt.rc('font', **font)

x0 = 0.5
for k in [5,6,7]:
    xt = odeint(lor_rhs, x0, t)
    plt.plot(t, xt)
    x0 = xt[-1]

plt.ylim(0)
plt.xlim(0,200)
plt.grid()

```



**Projekt „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca”
współfinansowany jest ze środków Unii Europejskiej w ramach Europejskiego
Funduszu Społecznego**

**Zadanie 35 pn. „Modyfikacja programu studiów II stopnia na kierunku
Robotyka i Automatyka” realizowane w ramach projektu „NERW 2 PW. Nauka
– Edukacja – Rozwój – Współpraca” współfinansowanego jest ze środków Unii
Europejskiej w ramach Europejskiego Funduszu Społecznego**

Lecture 4 - Calling external programs

Stanisław Gepner

Automation of design process with Python

Projekt „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca” współfinansowany jest ze środków Unii Europejskiej w ramach Europejskiego Funduszu Społecznego

Zadanie 35 pn. „Modyfikacja programu studiów II stopnia na kierunku Robotyka i Automatyka” realizowane w ramach projektu „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca” współfinansowanego jest ze środków Unii Europejskiej w ramach Europejskiego Funduszu Społecznego

1 Lecture 4

We will now look at ways to run a computational process and monitor it as it performs the assigned task. At some moment our algorithm will decide that solution progressed enough to make some decision and will restart it with parameters changed according to what we programmed. The task itself, the communication and the process will vary greatly from case to case, but in general we will need to:

- * Start an external application with proper command line arguments.
- * This can be done either in a blocking (the thread serving our Python console will not be accessible, and will lock for the execution of the called program)
- * Or in a nonblocking way, by spawning a separate process to deal with the problem, while our Python console will return control for us to perform various tasks.
- * Read data from a file to which the program stores the interesting value.
- * Modify settings of the program, either via command line arguments or a setting file.
- * Make decisions on what to do next.

Note: I use more less this approach to (attempt) track the edge that possibly exists between the laminar and turbulent solution, by iteratively changing perturbation applied to the flow end examining its evolution in time, on the way deciding if the flow is going turbulent or not. The process is very much like trying to push the ball to land on the top of a hill, push it to hard and it passes to the other side, be to gentle and it will not reach the top. But in an infinite number of dimensions (before discretization).

1.1 The model program

For a given task you will need to use a specific program. For me this is a flow solver, for you it might be ANSYS or ADAMS or something completely different. For the purpose of this presentation we will use a simple C++ program, attached to this repository. (This might not be working with MyBinder though). This program:

- * stores a random value from a selected range to a file - our data.
- * accepts some simple command line arguments.
- * Reads a simplistic setting file.
- * The source is in the tick-tick folder.

1.2 Use `os.system` call

We start by simply calling the system command, in our case our program. To do this we need to import `os` module:

```
[1]: import os
```

```
[2]: ls
```

'Lecture 4.ipynb' [tick-tick/](#)

Our test program will be a simple program. The source is given here for the sake of completeness:

```
[1]: cat ../tick-tick/main.cpp
```

```
#include <iostream>
#include <fstream>
#include <chrono>
#include <thread>
#include <filesystem>
#include <algorithm>
```

```

#include <random>

using namespace std;

int main(int argc, char *argv[])
{
    // Settings strings
    string period_option = "--period";
    string sett_option = "--sett";

    // Settings values
    int period = 1000;
    string sett_file = "";

    // Values read from filw
    double dt = 0.1; // time step
    double xmax = 1.0; // max
    double xmin = -1.0; // min
    int niter = 10; // maximum number of generated values

    // parsing command line args
    for(int i=0; i<argc; ++i)
    {
        if(period_option.compare(argv[i]) == 0) // period
            period = stoi(argv[i+1]); // convert to int
        if(sett_option.compare(argv[i]) == 0) // setting file
        {
            sett_file = argv[i+1];
            if(filesystem::exists(sett_file)) // file exists, read settings
            {
                ifstream myfile(sett_file); // open file as an input stream
                std::string line;
                while( getline(myfile, line) ) // go through all lines
                {
                    if(line[0] == '#') continue; // skip comments
                    if(line.find_first_of('=') != string::npos) // consider only
if there is a '='
                {
                    istringstream is_line(line); // to use with getline
                    std::string key, value;
                    getline(is_line, key, '='); // first, get the key
                    getline(is_line, value, '='); // get the value
                    key.erase (remove (key.begin(), key.end(), ' '),
key.end()); // get rid of whitespaces
                    if(key.compare("DT") == 0) dt = stod(value); // string
to double, whitespaces removed
                    else if(key.compare("XMAX") == 0) xmax = stod(value);
                    else if(key.compare("XMIN") == 0) xmin = stod(value);

```

```

        else if(key.compare("N") == 0) niter = stoi(value);
        else { cout << key << " " << " with vaue " << value << "
is not regcognized" << '\n'; }
    }
}
myfile.close();
}
else cout << "Setting file " << sett_file << "does not exist, usin
defaults" << '\n';
}
}

// sum up the settings
cout << "==== Starting process =====" << '\n';
cout << "Period:" << period << '\n';
cout << "Dt:\t" << dt << '\n';
cout << "Xmax:\t" << xmax << '\n';
cout << "Xmin:\t" << xmin << '\n';
cout << "N:\t" << niter << '\n';

// File to store to
ofstream myfile; // open output stream
myfile.open ("data.dat"); // file to store results
myfile << "# Recording precious random data\n"
    << "# t, value" << '\n'
    << "# Settings: " << period << " " << dt << " " << xmax << " " <<
xmin << " " << niter << '\n';
myfile.flush();

// The "computational" process!
uniform_real_distribution<double> unif(xmin, xmax);
default_random_engine re;

for(int i = 0; i < niter; ++i)
{
    this_thread::sleep_for(chrono::milliseconds(period)); // a nice do
nothing part
    double random_double = unif(re);
    myfile << i*dt << ",\t " << random_double << '\n';
    myfile.flush();
}

// Tidy up
myfile.close();
return 0;
}

```

```
[4]: #os.system('./tick-tick/build_debug/tick-tick')
os.system('./tick-tick/build_debug/tick-tick --period 10')
```

[4]: 32512

This executes `tick-tick` program, the working directory (place where work is performed is) is the same as our current working directory. As a result `data.dat` should appear. The file content is:

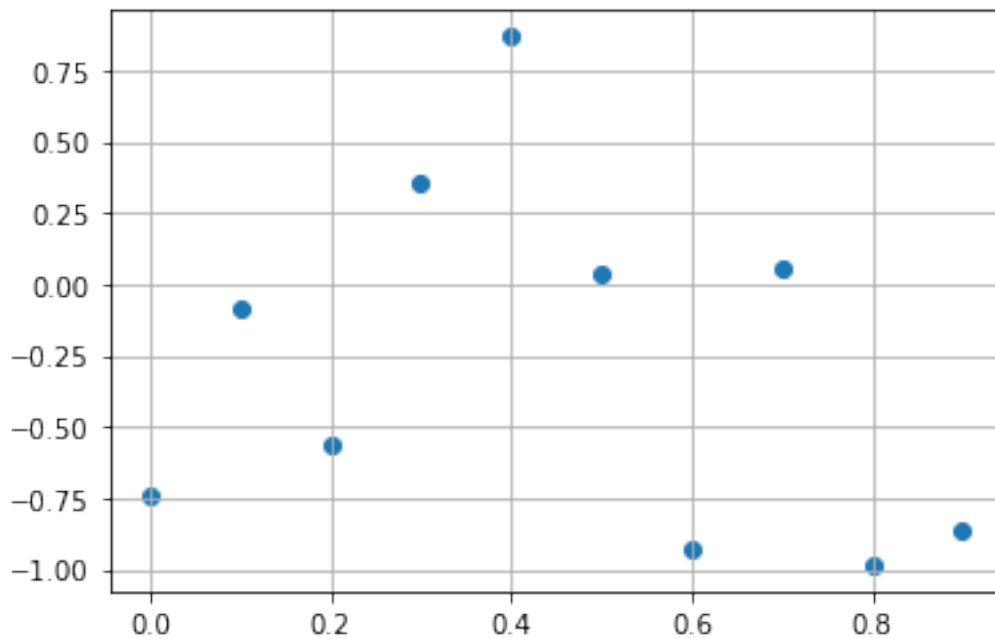
```
[5]: cat data.dat
```

cat: data.dat: No such file or directory

```
[21]: import numpy as np
import matplotlib.pyplot as plt
```

```
[6]: t, v = np.loadtxt('data.dat', comments="#", skiprows=2, usecols=(0,1),
↳unpack=True, delimiter=',')
```

```
[7]: plt.scatter(t, v)
plt.grid()
```



Before proceeding lets add setting file we can manipulate:

```
[12]: f = open('./settings', 'w')
f.writelines('DT = 0.5\n')
f.write('XMAX = 5.0\n')
```

```
f.write('XMIN = -5.0\n')
f.write('N    = 1000\n')
f.close()
```

```
[13]: cat settings
```

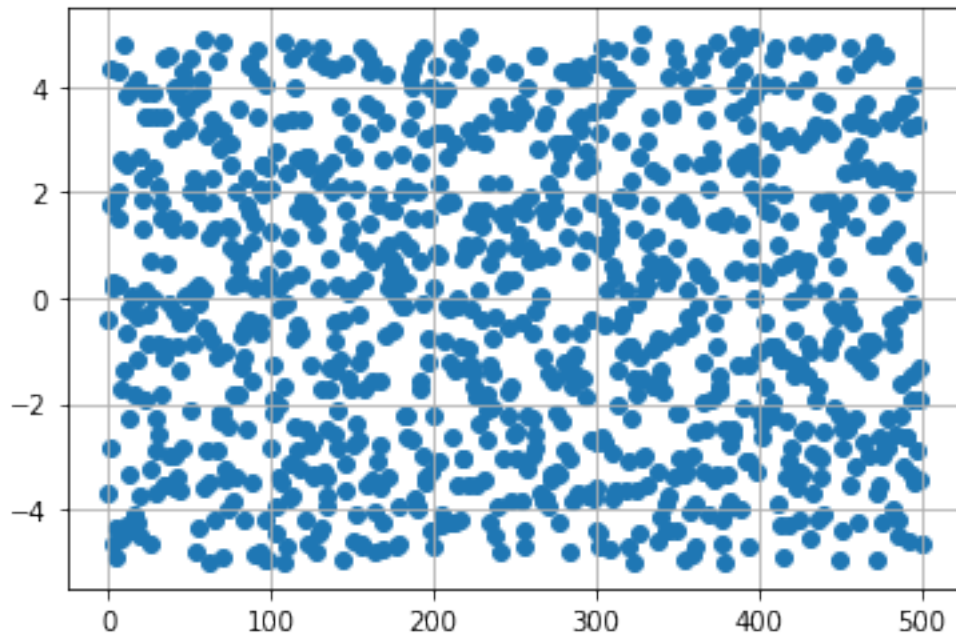
```
DT    = 0.5
XMAX  = 5.0
XMIN  = -5.0
N     = 1000
```

Now we have a settings file to be passed to our process.

```
[14]: os.system('./tick-tick/build_debug/tick-tick --period 1 --sett settings')
```

```
[14]: 0
```

```
[15]: t, v = np.loadtxt('data.dat', comments="#", skiprows=2, usecols=(0,1),
    ↪unpack=True, delimiter=',')
plt.scatter(t, v)
plt.grid()
```



1.3 subprocess

1.3.1 Blocking call:

Ultimately we would like to run the program and regain control. To this end we will use the `subprocess` module and a couple of `os` tools. The `subprocess` module allows to create new processes, obtain their returns and outputs. We will show only some of the functionality. Let's start with getting the output with `check_output`, that we can later parse and use:

```
[7]: import subprocess
```

```
[8]: s = subprocess.check_output(["ls", "-l"])
      print(s)
```

```
b'total 156\n-rw-rw-r-- 1 sgepner sgepner 153959 kwi 20 16:57 Lecture
4.ipynb\n-drw-rw-r-x 3 sgepner sgepner 4096 kwi 21 10:27 tick-tick\n'
```

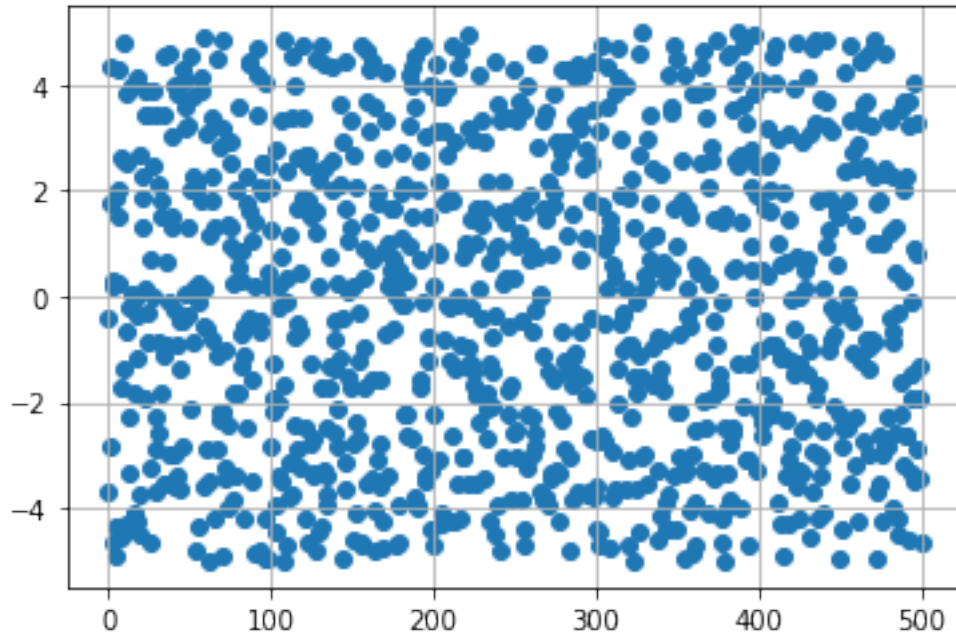
For us, more useful is `call()`. It runs command passed as argument and waits (locks control) until command completes.

```
[19]: subprocess.call("tick-tick/build_debug/tick-tick --period 50 --sett settings",
      ↪shell=True)
```

```
[19]: 0
```

Note the additional `shell=True` argument. This specifies that the command will be executed through the system shell (eg. Bash) and is useful if we need to pass arguments. We can illustrate the result:

```
[43]: t, v = np.loadtxt('data.dat', comments="#", skiprows=2, usecols=(0,1),
      ↪unpack=True, delimiter=',')
      plt.scatter(t, v)
      plt.grid()
```



Example: Using the `tick-tick` program and the `call` function write a script that runs the `tick-tick` changing the number of iterations `N` and plots average and variance as functions of `N`.

1. Write a function that modifies the setting file accordingly.
2. Write a function that reads the result file and stores the results.
3. Run the process in a loop, choose some stopping criteria.

```
[21]: def settfile(fname, N):
    '''
    Modifies the setting file
    param: fname - file to modify
    param: N - nuber of iteration do perform
    '''
    f = open(fname, 'w')
    f.writelines('DT = 0.5\n')
    f.write('XMAX = 5.0\n')
    f.write('XMIN = -5.0\n')
    st = 'N = ' + str(N) + '\n'
    f.write(st)
    f.close()
```

```
[46]: def calculate(fname):
    '''
    Using fname calculates average and variance
    '''
```

```

    t, v = np.loadtxt(fname, comments="#", skiprows=2, usecols=(0,1),
↳unpack=True, delimiter=',')
    s = v.sum()
    avg = s / len(t)
    s = 0
    for val in v:
        s += (val - avg)**2
    var = s / len(t)
    return len(t), avg, var

```

```

[56]: '''
      Run the process here
      '''
n = []
av = []
for N in range(5,1000,5):
    st = 'setting'
    settfile(st, N)
    #     print('Running for N=', N)
    subprocess.call("tick-tick/build_debug/tick-tick --period 0 --sett setting",
↳shell=True)
    a, v = calculate('data.dat')
    n.append(N)
    av.append(a)
    #     print(a, v)

```

```

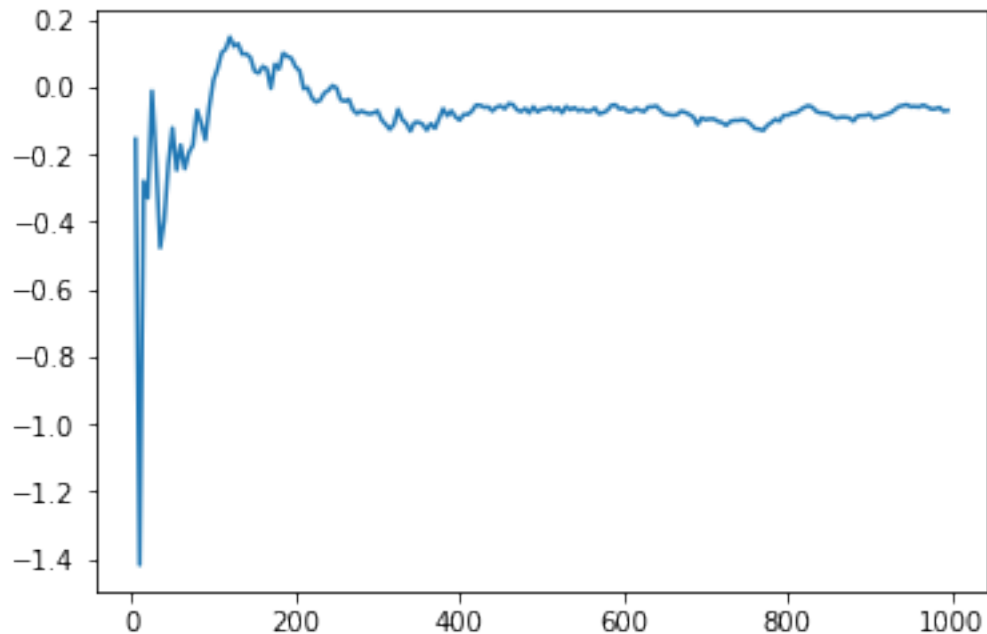
[57]: plt.plot(n, av)

```

```

[57]: [<matplotlib.lines.Line2D at 0x7fb3edc943a0>]

```



1.3.2 Run and regain control with `popen()`

We will now look at a slight modification. Our goal is to be able to monitor (programmatically) the process we call and if necessary make decision, change parameters and eventually stop execution or restart. To do this we need to add a couple of things to our tool set. First is the `subprocess.Popen()`, which works very much like `call()`, with a difference that it executes the child (mercy!) in a new process! What that means is that new, separate from our Python interpreter, process is started to handle the program, and control is returned to the interpreter to perform whatever task we wish it to do. When called `Popen` object is returned. We will outline some of functionality it provides (also see the documentation).

The most important feature is that `Popen()` allows a nonblocking call!

```
[1]: import subprocess
```

Note: It is good to show the process tree at this moment.

```
[23]: p=subprocess.Popen('tick-tick/build_debug/tick-tick --period 100 --sett_
↳ settings', shell=True)
```

`p` is going to be a `Popen` with which we (or our smart algorithm) can interact. There is a number of things we could do. The most important is to check if the child process is alive (`Popen.poll()`), wait for it to finish with (`Popen.wait(timeout)`), or send it a signal for termination or kill (`Popen.terminate()` `.kill()`). There is also an option to interact with the process passing data to `stdin` and to read from `stdout` with `Popen.communicate()`, but our program does not support this (you can try it out).

Examine: * p.poll() * p.wait() * p.pid * p.terminate() and p.kill()

```
[10]: print(p)
```

<subprocess.Popen object at 0x7f3ea84f3c10>

```
[19]: p.terminate()
```

```
[16]: try:
      print(p.wait(timeout=10))
      except:
      print('some error!')
```

some error!

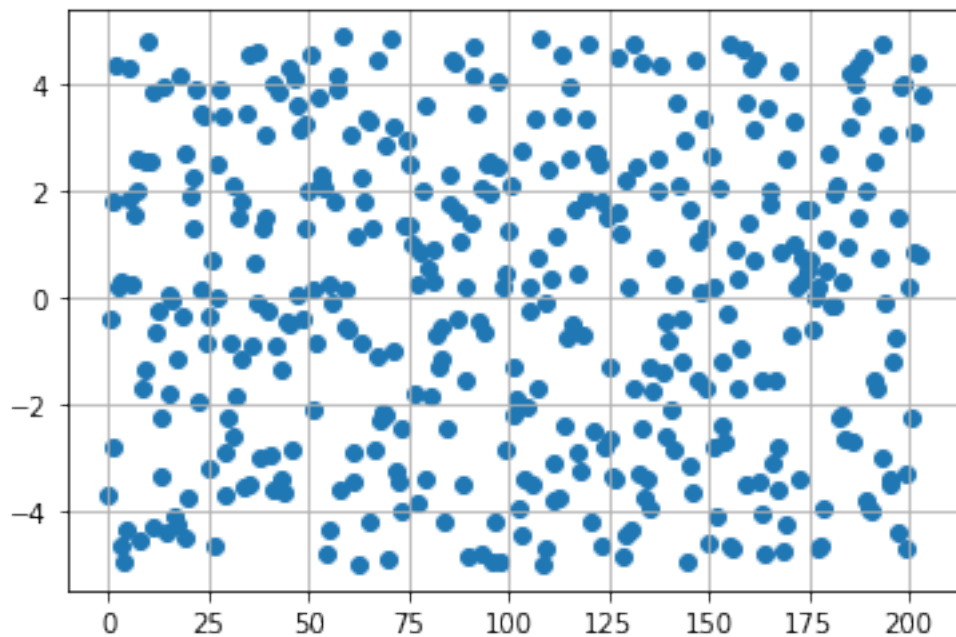
```
[17]: print(p.returncode)
```

None

```
[18]: print(p.pid)
```

3157917

```
[30]: t, v = np.loadtxt('data.dat', comments="#", skiprows=2, usecols=(0,1),
      ↪unpack=True, delimiter=',')
      plt.scatter(t, v)
      plt.grid()
```



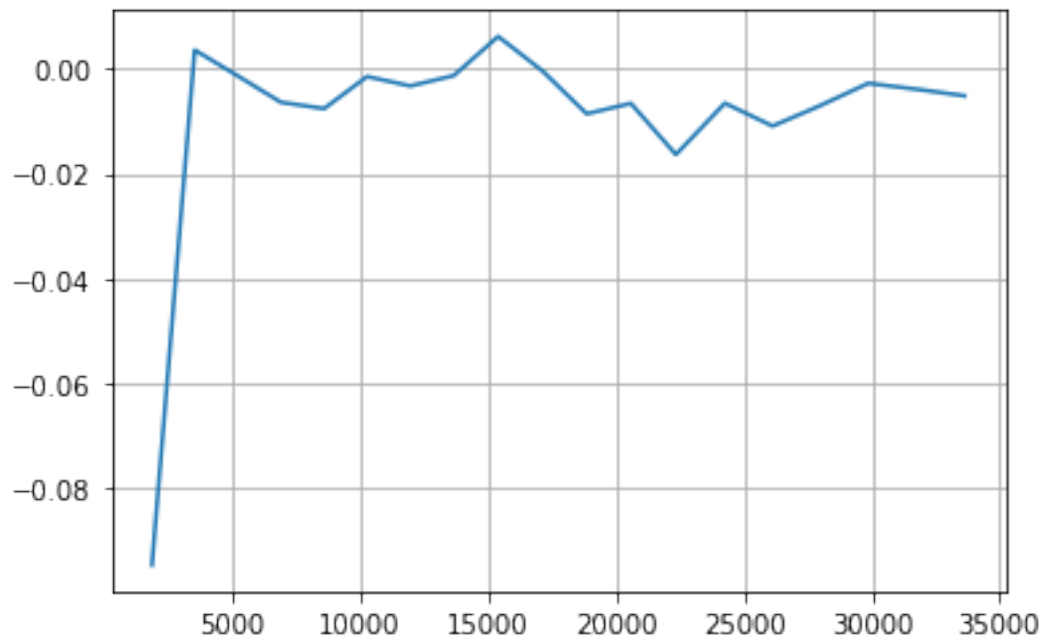
```
[33]: import time
```

```
[48]: '''  
      Run the process here  
      '''  
      p=subprocess.Popen('tick-tick/build_debug/tick-tick --period 1 --sett settings',  
                           ↪shell=True)
```

```
[49]: nn = []  
      av = []  
      while True:  
          if p.poll() != None:  
              print(p.pid, 'Is not working')  
              break  
          time.sleep(2)  
  
          n, a, v = calculate('data.dat')  
          nn.append(n)  
          av.append(a)  
          print(n, a, v)  
          if n > 3e4:  
              p.terminate()  
              print(p.pid, 'is now dead')  
  
      plt.plot(nn, av)  
      plt.grid()
```

```
1853 -0.09466136119266054 8.1787814398507  
3523 0.003640004419528826 8.339356039806566  
5197 -0.0012680806965557087 8.384304401072745  
6893 -0.006334093036413753 8.44479097993249  
8584 -0.007548869544501388 8.368534248803416  
10265 -0.0014202789108621456 8.308905288880178  
11953 -0.003220258300844966 8.29689039244658  
13654 -0.001246310110736774 8.281696472505564  
15391 0.00623577575714379 8.261625307929208  
17113 -0.00031409062337403685 8.240807246883527  
18852 -0.008540249606715462 8.218880373412189  
20579 -0.006544184480091349 8.236289048549509  
22340 -0.016313612861942696 8.270705500994046  
24255 -0.006511834050950318 8.25710280940719  
26121 -0.010877890422870483 8.25848221886428  
27918 -0.007115430406169137 8.241168313505199  
29858 -0.002715046840928058 8.252513093975  
31847 -0.003905710262079 8.264599524799124  
3166988 is now dead  
33628 -0.005111306575664025 8.277621084602673
```

3166988 is now dead
3166988 Is not working



[]:

**Projekt „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca”
współfinansowany jest ze środków Unii Europejskiej w ramach Europejskiego
Funduszu Społecznego**

**Zadanie 35 pn. „Modyfikacja programu studiów II stopnia na kierunku
Robotyka i Automatyka” realizowane w ramach projektu „NERW 2 PW. Nauka
– Edukacja – Rozwój – Współpraca” współfinansowanego jest ze środków Unii
Europejskiej w ramach Europejskiego Funduszu Społecznego**

Lecture 5 - Data serialization and external API

Stanisław Gepner

Automation of design process with Python

Projekt „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca” współfinansowany jest ze środków Unii Europejskiej w ramach Europejskiego Funduszu Społecznego

Zadanie 35 pn. „Modyfikacja programu studiów II stopnia na kierunku Robotyka i Automatyka” realizowane w ramach projektu „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca” współfinansowanego jest ze środków Unii Europejskiej w ramach Europejskiego Funduszu Społecznego

1 Lecture 5

Serialization of data and use of external application API (Application Programmable Interface)

Often Python scripts are used to process data in consequence creating new data for further use. The processing part can itself be computationally involving and being able to store data in the easily accessible form is very convenient. Especially so if the processing part takes days (but even hours) and access to data is required at some latter stages of the project. One, obvious solution is to store resulting data in simple CSV type files, but this approach is limited, especially if data structure becomes complex. For example, we might be storing time history of velocity profiles (u, v, w) through a flow field retrieved from a number of locations and performed across snapshots in time In general we have a lot of complex data to deal with, and would like to store it in some organized manner, to be retrieved and worked on latter. For this we can use **serialization**, which in the simplest terms means to store data. The concept comes from the need to communicate different application forming a system and working possibly across different platforms (or simply form game saves ...).

Here we will illustrate how to **store** and **retrieve** data we process and produce. Consider a process where we have a number of snapshots from computations (data could be data from experiments) describing some arbitrary vector field in time (or positions of a robot, some mappings of its surroundings, readings from sensors etc.). We will store such data within objects of the following class:

```
[1]: import numpy as np
      from matplotlib import pyplot as plt

[2]: class Case:
      def __init__(self, fname):
          self.file_name = fname # File name from which we will read

          self.time = 0 # solution time
          self.number_of_pointst = 0 # number of data points

          self.profile_pos = np.array([0,0,0]) # position of the profile cut

          self.Y = np.array # Y coordinate
          self.u = np.array # Components of velocity across Y
          self.v = np.array
          self.w = np.array
```

So we have a simple class that will hold (u, v, w) components of a vector obtained at *time* and at a point with *Y* the coordinate across the cut. Lets now populate it with some made up data:

```
[3]: clist = []
      c = Case("file_0.dat") # some file we read from
      c.time = 0
      c.number_of_pointst = 1000
      c.profile_pos[0] = 5 # move at x=5
      c.profile_pos[1] = 10 # move at y=5
```

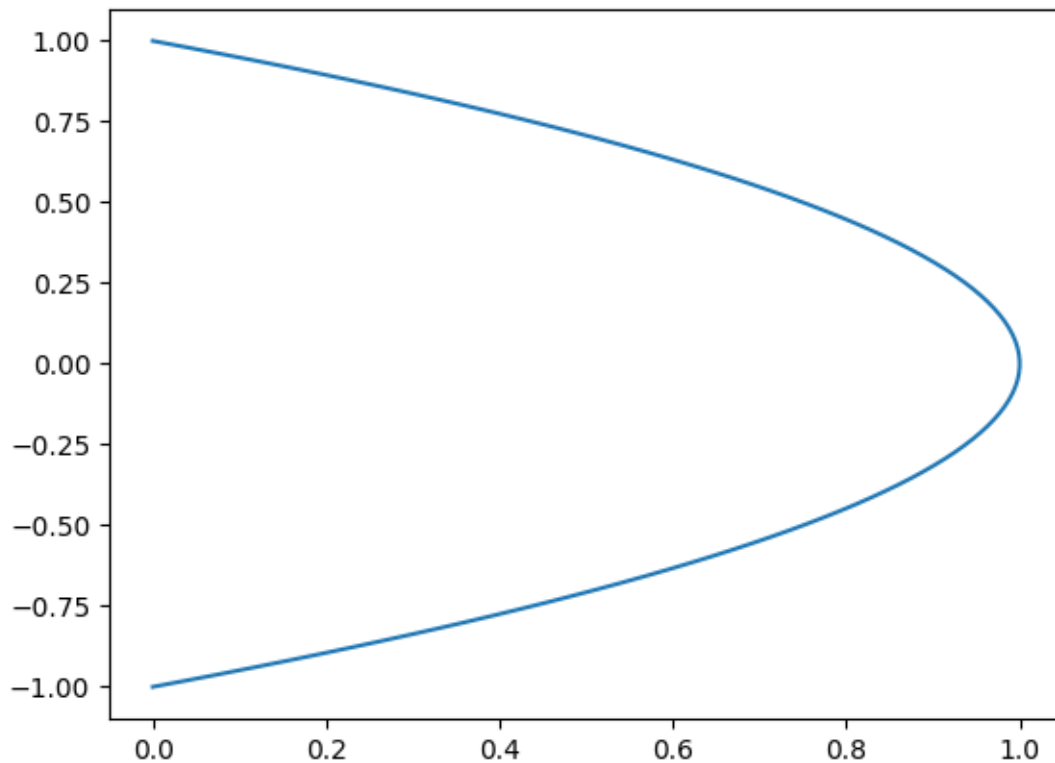
```

c.Y = np.linspace(-1,1,100)
# Normally u,v,w are read from file, here made up
c.u = 1-c.Y**2
c.v = c.Y * 0
c.w = c.Y * 0
clist.append(c) # add to a list

```

```
[4]: plt.plot(c.u, c.Y)
```

```
[4]: [<matplotlib.lines.Line2D at 0x7f8130e6da80>]
```



Let have a second one:

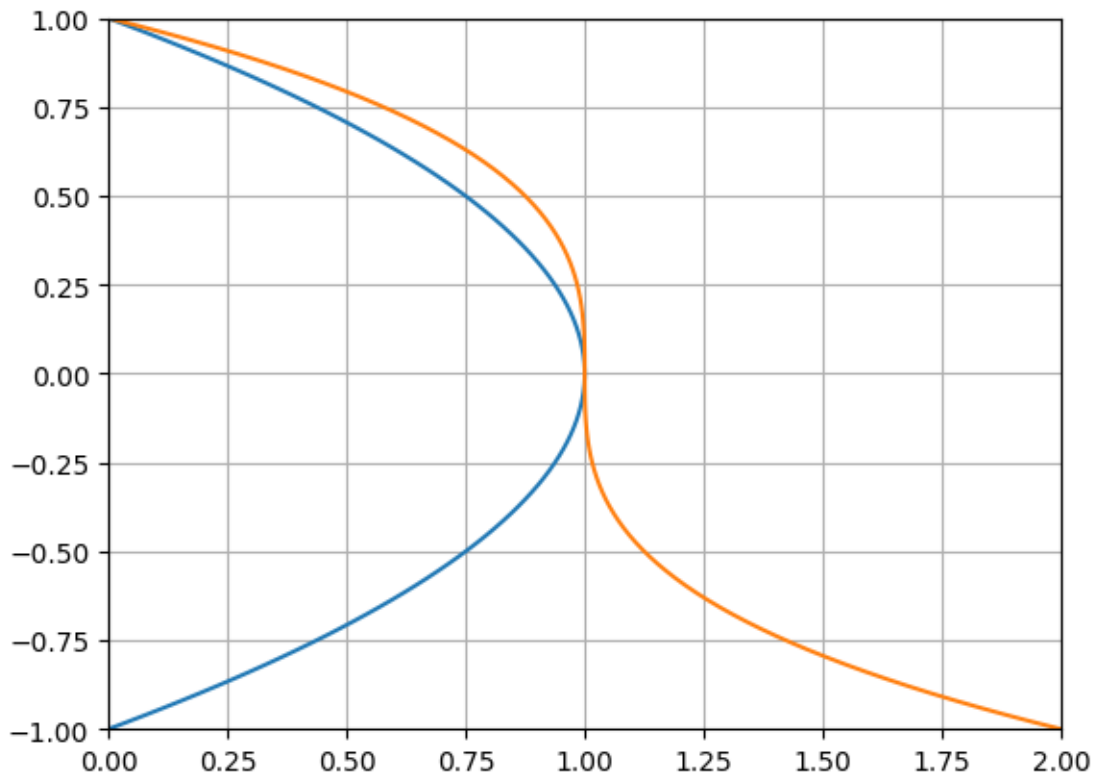
```

[5]: c = Case("filw_1.dat") # some file we read from
c.time = 1
c.number_of_pointst = 1000
c.profile_pos[0] = 5 # move at x=5
c.profile_pos[1] = 10 # move at y=5
c.Y = np.linspace(-1,1,100)
# Normally u,v,w are read from file, here made up
c.u = 1-c.Y**3
c.v = c.Y * 0

```

```
c.w = c.Y * 0
clist.append(c) # add to a list
```

```
[6]: for c in clist:
      plt.plot(c.u, c.Y)
plt.xlim(0,2)
plt.ylim(-1,1)
plt.grid()
```



usually there will be more data, the structure might be more complex, or some additional process will be used to do something with the data etc.

For data storage we will use the **msgpack**, which is a serialization format for use in data exchange across different languages, but not only. Here is the project:

<https://github.com/msgpack/msgpack-python>

and its API documentation:

<https://msgpack-python.readthedocs.io/en/latest/api.html>

In short, hands on How to Use we will require an instruction on how to handle data. How to encode and pack it and how to unpack and decode it. So we need to add encoding as well as decoding functions. Those function will accept an object of our data storage class and inform **msgpack** what to do with it. The whole process is in fact quite simple. We just do:

```
[7]: import msgpack
```

To import an appropriate module, and:

```
[8]: type(clist[0]) == int
```

```
[8]: False
```

```
[9]: """Encoding function"""
def enc_full(obj):
    if type(obj) == Case: # make sure obj is of some type
        code = 1 # A flag determining the type
        # pack and return
        return msgpack.ExtType(code, msgpack.packb([obj.file_name, obj.time, obj.
↪number_of_pointst,
                                                    obj.profile_pos.tostring(),
                                                    obj.Y.tostring(),
                                                    obj.u.tostring(), obj.v.tostring(),
↪obj.w.tostring()])))
        raise TypeError # Rise an error if type not handled
```

And we have the encoding part. Now, all simple data types go as they are (file_name, time ...) and Numpy arrays need to be converted to strings. The `packb` simply packs an object into an appropriate format. One comment here is that we are not packing the `Case` object itself, but rather a list (note `[ ]`) of attributes such object has. The `code` identifies the stored type, since we could have more than one class to deal with the `code` would let us distinguish recovered data. We can now store this data to a file, simply with:

```
[10]: msgpack.dump(clist, open( 'aaa.dat', "wb" ), default=enc_full)
```

```
/tmp/ipykernel_2203486/1148143230.py:7: DeprecationWarning: tostring() is
deprecated. Use tobytes() instead.
```

```
    obj.profile_pos.tostring(),
/tmp/ipykernel_2203486/1148143230.py:8: DeprecationWarning: tostring() is
deprecated. Use tobytes() instead.
```

```
    obj.Y.tostring(),
/tmp/ipykernel_2203486/1148143230.py:9: DeprecationWarning: tostring() is
deprecated. Use tobytes() instead.
```

```
    obj.u.tostring(), obj.v.tostring(), obj.w.tostring()])))
```

The `"wb"` lets us store this data in binary format. We will need to recover the data appropriately with `"rb"`. See if the file was created, and look inside.

Now restart the Python interpreter and try to recover the data we put on disk. The `Case` class still needs to be defined, so in case you restarted the interpreter, do run the appropriate cell.

To decode we will need a decoding function. It will contain instructions on how to rebuild objects from storage, create an object, feed it with data and return:

```
[11]: import msgpack
```

```
[12]: def dnc(code, data):
    if code == 1: # a flag determining the type
        # call the unpack on the data passed in here
        file_name, time, num_pts, pos, Y, u, v, w = msgpack.unpackb(data)

        # make the object
        c = Case(file_name)

        # feed it with data
        c.time = time
        c.number_of_pointst = num_pts
        # Note that numpy arrays need to be handled in a special way
        c.profile_pos = np.frombuffer(pos)
        c.Y = np.frombuffer(Y)
        c.u = np.frombuffer(u)
        c.v = np.frombuffer(v)
        c.w = np.frombuffer(w)

        # return the object outside
        return c
    # in case the type is either a build in or other
    return msgpack.ExtType(code, data)
```

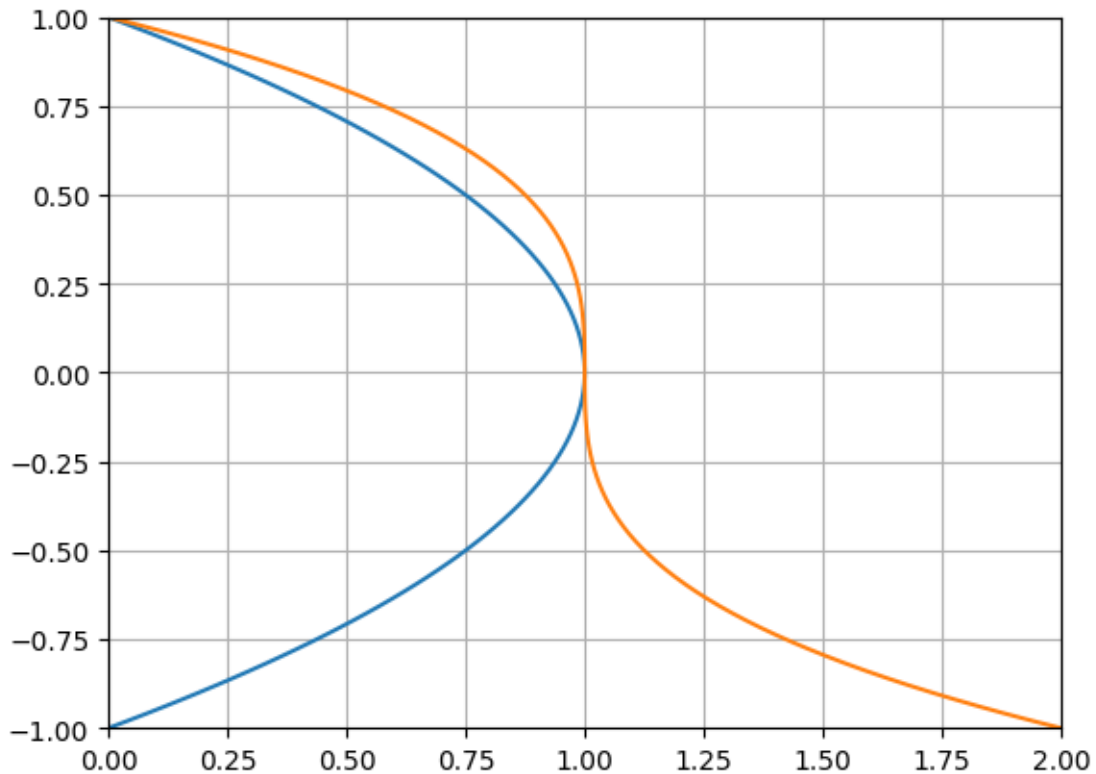
We have just one class, and its code is simply 1. `dnc()` will be passed this `code` as well as the binary data, which will be unpacked and returned as a tuple. It is up to us to create a new object and populate it with data and then to return it.

Lifting data from storage is now simple. Note that we need to pass the decoding function to be used by `load()`, the file name and "rb" since we read binary data:

```
[13]: recovered_list = msgpack.load(open( 'aaa.dat', "rb" ), ext_hook=dnc)
```

Now a copy of our data should be in `recovered_list`:

```
[14]: for c in recovered_list:
    plt.plot(c.u, c.Y)
plt.xlim(0,2)
plt.ylim(-1,1)
plt.grid()
```



1.1 Working with API

Various application provide some form of interface to interact with it. Python API becomes more and more a standard. How to connect to, how to use and to what it might be beneficial is very much problem dependant. For this I decided to show you connection to the Open Source visualization package ParaView. I use it to visualize CFD data, but it also used in other fields of engineering.

So the first step is to get your hands on the API, it will come as a Python module, it might need to be compiled, or be available as a software package. Here, I have the module compiled to a library placed in `/opt/paraview/lib/python3.8/`. Also I know I will need some of the Numpy functionality to process data:

```
[15]: import sys
sys.path.append('/opt/paraview/lib/python3.8/site-packages/')
sys.path.append('/opt/paraview/lib/python3.8/')
sys.path.append('/opt/paraview/lib/')

from paraview.simple import *
from paraview.numeric import fromvtkarray

import numpy as np
```

```

from vtk.util.numpy_support import vtk_to_numpy
from vtk.numpy_interface import dataset_adapter as dsa
from vtk.numpy_interface import algorithms as algs

import matplotlib.pyplot as plt

```

```

-----
ModuleNotFoundError                                Traceback (most recent call last)
Cell In [15], line 6
      3 sys.path.append('/opt/paraview/lib/python3.8/')
      4 sys.path.append('/opt/paraview/lib/')
----> 6 from paraview.simple import *
      7 from paraview.numeric import fromvtkarray
      9 import numpy as np

ModuleNotFoundError: No module named 'paraview'

```

What can be done with the API is very much up to the providing application and the level of documentation. In the case of Open Source project documentation is often lacking. This is the case with ParaView (was some time ago, could be better now). In what follows I will load a flow field snapshot and plot velocity profiles at selected positions:

1. Read the data

```

[16]: geom = VisItTecplotBinaryReader(FileName=['geom.plt'])
      geom.PointArrayStatus = ['u', 'v', 'w']

```

```

-----
NameError                                Traceback (most recent call last)
Cell In [16], line 1
----> 1 geom = VisItTecplotBinaryReader(FileName=['geom.plt'])
      2 geom.PointArrayStatus = ['u', 'v', 'w']

NameError: name 'VisItTecplotBinaryReader' is not defined

```

```

[17]: plot_over_line = PlotOverLine(Input=geom, Source='High Resolution Line Source')
      plot_over_line.Source.Point1 = [0.0, -1.0, 12.0]
      plot_over_line.Source.Point2 = [0.0, 1.0, 12.0]

```

```

-----
NameError                                Traceback (most recent call last)
Cell In [17], line 1
----> 1 plot_over_line = PlotOverLine(Input=geom, Source='High Resolution Line_
↳Source')
      2 plot_over_line.Source.Point1 = [0.0, -1.0, 12.0]
      3 plot_over_line.Source.Point2 = [0.0, 1.0, 12.0]

```

```
NameError: name 'PlotOverLine' is not defined
```

```
[18]: pl = servermanager.Fetch(plot_over_line)

pos = fromvtkarray(pl.GetPoints().GetData())
valsU = fromvtkarray(pl.GetPointData().GetScalars("u"))
valsU = np.hstack([val for val in valsU])
valsV = fromvtkarray(pl.GetPointData().GetScalars("v"))
valsV = np.hstack([val for val in valsV])
valsW = fromvtkarray(pl.GetPointData().GetScalars("w"))
valsW = np.hstack([val for val in valsW])
```

```
-----
NameError                                Traceback (most recent call last)
Cell In [18], line 1
----> 1 pl = servermanager.Fetch(plot_over_line)
      3 pos = fromvtkarray(pl.GetPoints().GetData())
      4 valsU = fromvtkarray(pl.GetPointData().GetScalars("u"))

NameError: name 'servermanager' is not defined
```

And here we have velocity components and positions, just what we need to make a nice plot. One of the things that one may, or not learn from documentation is that `pos` contains a transposed coordinates:

```
[19]: plt.plot(pos.T[1], valsU)
      plt.plot(pos.T[1], valsV)
      plt.grid()
```

```
-----
NameError                                Traceback (most recent call last)
Cell In [19], line 1
----> 1 plt.plot(pos.T[1], valsU)
      2 plt.plot(pos.T[1], valsV)
      3 plt.grid()

NameError: name 'pos' is not defined
```

```
[20]: plt.plot(pos.T[1], valsW)
      plt.grid()
```

```
-----
NameError                                Traceback (most recent call last)
Cell In [20], line 1
----> 1 plt.plot(pos.T[1], valsW)
```

```
2 plt.grid()
```

```
NameError: name 'pos' is not defined
```

We can modify the direction data is taken from:

```
[21]: plot_over_line.Source.Point1 = [0.0, 0.0, 0.0]
      plot_over_line.Source.Point2 = [0.0, 0.0, 25.13]
```

```
-----
NameError                                Traceback (most recent call last)
Cell In [21], line 1
----> 1 plot_over_line.Source.Point1 = [0.0, 0.0, 0.0]
      2 plot_over_line.Source.Point2 = [0.0, 0.0, 25.13]

NameError: name 'plot_over_line' is not defined
```

```
[22]: pl = servermanager.Fetch(plot_over_line)

pos = fromvtkarray(pl.GetPoints().GetData())
valsU = fromvtkarray(pl.GetPointData().GetScalars("u"))
valsU = np.hstack([val for val in valsU])
valsV = fromvtkarray(pl.GetPointData().GetScalars("v"))
valsV = np.hstack([val for val in valsV])
valsW = fromvtkarray(pl.GetPointData().GetScalars("w"))
valsW = np.hstack([val for val in valsW])
```

```
-----
NameError                                Traceback (most recent call last)
Cell In [22], line 1
----> 1 pl = servermanager.Fetch(plot_over_line)
      3 pos = fromvtkarray(pl.GetPoints().GetData())
      4 valsU = fromvtkarray(pl.GetPointData().GetScalars("u"))

NameError: name 'servermanager' is not defined
```

Now plotting along z coordinate:

```
[23]: plt.plot(pos.T[2], valsW)
      plt.grid()
```

```
-----
NameError                                Traceback (most recent call last)
Cell In [23], line 1
----> 1 plt.plot(pos.T[2], valsW)
      2 plt.grid()
```

```
NameError: name 'pos' is not defined
```

**Projekt „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca”
współfinansowany jest ze środków Unii Europejskiej w ramach Europejskiego
Funduszu Społecznego**

**Zadanie 35 pn. „Modyfikacja programu studiów II stopnia na kierunku
Robotyka i Automatyka” realizowane w ramach projektu „NERW 2 PW. Nauka
– Edukacja – Rozwój – Współpraca” współfinansowanego jest ze środków Unii
Europejskiej w ramach Europejskiego Funduszu Społecznego**

Lecture 6 - Writing Python libraries in C

Stanisław Gepner

Automation of design process with Python

Projekt „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca” współfinansowany jest ze środków Unii Europejskiej w ramach Europejskiego Funduszu Społecznego

Zadanie 35 pn. „Modyfikacja programu studiów II stopnia na kierunku Robotyka i Automatyka” realizowane w ramach projektu „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca” współfinansowanego jest ze środków Unii Europejskiej w ramach Europejskiego Funduszu Społecznego

1 Lecture 6

We will now learn how to write code and make it available as a Python module.

The ability to delegate certain tasks, or use external, already existing (C/C++) libraries to do work is very useful. The goal here is not so much to write meaningful code, but rather to present to you the possibilities that Python / C binding might give you. Essentially we will look at ways to call functions and pass data to C/C++ from Python.

Why would we want to do this: * We have a working C/C++ code we want to use. * We need the efficiency of execution of the compiled C/C++ code. * We want to perform tasks using multiple-threads, and Python does not allow us to do that, but our underlying C code might.

1.1 Passing data to and from

The way Python and C/C++ store data is different. C does it very efficiently, and Python less so. Consequently Python integer is not the same as C/C++ integer and needs to be “translated”. Most often this translation effort is up to you, meaning some code will need to be written, possibly on the both ends of the link. Still we are lucky as we get to use ready libraries for the most difficult tasks.

This “translating” is called data marshalling. For build-in, simple types (floats, integers etc.) marshalling might be almost easy, for composite variables (structures and classes) it might be tricky.

1.2 Passing-by-value or by-reference

You recall that when passing arguments to a function, by default C/C++ passes them as copies of values, so if you need data to be modified by the called function you need to pass the pointer to data, or in case of C++ a reference. Python in general will defend against giving you pointers, so writing functions that manipulate arguments passed to them might need extra work.

1.3 Problems with memory

In C/C++ memory allocated by the programmer must be freed by the programmer. Recall:

```
int *p = (int*)malloc(sizeof(int));  
...  
free(p);
```

In Python all issues of managing memory is handled for us, and freeing happens by the garbage collector. Consequently we might need to ensure that memory used in the C/C++ part is properly freed.

1.4 C or C++

In what follows we will be using `cffi` which is only compatible with C and has no support for C++. In case you need to work with C++ have a look at `ctypes`.

1.5 Some things we need

We need to add some libraries to be able to write C/C++ code that Python can use. This is easiest to do on Linux, and comes down to installing `python3-dev` package, or something similar,

depending on the distribution. (Windows might be more difficult, but look [here](#)). While `ctypes` is a standard in Python, we will use `cffi` to help us deal with communication and data marshalling.

We also need some code in C/C++ compiled to a library.

Note: See the example library attached to this lecture.

We start with a very simple function written in C:

```
double add(double a, double b)
{
    return a + b;
}
```

Import an initialize `cffi`:

```
[1]: from cffi import FFI
    ffi = FFI()
```

Point the library to be loaded. This is the one we just build:

```
[2]: lib = ffi.dlopen('examplelib/build/libc_example1_lib.so')
```

Just like a prototype of a function in C, we need to inform `cffi` on the data types that our function accepts and returns:

```
[3]: ffi.cdef('''
    double add(double a, double b);
    ''')
```

And that is it, we can use our C function:

```
[4]: print(lib.add(5.0, 9))
```

45.0

Note: What is returned from `lib.add()`?

```
[ ]: a = lib.add(5.0, 9)
    print(type(a))
```

Simple, right. Lets complicate this a bit with different types. While `int`, `float` etc. are already mapped (and simple to use), working with pointers, structures and arrays needs some additional effort, as those types have no direct counterparts on in Python. With `cffi` those types are mapped to a Python object `cdata` that contains them underneath and all access happens through this object. Also we need a way to communicate to the C part of our link the need to allocate appropriate resources. To this end `cffi` uses `new()`:

- `ffi.new('ctype string description', [initial values])`

`ffi.new()` creates a new `cdata` object wrapped around the underlying data. The data is defined by a string describing the `ctype`, either a pointer or an array, and initialization. If no initialization string is provided initial value is zeroed.

Regarding the necessity to free the memory. Contrary to C, memory allocated with `ffi.new()` is *owned* by the `ctype` object. Consequently when this object is destroyed (garbage collected) memory is freed, and no action is necessary on the Python part of the link. Note, however that in the case memory is allocated on the side of C it needs to be deallocated properly, and this is entirely up to you.

Let's try to create a C integer:

```
[ ]: # a = ffi.new('int') # this will not work
a = ffi.new('int *')
print(a)
```

Let us see what we can do with `a`. Watch out for dynamic typing! See below:

```
[ ]: a = 5
```

What is `a` now?

```
[ ]: print(a)
```

So we lost the original `a`, and data underneath. Note, not to do this.

Accessing the data: dereferencing of a pointer in C is done with `*p`, which is not available in Python, but not all is lost, we can still access the data with the `\[\]` operator! `p[0]`, so just treat this integer as an array! (everything is an array!). Note, that accessing out of the allocated memory will cause `IndexError`, meaning `cdata` keeps track of what can be accessed.

```
[ ]: a = ffi.new('int *')
# print(*a) # try me
print(a[0])
a[0] = 5
print(a[0])
# a[1] = 9 # try me
```

Extending to arrays is simple:

```
[ ]: n = 20
a = ffi.new('double['+str(n)+']') # a C style array full of zeros
# a = ffi.new('double[10]') # will work as well!
```

And `example2` library shows how to pass arrays to functions.

```
[ ]: lib = ffi.dlopen('examplelib/build/libc_example2_lib.so')
ffi.cdef('''
void fibonacci(double a[], int n);
''')
```

```
[ ]: lib.fibonacci(a, n)
```

```
[ ]: for i in a:
      print(i, end=", ")
```

Consider example 3:

```
[ ]: lib = ffi.dlopen('examplelib/build/libc_example3_lib.so')
      ffi.cdef('''
      void manipuator(double *pd);
      ''')
```

```
[ ]: a = ffi.new('double *')
      a[0] = 0
      print(a[0])
      lib.manipuator(a)
      print(a[0])
```

We might also work with composite types. In C those are obtained with structures. Consider example 4 with a simple structure representing a point:

```
typedef struct{
    double x;
    double y;
} Point;
```

and some functions to operate it:

```
[ ]: lib = ffi.dlopen('examplelib/build/libc_example4_lib.so')
      ffi.cdef('''
      typedef struct {
          double x;
          double y;
      } Point;

      Point make_Point(double x, double y);
      Point copy_Point(Point *p);
      void move_Point(Point *p, double x, double y);
      ''')
```

With `ffi.new()` we can make some points. Note initialization used this time:

```
[ ]: # Create Point
      point0 = ffi.new('Point*', {'y' : 1, 'x' : 10})
      point1 = ffi.new('Point*', [1, 8])
```

```
[ ]: print(point0, point0.x, point0.y)
      print(point1, point1.x, point1.y)
```

We can use a function that returns a Point. Note the type of the returned variable:

```
[ ]: a = lib.make_Point(10, 10)
      print(a, a.x, a.y)
```

There is a difference as to what `point0` and `a` are! One is a pointer wrapped in a `ctype` and the other the value (also wrapped)! This causes that we should consider how we design interfaces on both sides of the language barrier!

Some consequences: In C access to structure data is done either with a dot `.` or an arrow `->`. in Python access is possible only with `a ..`

While obtaining an address in Python is rather hard, in the case of `cdata` we might require it, for example to pass addresses of variable (such as one stored by `a`) to and from C functions. This is done with `ffi.addressof(a)`. Consider `copy_Point()`. We might try first this:

```
[ ]: print(a, a.x, a.y)
      b = lib.copy_Point(a)
      print(a, a.x, a.y)
      print(b, a.x, a.y)
```

And fail, since `copy_Point()` needs a pointer, not a value. To get the address:

```
[ ]: print(ffi.addressof(a))
```

```
[ ]: b = ffi.addressof(a)
      c = lib.copy_Point(b)
      print(b, b.x, b.y)
      print(ffi.addressof(c), c.x, c.y)
```

`b` and `c` are both holding a point `Point`:

```
[ ]: print(b, b.x, b.y)
      print(ffi.addressof(c), c.x, c.y)
```

Finally, we can manipulate the point with `move_Point()`:

```
[ ]: print(point0, point0.x, point0.y)
      lib.move_Point(point0, 4, 7)
      print(point0, point0.x, point0.y)
```

```
[ ]: lib.move_Point(ffi.addressof(c), 4, 7)
      print(c, c.x, c.y)
```

We will conclude by showing how to use `numpy.array` collection together with a C developed library. To this end we will generate a Mandlbrot fractal (the famous one):

```
[5]: from cffi import FFI
      ffi = FFI()

      lib = ffi.dlopen('examplelib/build/libfinal.so')
```

```
ffi.cdef('''
double add(double a, double b);
void fibonacci(double a[], int n);
void manipulator(double *pd);
typedef struct {
    double x;
    double y;
} Point;
Point make_Point(double x, double y);
Point copy_Point(Point *p);
void move_Point(Point *p, double x, double y);

void mandelbrot(int size, int iterations, int *col);
''')
# We show that here we imported all, but we will use only the mandelbrot()
```

```
[6]: import numpy as np

size = 500
iterations = 1000

# create a size x size array of integers
A = np.empty((size, size), dtype=np.int32)
```

NumPy provides a `ctypes` object to allow for passing array data to and from a `ctypes` module:

```
[7]: print(type(A))
      print(type(A.ctypes))
      print(type(A.ctypes.data))
      print(type(A.ctypes.shape))
```

```
<class 'numpy.ndarray'>
<class 'numpy.core._internal._ctypes'>
<class 'int'>
<class 'numpy.core._internal.c_long_Array_2'>
```

To use with `mandelbrot()` function we need to cast `A.ctypes.data` to `int*`. For this `ffi` offers `cast()`:

```
[8]: print(ffi.cast("int *", A.ctypes.data))
```

```
<cdata 'int *' 0x7f2b85b3f010>
```

Note: code below comes from <https://ipython-books.github.io/54-wrapping-a-c-library-in-python-with-ctypes/>, a source which I used preparing this lecture.

Let's start with what is the Mandelbrot Set. Consider the following iterative formula:

$$z_{n+1} = z_n^2 + c,$$

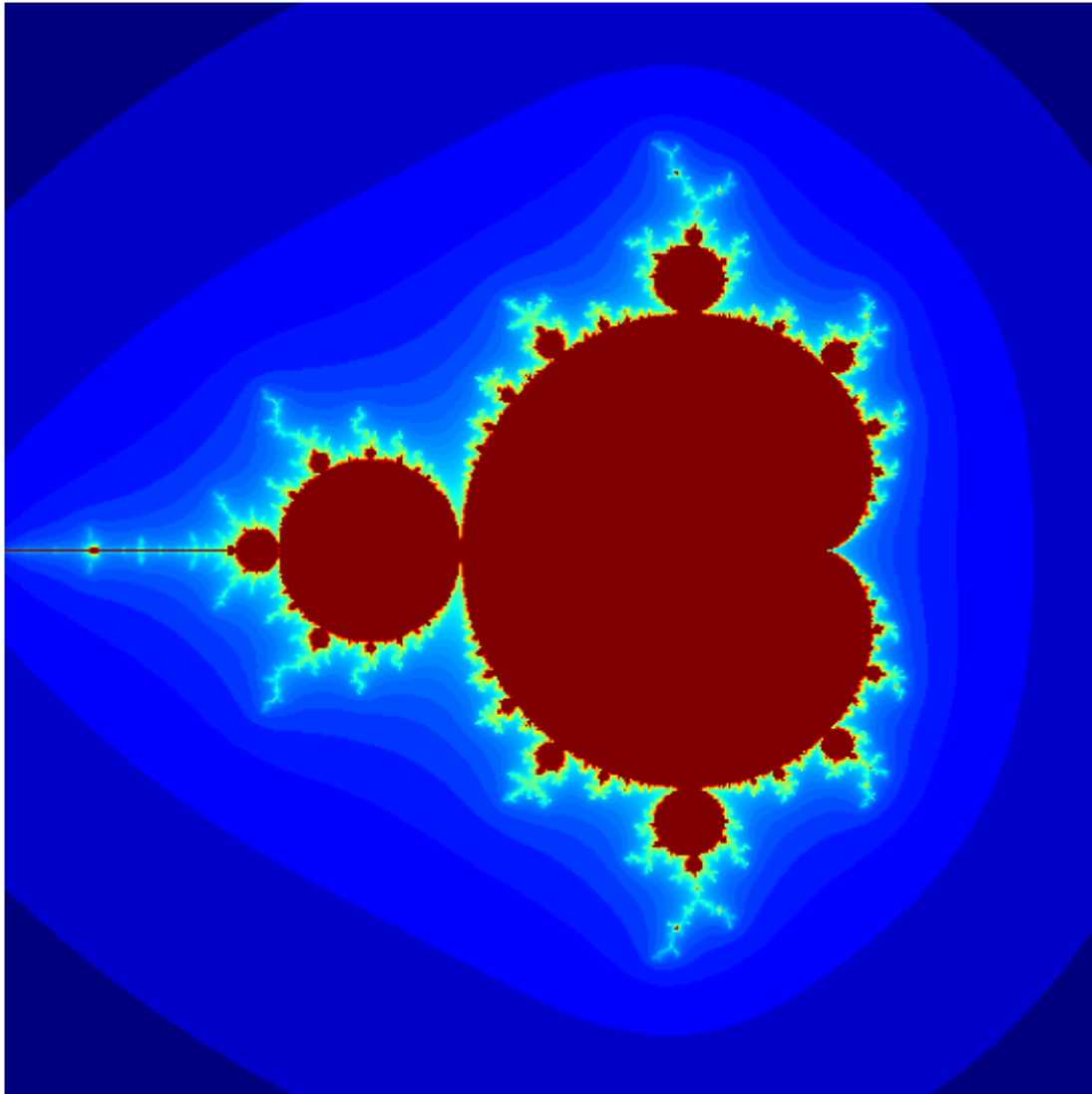
with z_n and c complex numbers. Now the Mandelbrot Set is the set of complex values c such that values of z , when starting with $z_0 = 0$ remains bounded.

Example: Check what z is for $c = 1$ and -1 .

Knowing what the Set is we can devise a simple, and naive algorithm to decide if points lie in or out of the set, the naive “escape time”. 1. Go through the (x,y) points of the complex plane, iterating for some number of iterations. 2. Check if the calculated value is larger than e.g. 2 and record the number of iterations. 3. Done

```
[9]: # mandelbrot() function fills the array with data  
lib.mandelbrot(size, iterations, ffi.cast("int *", A.ctypes.data))
```

```
[10]: import matplotlib.pyplot as plt  
%matplotlib inline  
  
fig, ax = plt.subplots(1, 1, figsize=(10, 10))  
  
ax.imshow(np.log(A), cmap=plt.cm.jet)  
  
ax.set_axis_off()
```



Now let's have a mandelbrot function in Python

```
[11]: def mandelbrot(size, iterations, col):  
    for i in range(0,size):  
        cy = -1.5 + i / size * 3.0  
        for j in range(0,size):  
            cx = -2.0 + j / size * 3.0  
            z0 = 0.0  
            z1 = 0.0  
            for n in range(0, iterations):  
                z0_2 = z0 * z0  
                z1_2 = z1 * z1  
                if z0_2 + z1_2 <= 100:
```

```
        z0_tmp = z0_2 - z1_2 + cx
        z1 = 2 * z0 * z1 + cy
        z0 = z0_tmp
        col[i,j] = n;
    else:
        break
```

```
[12]: size = 500
      iterations = 1000

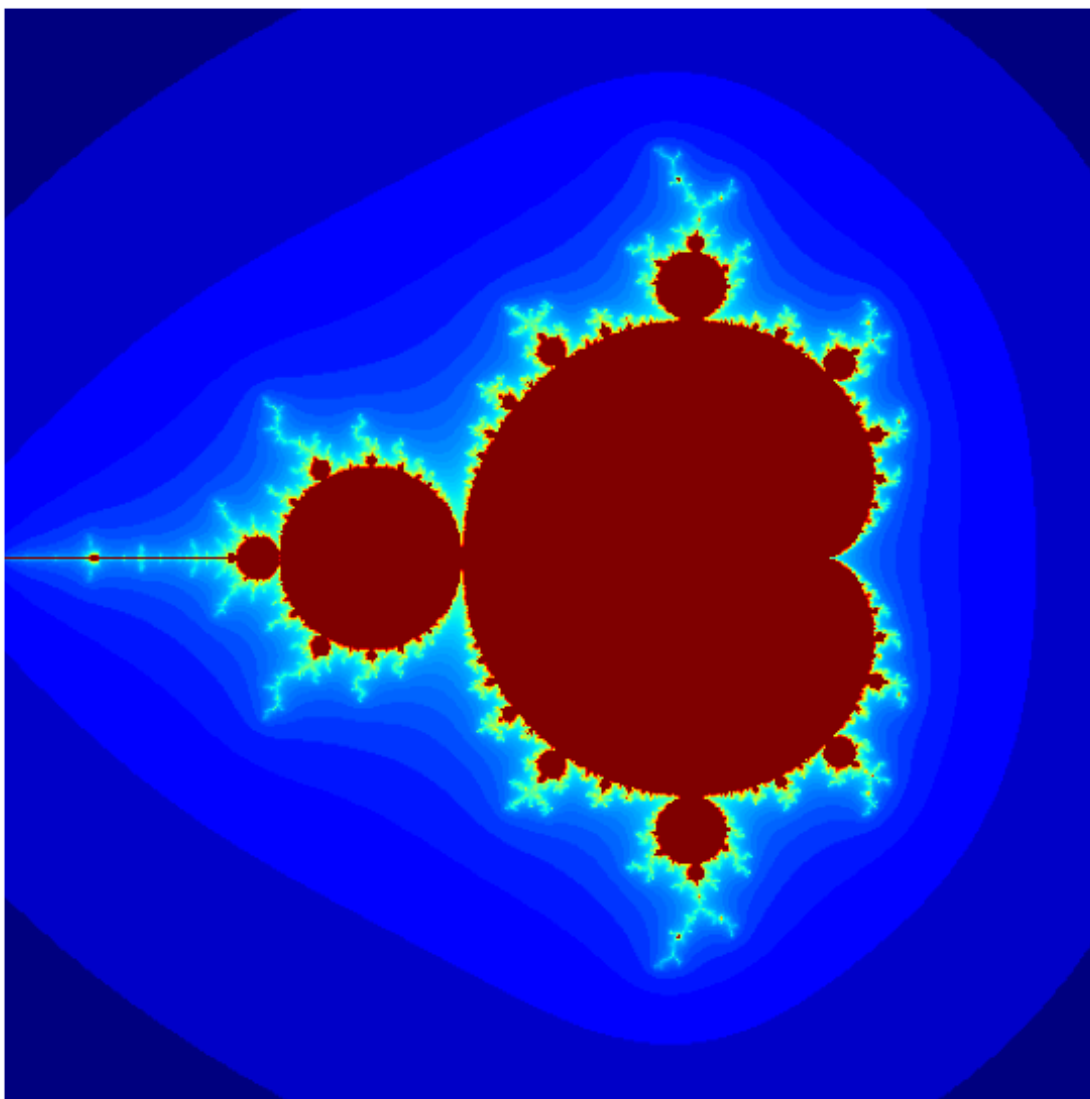
      # create a size x size array of integers
      B = np.empty((size, size), dtype=np.int32)
```

```
[13]: mandelbrot(size, iterations, B)
```

```
[12]: fig, ax = plt.subplots(1, 1, figsize=(10, 10))

      ax.imshow(np.log(B), cmap=plt.cm.jet)

      ax.set_axis_off()
```



```
[13]: import time
```

```
[14]: a = time.time()  
lib.mandelbrot(size, iterations, ffi.cast("int *", A.ctypes.data))  
b = time.time()  
print(b-a)
```

0.33275389671325684

```
[15]: a = time.time()  
mandelbrot(size, iterations, B)  
b = time.time()  
print(b-a)
```

11.339727401733398

Coding Time!: Examine the Mandelbrot set function and develop something similar for the Julia Set:

That is, find such points $p \in C$ that the following series:

$$\begin{cases} z_0 = p \\ z_{n+1} = z_n^2 + c \end{cases}$$

does not diverge to ∞ . c should be a parameter.

```
[16]: from cffi import FFI
      ffi = FFI()

      lib = ffi.dlopen('examplelib/build/libjulia.so')
      ffi.cdef('''
      void julia(int size, int iterations, int *col, double dx, double dy);
      ''')
      # We show that here we imported all, but we will use only the mandelbrot()
```

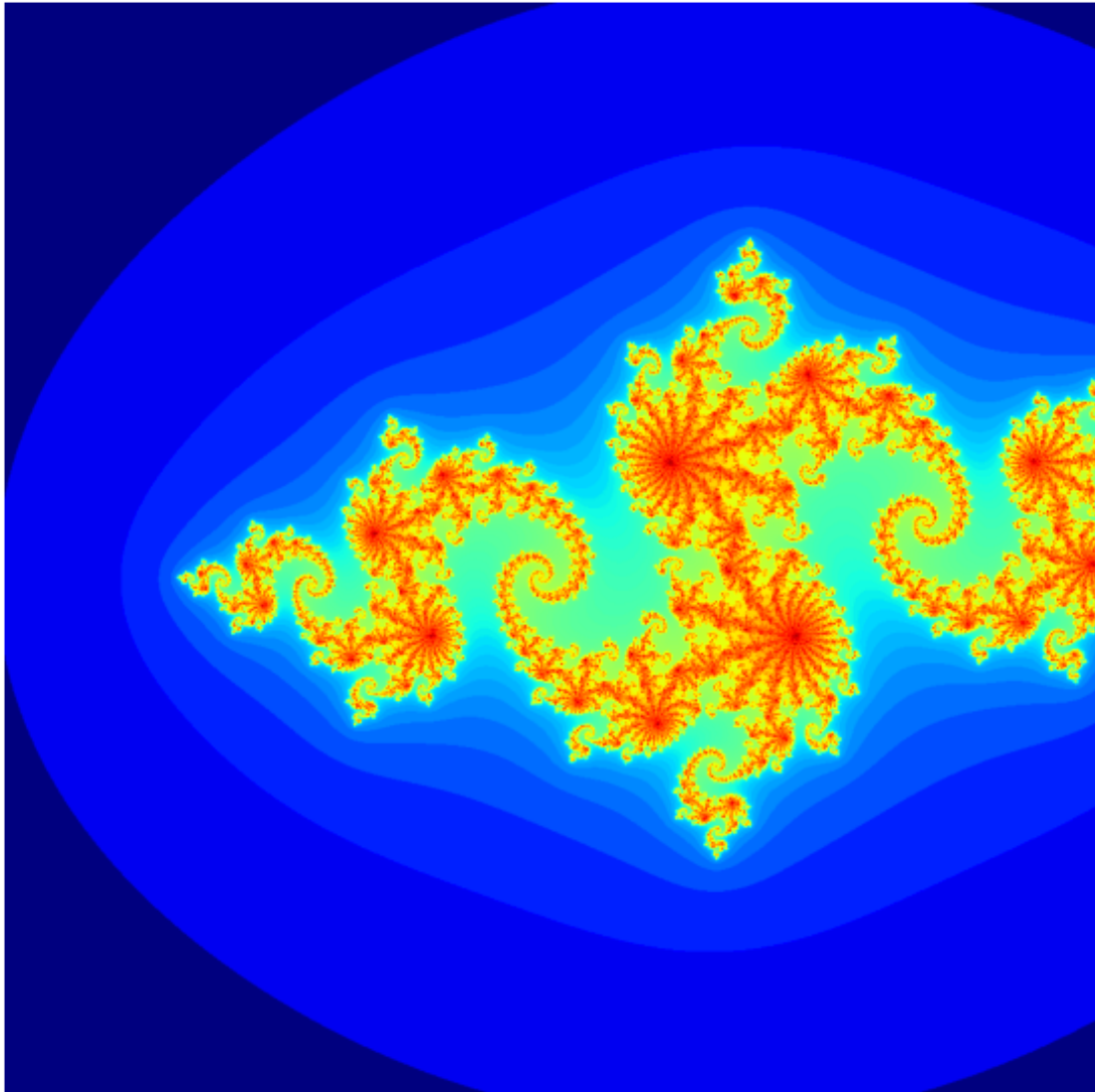
```
[17]: import numpy as np

      size = 1000
      iterations = 1000
      dx = -0.8
      dy = 0.156

      # create a size x size array of integers
      A = np.empty((size, size), dtype=np.int32)
```

```
[18]: # mandelbrot() function fills the array with data
      lib.julia(size, iterations, ffi.cast("int *", A.ctypes.data), dx, dy)
```

```
[19]: import matplotlib.pyplot as plt
      %matplotlib inline
      fig, ax = plt.subplots(1, 1, figsize=(10, 10))
      ax.imshow(np.log(A), cmap=plt.cm.jet)
      ax.set_axis_off()
```



2 Appendix

Here sources of programs used for this lecture are provided for reference.

CMakeLists.txt:

```
[1]: cat ./examplelib/CMakeLists.txt
```

```
cmake_minimum_required(VERSION 3.0)
```

```
project(cexamplelib)
```

```
add_library(c_example1_lib SHARED example1.c)
```

```

add_library(c_example2_lib SHARED example2.c)
add_library(c_example3_lib SHARED example3.c)
add_library(c_example4_lib SHARED example4.c)

add_library(final SHARED example1.c example2.c example3.c example4.c
mandelbrot.c)

add_library(julia SHARED julia.c)
example1.c, example2.c, example3.c, example4.c:

```

```
[2]: cat ./examplelib/example1.c
```

```

double add(double a, double b)
{
    return a * b;
}

```

```
[4]: cat ./examplelib/example2.c
```

```

void fibonaci(double a[], int n)
{
    int i=2;
    if(n <= 0) return;
    a[0] = 0;
    if(n == 1) return;
    a[1] = 1;
    for(; i<n; ++i)
    {
        a[i] = a[i-1] + a[i-2];
    }
    return;
}

```

```
[5]: cat ./examplelib/example3.c
```

```

#include <stdlib.h>
#include <time.h>

void manipuator(double *pd)
{
    srand(time(NULL));
    *pd = (double)rand() / RAND_MAX;
}

```

```
[6]: cat ./examplelib/example4.c
```

```
typedef struct{
    double x;
    double y;
} Point;

Point make_Point(double x, double y)
{
    Point rp;
    rp.x = x;
    rp.y = y;
    return rp;
}

Point copy_Point(Point *p)
{
    Point rp;
    rp.x = p->x;
    rp.y = p->y;
    return rp;
}

void move_Point(Point *p, double x, double y)
{
    p->x += x;
    p->y += y;
}
```

julia.c and mandelbrot.c:

```
[7]: cat ./examplelib/julia.c
```

```
void julia(int size, int iterations, int *col, double dx, double dy)
{
    // Variable declarations.
    int i, j, n, index;
    double cx, cy;
    double z0, z1, z0_tmp, z0_2, z1_2;

    // Loop within the grid.
    for (i = 0; i < size; i++)
    {
        cy = -1.5 + (double)i / size * 3;
        for (j = 0; j < size; j++)
        {
```

```

        // We initialize the loop of the system.
        cx = -2.0 + (double)j / size * 3;
        index = i * size + j;
        // Let's run the system.
        z0 = cx;
        z1 = cy;
        for (n = 0; n < iterations; n++)
        {
            z0_2 = z0 * z0;
            z1_2 = z1 * z1;
            if (z0_2 + z1_2 <= 100)
            {
                // Update the system.
                z0_tmp = z0_2 - z1_2 + dx;
                z1 = 2 * z0 * z1 + dy;
                z0 = z0_tmp;
                col[index] = n;
            }
            else
            {
                break;
            }
        }
    }
}

```

```
[8]: cat ./examplelib/mandelbrot.c
```

```

#include "stdio.h"
#include "stdlib.h"

/*
Code from: https://ipython-books.github.io/54-wrapping-a-c-library-in-python-with-ctypes/
*/

/*
A version of the escape time algorithm
*/
void mandelbrot(int size, int iterations, int *col)
{
    // Variable declarations.
    int i, j, n, index;
    double cx, cy;
    double z0, z1, z0_tmp, z0_2, z1_2;

    // Loop within the grid.

```

```

for (i = 0; i < size; i++)
{
    cy = -1.5 + (double)i / size * 3;
    for (j = 0; j < size; j++)
    {
        // We initialize the loop of the system.
        cx = -2.0 + (double)j / size * 3;
        index = i * size + j;
        // Let's run the system.
        z0 = 0.0;
        z1 = 0.0;
        for (n = 0; n < iterations; n++)
        {
            z0_2 = z0 * z0;
            z1_2 = z1 * z1;
            if (z0_2 + z1_2 <= 100)
            {
                // Update the system.
                z0_tmp = z0_2 - z1_2 + cx;
                z1 = 2 * z0 * z1 + cy;
                z0 = z0_tmp;
                col[index] = n;
            }
            else
            {
                break;
            }
        }
    }
}

```

[]:

**Projekt „NERW 2 PW. Nauka – Edukacja – Rozwój – Współpraca”
współfinansowany jest ze środków Unii Europejskiej w ramach Europejskiego
Funduszu Społecznego**

**Zadanie 35 pn. „Modyfikacja programu studiów II stopnia na kierunku
Robotyka i Automatyka” realizowane w ramach projektu „NERW 2 PW. Nauka
– Edukacja – Rozwój – Współpraca” współfinansowanego jest ze środków Unii
Europejskiej w ramach Europejskiego Funduszu Społecznego**